

STAR RX72N - rx_fec Library
1.0.0

Generated by Doxygen 1.16.1

1 Test List	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 libs/rx_fec/inc Directory Reference	9
5.2 libs Directory Reference	9
5.3 libs/rx_fec Directory Reference	10
5.4 libs/rx_fec/src Directory Reference	10
6 Data Structure Documentation	11
6.1 rx_fec_decoder_t Struct Reference	11
6.1.1 Detailed Description	11
6.1.2 Field Documentation	12
6.1.2.1 branch_table	12
6.1.2.2 initialized	12
6.1.2.3 new_path_metrics	12
6.1.2.4 path_metrics	12
6.1.2.5 survivors	12
6.1.2.6 survivors_len	12
7 File Documentation	13
7.1 libs/rx_fec/inc/rx_fec.h File Reference	13
7.1.1 Detailed Description	15
7.1.1.1 Features	15
7.1.1.2 Convolutional Code Theory	15
7.1.1.3 Protocol Integration	17
7.1.1.4 Performance Characteristics	17
7.1.1.5 Coding Gain	17
7.1.2 Data Structure Documentation	19
7.1.2.1 struct rx_fec_encoder_t	19
7.1.2.2 Rationale for Stateless Design	19
7.1.2.3 Memory Layout	20
7.1.2.4 struct rx_fec_decode_soft_params_t	21
7.1.2.5 struct rx_fec_decode_hard_params_t	22
7.1.3 Typedef Documentation	22
7.1.3.1 rx_soft_bit_t	22
7.1.3.2 Soft Bit Interpretation	23

7.1.3.3 Mathematical Model	23
7.1.3.4 Conversion from Hard Bits	23
7.1.3.5 Why Signed 8-bit?	23
7.1.4 Enumeration Type Documentation	24
7.1.4.1 rx_fec_buffer_limits_t	24
7.1.4.2 Size Calculations	25
7.1.4.3 rx_fec_params_t	26
7.1.4.4 Parameter Relationships	26
7.1.4.5 Generator Polynomial Selection	26
7.1.4.6 rx_soft_bit_limits_t	28
7.1.4.7 Range Justification	28
7.1.4.8 Why Not -128?	28
7.1.5 Function Documentation	30
7.1.5.1 rx_fec_decode_hard()	30
7.1.5.2 rx_fec_decode_soft()	32
7.1.5.3 rx_fec_decoder_deinit()	35
7.1.5.4 rx_fec_decoder_init()	35
7.1.5.5 rx_fec_encode()	36
7.1.5.6 Algorithm Steps	37
7.1.5.7 Mathematical Model	37
7.1.5.8 Output Length Formula	37
7.1.5.9 Performance Analysis	37
7.1.5.10 State Machine	38
7.1.5.11 rx_fec_encoded_len()	41
7.1.5.12 Formula Derivation	42
7.1.5.13 Example Calculations	42
7.1.5.14 Performance	42
7.1.5.15 rx_fec_encoder_deinit()	45
7.1.5.16 rx_fec_encoder_init()	47
7.1.5.17 rx_fec_hard_to_soft()	48
7.1.5.18 rx_fec_soft_to_hard()	49
7.2 rx_fec.h	49
7.3 libs/rx_fec/src/rx_fec.c File Reference	62
7.3.1 Detailed Description	64
7.3.1.1 Implementation Architecture	65
7.3.1.2 Algorithm Complexity	65
7.3.1.3 Viterbi Algorithm Overview	65
7.3.1.4 Branch Metric Calculation	66
7.3.1.5 Bit Ordering Convention	66
7.3.2 Macro Definition Documentation	67
7.3.2.1 RX_HAS_STDBIT	67
7.3.2.2 RX_HAS_STDCKDINT	67
7.3.3 Enumeration Type Documentation	67

7.3.3.1 rx_fec_impl_constants_t	67
7.3.3.2 rx_fec_output_index_t	68
7.3.4 Function Documentation	68
7.3.4.1 internal_branch_metric()	68
7.3.4.2 internal_encode_bit()	69
7.3.4.3 internal_encode_data_bytes()	70
7.3.4.4 internal_encode_tail_bits()	71
7.3.4.5 internal_fec_compute_encoded_bytes()	71
7.3.4.6 internal_get_bit()	73
7.3.4.7 internal_init_branch_table()	73
7.3.4.8 internal_parity()	74
7.3.4.9 internal_set_output_bit()	75
7.3.4.10 internal_validate_decode_params()	75
7.3.4.11 internal_viterbi_forward_pass()	77
7.3.4.12 internal_viterbi_process_symbol()	78
7.3.4.13 internal_viterbi_process_transition()	79
7.3.4.14 internal_viterbi_traceback()	80
7.3.4.15 internal_viterbi_traceback_validate()	81
7.3.4.16 rx_fec_decode_hard()	81
7.3.4.17 rx_fec_decode_soft()	83
7.3.4.18 rx_fec_decoder_deinit()	85
7.3.4.19 rx_fec_decoder_init()	86
7.3.4.20 rx_fec_encode()	87
7.3.4.21 rx_fec_encoded_len()	88
7.3.4.22 rx_fec_encoder_deinit()	89
7.3.4.23 rx_fec_encoder_init()	91
7.4 rx_fec.c	92

Chapter 1

Test List

File [rx_fec.h](#)

test_rx_fec.c contains unit tests for encoder and decoder

Global [rx_fec_encode](#) (const [rx_fec_encoder_t](#) *enc, const uint8_t *input, const uint32_t input_len, uint8_t *output, uint32_t *output_len)

test_rx_fec.c::test_encode_basic() Tests basic encoding

test_rx_fec.c::test_encode_tail_bits() Verifies tail bit termination

Chapter 2

Directory Hierarchy

2.1 Directories

inc	9
rx_fec.h	13
libs	9
rx_fec	10
inc	9
rx_fec.h	13
src	10
rx_fec.c	62
rx_fec	10
inc	9
rx_fec.h	13
src	10
rx_fec.c	62
src	10
rx_fec.c	62

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

rx_fec_decoder_t	
FEC decoder state	11

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

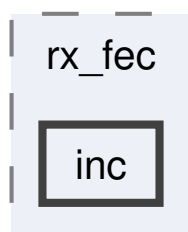
libs/rx_fec/inc/ rx_fec.h	
Forward Error Correction (FEC) Codec for RX72N	13
libs/rx_fec/src/ rx_fec.c	
Forward Error Correction (FEC) Codec Implementation	62

Chapter 5

Directory Documentation

5.1 libs/rx_fec/inc Directory Reference

Directory dependency graph for inc:

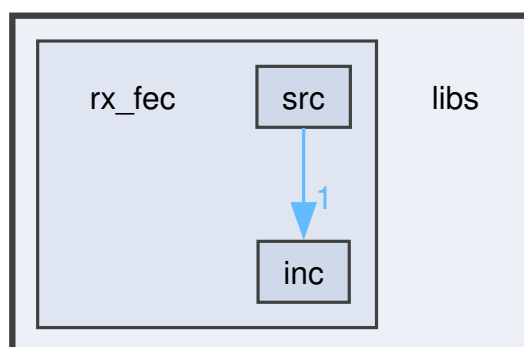


Files

- file [rx_fec.h](#)
Forward Error Correction (FEC) Codec for RX72N.

5.2 libs Directory Reference

Directory dependency graph for libs:

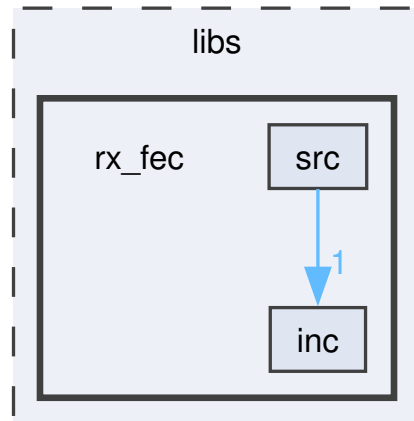


Directories

- directory [rx_fec](#)

5.3 libs/rx_fec Directory Reference

Directory dependency graph for rx_fec:

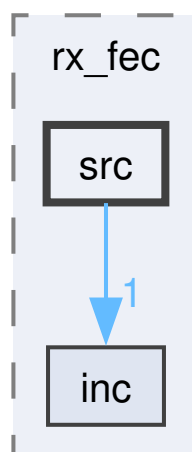


Directories

- directory [inc](#)
- directory [src](#)

5.4 libs/rx_fec/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_fec.c](#)
Forward Error Correction (FEC) Codec Implementation.

Chapter 6

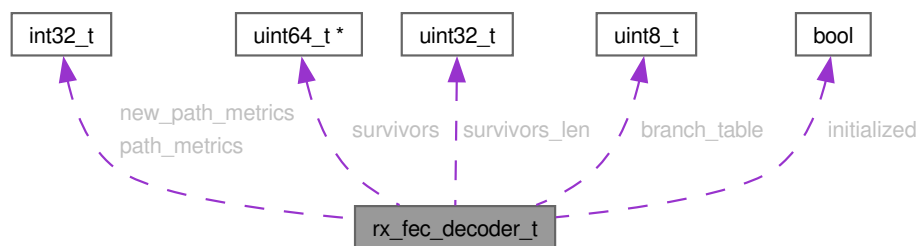
Data Structure Documentation

6.1 rx_fec_decoder_t Struct Reference

FEC decoder state.

```
#include <rx_fec.h>
```

Collaboration diagram for rx_fec_decoder_t:



Data Fields

- `int32_t path_metrics [k_fec_num_states]`
- `int32_t new_path_metrics [k_fec_num_states]`
- `uint64_t * survivors`
- `uint32_t survivors_len`
- `uint8_t branch_table [k_fec_num_states][k_fec_num_input_values][k_fec_num_outputs]`
- `bool initialized`

6.1.1 Detailed Description

FEC decoder state.

Contains all working buffers for Viterbi decoding. The survivors buffer must be provided by the caller for static allocation.

Definition at line 1003 of file `rx_fec.h`.

6.1.2 Field Documentation

6.1.2.1 branch_table

`uint8_t rx_fec_decoder_t::branch_table[k_fec_num_states][k_fec_num_input_values][k_fec_num_outputs]`

Precomputed outputs

Definition at line 1008 of file [rx_fec.h](#).

Referenced by [internal_viterbi_process_transition\(\)](#), and [rx_fec_decoder_init\(\)](#).

6.1.2.2 initialized

`bool rx_fec_decoder_t::initialized`

True if initialized

Definition at line 1010 of file [rx_fec.h](#).

Referenced by [internal_validate_decode_params\(\)](#), [rx_fec_decode_hard\(\)](#), [rx_fec_decoder_deinit\(\)](#), and [rx_fec_decoder_init\(\)](#).

6.1.2.3 new_path_metrics

`int32_t rx_fec_decoder_t::new_path_metrics[k_fec_num_states]`

Next path metrics

Definition at line 1005 of file [rx_fec.h](#).

Referenced by [internal_viterbi_process_symbol\(\)](#), and [internal_viterbi_process_transition\(\)](#).

6.1.2.4 path_metrics

`int32_t rx_fec_decoder_t::path_metrics[k_fec_num_states]`

Current path metrics

Definition at line 1004 of file [rx_fec.h](#).

Referenced by [internal_viterbi_forward_pass\(\)](#), [internal_viterbi_process_symbol\(\)](#), and [internal_viterbi_process_transition\(\)](#).

6.1.2.5 survivors

`uint64_t* rx_fec_decoder_t::survivors`

Survivor bits (caller-provided)

Definition at line 1006 of file [rx_fec.h](#).

Referenced by [internal_viterbi_process_symbol\(\)](#), [internal_viterbi_process_transition\(\)](#), [internal_viterbi_traceback_val\(\)](#), [rx_fec_decoder_deinit\(\)](#), and [rx_fec_decoder_init\(\)](#).

6.1.2.6 survivors_len

`uint32_t rx_fec_decoder_t::survivors_len`

Length of survivors buffer

Definition at line 1007 of file [rx_fec.h](#).

Referenced by [internal_validate_decode_params\(\)](#), [internal_viterbi_process_symbol\(\)](#), [internal_viterbi_traceback_val\(\)](#), [rx_fec_decoder_deinit\(\)](#), and [rx_fec_decoder_init\(\)](#).

The documentation for this struct was generated from the following file:

- [libs/rx_fec/inc/rx_fec.h](#)

Chapter 7

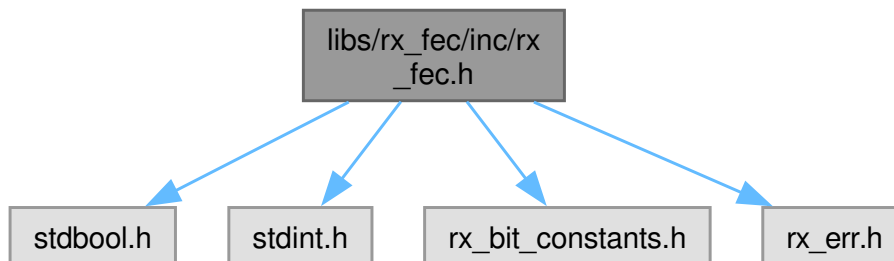
File Documentation

7.1 libs/rx_fec/inc/rx_fec.h File Reference

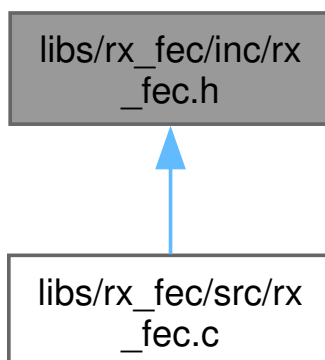
Forward Error Correction (FEC) Codec for RX72N.

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_bit_constants.h"
#include "rx_err.h"
```

Include dependency graph for rx_fec.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_fec_encoder_t](#)
FEC convolutional encoder state.
- struct [rx_fec_decoder_t](#)
FEC decoder state.
- struct [rx_fec_decode_soft_params_t](#)
Soft-decision decode parameters.
- struct [rx_fec_decode_hard_params_t](#)
Hard-decision decode parameters.

Typedefs

- typedef int8_t [rx_soft_bit_t](#)
Soft bit type for soft-decision decoding.

Enumerations

- enum [rx_fec_params_t](#) : uint8_t {
 [k_fec_constraint_length](#) = 7 , [k_fec_num_states](#) = 64 , [k_fec_tail_bits](#) = 6 , [k_fec_g1_octal](#) = 0171 ,
 [k_fec_g2_octal](#) = 0133 , [k_fec_traceback_depth](#) = 35 , [k_fec_num_input_values](#) = 2 ,
 [k_fec_num_outputs](#) = 2 }
FEC convolutional code parameters.
- enum [rx_soft_bit_limits_t](#) : int8_t { [k_soft_bit_max](#) = 127 , [k_soft_bit_min](#) = -127 ,
 [k_soft_bit_zero](#) = 0 }
Soft bit value range limits.
- enum [rx_fec_buffer_limits_t](#) : uint16_t { [k_fec_max_symbols](#) = 8200 , [k_fec_max_input_bytes](#) = (uint16_t)(([k_fec_max_symbols](#) - [k_fec_tail_bits](#)) / [k_rx_bits_per_byte](#)) }
Maximum buffer sizes for static FEC decoder allocation.

Functions

- rx_err_t [rx_fec_encoder_init](#) ([rx_fec_encoder_t](#) *enc)
Initialize FEC encoder.
- rx_err_t [rx_fec_encoder_deinit](#) ([rx_fec_encoder_t](#) *enc)
Deinitialize FEC encoder.
- rx_err_t [rx_fec_encode](#) (const [rx_fec_encoder_t](#) *enc, const uint8_t *input, uint32_t input_len, uint8_t *output, uint32_t *output_len)
Encode data using K=7, rate 1/2 convolutional code.
- uint32_t [rx_fec_encoded_len](#) (uint32_t input_len)
Calculate FEC encoded output length.
- rx_err_t [rx_fec_decoder_init](#) ([rx_fec_decoder_t](#) *dec, uint64_t *survivors_buf, uint32_t survivors_len)
Initialize FEC decoder.
- rx_err_t [rx_fec_decoder_deinit](#) ([rx_fec_decoder_t](#) *dec)
Deinitialize FEC decoder.
- rx_err_t [rx_fec_decode_soft](#) ([rx_fec_decoder_t](#) *dec, const [rx_fec_decode_soft_params_t](#) *params)
Decode soft bits using Viterbi algorithm.

- `rx_err_t rx_fec_decode_hard (rx_fec_decoder_t *dec, const rx_fec_decode_hard_params_t *params)`
Decode hard bits using Viterbi algorithm.
- `static rx_soft_bit_t rx_fec_hard_to_soft (uint8_t bit)`
Convert hard bit to soft bit.
- `static uint8_t rx_fec_soft_to_hard (rx_soft_bit_t soft)`
Convert soft bit to hard bit.

7.1.1 Detailed Description

Forward Error Correction (FEC) Codec for RX72N.

Implements NASA-standard K=7 constraint length convolutional encoder and soft-decision Viterbi decoder for reliable communication over noisy channels. This is a C port of the Go FEC implementation in star-gateway, ensuring bit-exact compatibility between RX72N firmware and RPi5 gateway.

7.1.1.1 Features

- Rate 1/2 Convolutional Encoding: 2 output bits per input bit
- Soft-Decision Viterbi Decoding: Uses soft bit reliability information
- Hard-Decision Fallback: Converts hard bits to soft for testing
- Static Memory Allocation: Zero dynamic allocation (NASA Rule 3)
- Tail-Biting Termination: Ensures encoder returns to zero state

7.1.1.2 Convolutional Code Theory

A convolutional code encodes input bits using a shift register and generator polynomials. The encoder maintains a state (shift register contents) and produces multiple output bits for each input bit.

7.1.1.2.1 Mathematical Model

For this K=7, rate 1/2 code:

- Constraint Length $K = 7$: Shift register holds 6 bits ($K-1$)
- Number of States: $2^{K-1} = 64$ states
- Code Rate: $R = \frac{1}{2}$ (1 input bit $\rightarrow$ 2 output bits)
- Generator Polynomials:
 - $G_1(D) = 1 + D^2 + D^3 + D^5 + D^6$ (octal 171, hex 0xF9)
 - $G_2(D) = 1 + D + D^2 + D^3 + D^6$ (octal 133, hex 0x5B)

7.1.1.2.2 Encoder Operation

Given shift register state $s = [s_5, s_4, s_3, s_2, s_1, s_0]$ and input bit u , the encoder produces output bits:

$$c_0 = u \oplus s_0 \oplus s_1 \oplus s_3 \oplus s_4 \oplus s_5 \quad (G_1)$$

$$c_1 = u \oplus s_0 \oplus s_1 \oplus s_2 \oplus s_5 \quad (G_2)$$

After outputting (c_0, c_1) , the state updates: $s \leftarrow [u, s_5, s_4, s_3, s_2, s_1]$ (right shift with u inserted)

7.1.1.2.3 Viterbi Decoder

The Viterbi algorithm finds the maximum likelihood path through the trellis of possible encoder states. It uses dynamic programming to compute path metrics incrementally.

7.1.1.2.3.1 Branch Metric

For received soft bits (r_0, r_1) and expected bits (e_0, e_1) , the correlation metric is:

$$\text{correlation} = r_0 \cdot e_0' + r_1 \cdot e_1'$$

where $e_i' = +127$ if $e_i = 1$, else $e_i' = -127$.

Branch metric (lower is better):

$$M_{\text{branch}} = 32768 - \text{correlation}$$

7.1.1.2.3.2 Path Metric Update

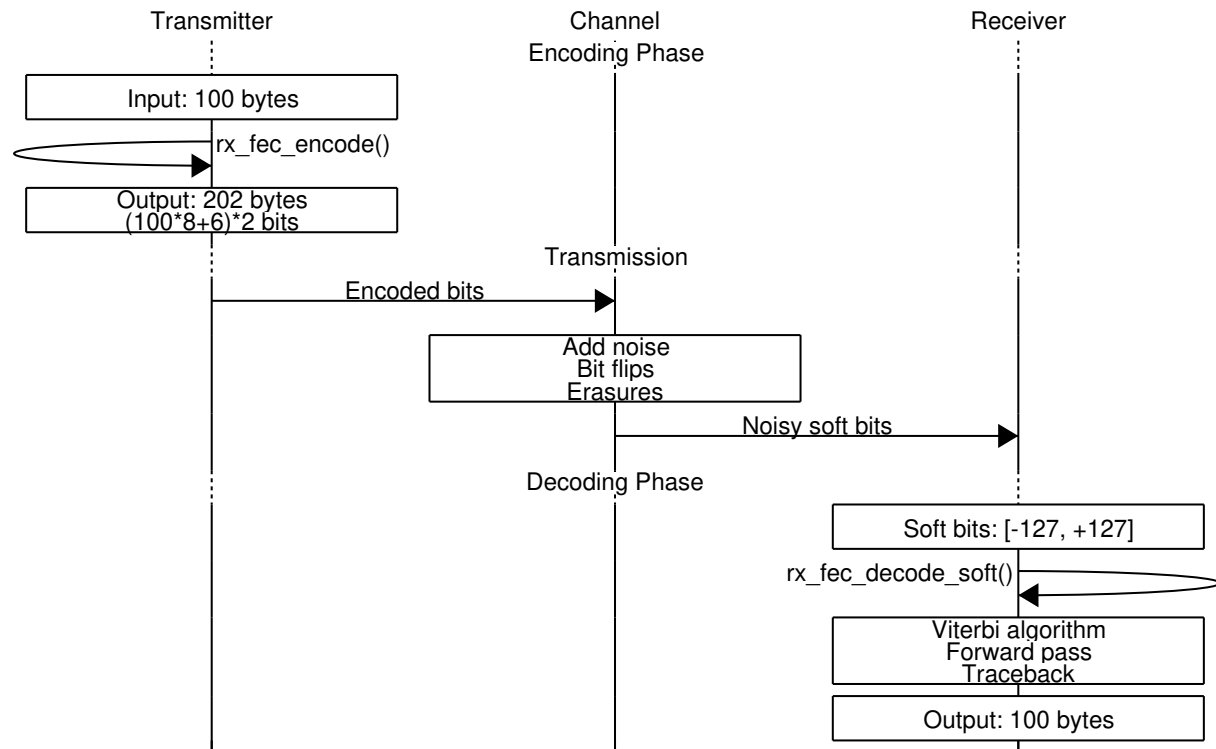
For each state s at time t :

$$M_s(t) = \min_{s'} [M_{s'}(t-1) + M_{\text{branch}}(s' \rightarrow s)]$$

7.1.1.2.3.3 Traceback

After processing all symbols, traceback from the zero state (encoder was flushed with tail bits) to recover the input sequence.

7.1.1.3 Protocol Integration



7.1.1.4 Performance Characteristics

- Encoding: $O(n)$ where n is input bits
 - Time: ~15 us per 100 bytes @ 240 MHz
 - Memory: Zero dynamic allocation
- Decoding: $O(n \cdot 2^{K-1} \cdot 2) = O(128n)$
 - Time: ~800 us per 100 bytes @ 240 MHz
 - Memory: 64 path metrics + survivors buffer

7.1.1.5 Coding Gain

At BER 10^{-5} , this code provides approximately 5 dB coding gain over uncoded transmission on AWGN channel with soft-decision decoding.

Hardware Requirements

Resource	Usage
CPU	RX72N @ 240 MHz
RAM (Encoder)	~4 bytes (state only)

Resource	Usage
RAM (Decoder)	~16 KB (survivors buffer for 1024B payload)
Flash	~6 KB (code size)

Module Dependencies

- `rx_err.h`: Error codes
- `rx_bit_constants.h`: Bit manipulation constants

NASA Power of 10 Compliance

- Rule 1: [OK] No recursion, no goto, no setjmp/longjmp
- Rule 2: [OK] All loops have fixed bounds (`k_fec_max_symbols`)
- Rule 3: [OK] Zero dynamic memory allocation (all static buffers)
- Rule 4: [OK] Functions < 60 lines (largest: `internal_viterbi_process_symbol` ~50 lines)
- Rule 5: [OK] All functions have ≥ 2 preconditions and postconditions
- Rule 8: [OK] Uses C23 typed enums instead of `#define` for all constants
- Rule 10: [OK] Compiles with `-Wall -Wextra -Werror`

SOLID Principles

- Single Responsibility: Encoder and decoder are separate concerns
- Open/Closed: FEC parameters configurable via enums, no hardcoded magic numbers
- Dependency Inversion: Uses abstract `rx_err_t` for error handling

Note

This implementation is thread-safe if encoder/decoder handles are not shared between threads. Caller must provide synchronization when sharing handles.

Warning

Soft bits must be in range `[-127, +127]`. Values outside this range will be clamped during processing, which may reduce decoding accuracy.

See also

`star-gateway/internal/fec/` for Go reference implementation
`docs/sections/01_nanopb_protocol.tex` for protocol specification
 "Error Control Coding" by Lin & Costello for theory

Author

Locked, Inc.

Date

2026-01-27

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

Version

1.0.0

[Test](#) `test_rx_fec.c` contains unit tests for encoder and decoder

Definition in file [rx_fec.h](#).

7.1.2 Data Structure Documentation

7.1.2.1 struct rx_fec_encoder_t

FEC convolutional encoder state.

The encoder is intentionally stateless between `encode()` calls. Each invocation of [rx_fec_encode\(\)](#) starts with a zeroed shift register and terminates with tail bits to return to zero state. This design simplifies frame-based encoding and eliminates inter-frame dependencies.

7.1.2.2 Rationale for Stateless Design

Advantages:

- Thread Safety: Multiple threads can share read-only encoder (after init)
- Determinism: Same input always produces same output (no hidden state)
- Testability: No need to track encoder history across test cases
- Frame Independence: Dropped frame doesn't corrupt future frames

Trade-off:

- Tail bits add 6 bits overhead per frame ($\sim 0.75\%$ for 100-byte frame)
- Alternative (stateful encoder) would save overhead but risk error propagation

7.1.2.3 Memory Layout

Offset	Size	Field	Type	Alignment	Description
0	1	initialized	bool	1	Initialization flag
1-3	3	(padding)	-	-	Compiler padding to 4B

Total size: 4 bytes (aligned to word boundary for optimal access)

Usage Example

```
// Initialize encoder
rx_fec_encoder_t encoder;
rx_err_t err = rx_fec_encoder_init(&encoder);
if (err != k_rx_ok) {
    return err;
}

// Encode multiple frames (stateless - no crosstalk)
uint8_t input1[100], input2[100];
uint8_t output1[250], output2[250];
uint32_t len1, len2;

rx_fec_encode(&encoder, input1, 100, output1, &len1); // Independent
rx_fec_encode(&encoder, input2, 100, output2, &len2); // Independent

// Cleanup
rx_fec_encoder_deinit(&encoder);
```

Note

The encoder does NOT store shift register state between calls. Internal state is local to `rx_fec_encode()` execution.

See also

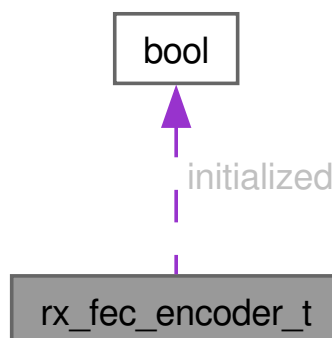
`rx_fec_encoder_init()` Initialize encoder
`rx_fec_encoder_deinit()` Deinitialize encoder
`rx_fec_encode()` Encode data with tail termination

Since

Version 1.0.0

Definition at line 516 of file `rx_fec.h`.

Collaboration diagram for `rx_fec_encoder_t`:



Data Fields

bool	initialized	Initialization status flag. < Set to true by rx_fec_encoder_init(), cleared by rx_fec_encoder_deinit(). Checked by rx_fec_encode() to prevent use-before-init errors (NASA Rule 5). Valid Values: true (initialized), false (not initialized) Default: false (uninitialized struct has zero bytes)
------	-------------	--

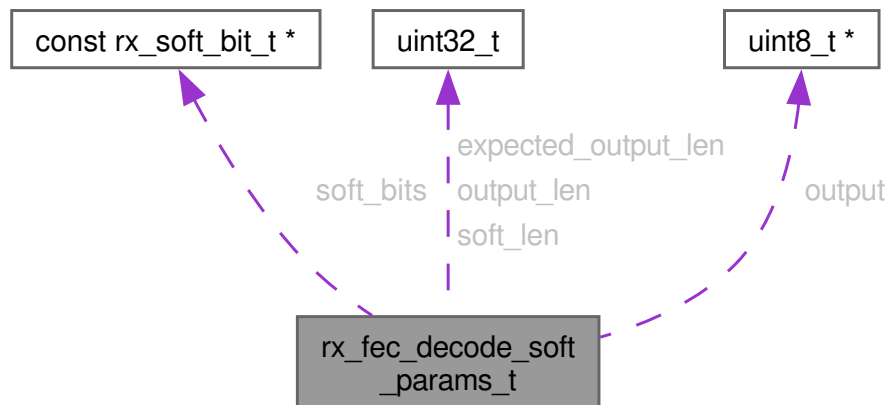
7.1.2.4 struct rx_fec_decode_soft_params_t

Soft-decision decode parameters.

Groups parameters for soft-decision Viterbi decoding to simplify API.

Definition at line 1042 of file [rx_fec.h](#).

Collaboration diagram for rx_fec_decode_soft_params_t:



Data Fields

uint32_t	expected_output_len	Expected decoded length in bytes
uint8_t *	output	Decoded output buffer
uint32_t *	output_len	Actual decoded length
const rx_soft_bit_t *	soft_bits	Received soft bits (pairs)
uint32_t	soft_len	Number of soft bits (must be even)

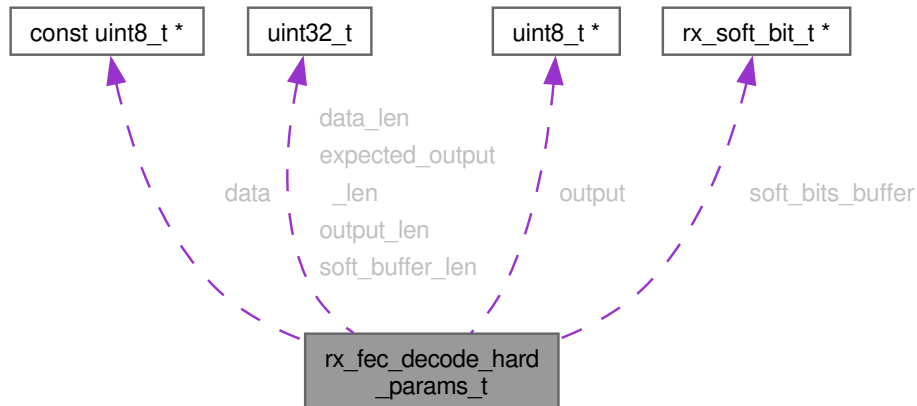
7.1.2.5 struct rx_fec_decode_hard_params_t

Hard-decision decode parameters.

Groups parameters for hard-decision Viterbi decoding to simplify API.

Definition at line 1072 of file [rx_fec.h](#).

Collaboration diagram for rx_fec_decode_hard_params_t:



Data Fields

const uint8_t *	data	Received hard bits (packed bytes)
uint32_t	data_len	Number of bytes
uint32_t	expected_output_len	Expected decoded length in bytes
uint8_t *	output	Decoded output buffer
uint32_t *	output_len	Actual decoded length
rx_soft_bit_t *	soft_bits_buffer	Working buffer for soft bit conversion
uint32_t	soft_buffer_len	Length of soft_bits_buffer

7.1.3 Typedef Documentation

7.1.3.1 rx_soft_bit_t

```
typedef int8_t rx_soft_bit_t
```

Soft bit type for soft-decision decoding.

Soft bits carry both demodulated bit value and reliability information. This enables the Viterbi decoder to make better decisions than hard-decision decoding, providing approximately 2 dB coding gain improvement.

7.1.3.2 Soft Bit Interpretation

The soft bit value $s \in [-127, +127]$ represents:

- Sign: Demodulated bit value
 - $s \geq 0$: Received bit is likely 1
 - $s < 0$: Received bit is likely 0
- Magnitude: Confidence in decision
 - $|s| = 127$: High confidence (strong signal, no noise)
 - $|s| \approx 0$: Low confidence (weak signal, high noise)
 - $s = 0$: Erasure (completely uncertain)

7.1.3.3 Mathematical Model

Given received signal r and expected values $+A$ (bit=1) or $-A$ (bit=0), the soft bit is computed as:

$$s = \text{clamp}\left(\frac{r}{A} \cdot 127, -127, +127\right)$$

7.1.3.4 Conversion from Hard Bits

Hard bit $b \in \{0, 1\}$ converts to soft bit:

$$s = \begin{cases} +127 & \text{if } b = 1 \\ -127 & \text{if } b = 0 \end{cases}$$

This represents maximum confidence in the hard decision.

7.1.3.5 Why Signed 8-bit?

- Range: $[-127, +127]$ provides sufficient quantization (8 levels above/below threshold)
- Memory: 1 byte per soft bit vs 1 bit per hard bit (8x overhead acceptable for 2 dB gain)
- Arithmetic: Signed addition in Chase Combining uses int16 to prevent overflow
- Performance: Native CPU type, no bit-packing overhead

Usage Example

```

// Hard bit conversion
uint8_t hard_bit = 1;
rx_soft_bit_t soft = rx_fec_hard_to_soft(hard_bit); // soft = +127

// Reverse conversion
uint8_t recovered = rx_fec_soft_to_hard(soft); // recovered = 1

// Erasure detection
rx_soft_bit_t uncertain = 0;
if (uncertain == k_soft_bit_zero) {
    // Channel reported complete uncertainty - may want to request retransmission
}

// Confidence thresholding
rx_soft_bit_t weak_signal = 10; // Low confidence
if (abs(weak_signal) < 50) {
    // Signal too weak, may need combining with retransmissions
}

```

Note

The `int8_t` type is signed, providing natural representation of positive/negative polarities without bit manipulation.

Warning

Values outside `[-127, +127]` may occur during Chase Combining accumulation (uses `int16_t`), but are clamped before decoding.

See also

[rx_fec_hard_to_soft\(\)](#) Convert hard bit to soft bit
[rx_fec_soft_to_hard\(\)](#) Convert soft bit to hard bit
[rx_chase_combiner_t](#) Accumulates soft bits across retransmissions

Since

Version 1.0.0

Definition at line 376 of file [rx_fec.h](#).

7.1.4 Enumeration Type Documentation

7.1.4.1 rx_fec_buffer_limits_t

enum [rx_fec_buffer_limits_t](#) : `uint16_t`

Maximum buffer sizes for static FEC decoder allocation.

Defines compile-time buffer limits to enable zero dynamic allocation (NASA Rule 3). Values calculated from maximum payload size (1024 bytes) and FEC parameters.

7.1.4.2 Size Calculations

For maximum 1024-byte payload:

- Input bits: $1024 \times 8 = 8192$
- With tail: $8192 + 6 = 8198$ bits
- Encoded symbols (rate 1/2): 8198 symbol pairs
- Safety margin: Round to 8200 symbols

Memory Usage

Buffer	Size	Bytes
Soft bits	8200 x 2 x 1 byte	16400
Survivors	8200 x 8 bytes (uint64_t)	65600
Path metrics	64 x 4 bytes (int32_t)	256
Branch table	64 x 2 x 2 x 1 byte	256
Total per decoder		**~82 KB**

Note

Large memory footprint limits number of concurrent decoders. RX72N has 512 KB SRAM, allowing ~6 decoders maximum.

See also

[rx_fec_decoder_t](#) Decoder structure using these limits

Since

Version 1.0.0

Enumerator

k_fec_max_symbols	Maximum symbols for Viterbi trellis: 8200. Supports up to 1024-byte payloads (8192 bits + 6 tail bits + 2 margin). Each symbol is a pair of soft bits (G1, G2). Maximum input bytes before FEC encoding: 1024 Derived from k_fec_max_symbols: $(8200 - 6) / 8 = 1024.25 \rightarrow 1024$ bytes. Larger payloads must be fragmented.
k_fec_max_input_bytes	

Definition at line 981 of file [rx_fec.h](#).

```

00981         : uint16_t {
00982     /**< @brief Maximum symbols for Viterbi trellis: 8200
00983     * @details Supports up to 1024-byte payloads (8192 bits + 6 tail bits + 2 margin).
00984     * Each symbol is a pair of soft bits (G1, G2). */
00985     k_fec_max_symbols = 8200,
00986
00987     /**< @brief Maximum input bytes before FEC encoding: 1024
00988     * @details Derived from k_fec_max_symbols:  $(8200 - 6) / 8 = 1024.25 \rightarrow 1024$  bytes.
00989     * Larger payloads must be fragmented. */
00990     k_fec_max_input_bytes = (uint16_t)((k_fec_max_symbols - k_fec_tail_bits) / k_rx_bits_per_byte),
00991 } rx_fec_buffer_limits_t;
```

7.1.4.3 rx_fec_params_t

enum `rx_fec_params_t` : `uint8_t`

FEC convolutional code parameters.

Defines the NASA-standard K=7, rate 1/2 convolutional code used throughout the STAR project. These parameters were selected for optimal balance between coding gain (~ 5 dB @ BER 10^{-5}) and computational complexity.

7.1.4.4 Parameter Relationships

The parameters satisfy these mathematical constraints:

- $\text{num_states} = 2^{\text{constraint_length}-1} = 2^6 = 64$
- $\text{tail_bits} = \text{constraint_length} - 1 = 6$
- $\text{code_rate} = \frac{1}{\text{num_outputs}} = \frac{1}{2}$

7.1.4.5 Generator Polynomial Selection

G1=171 (octal) and G2=133 (octal) provide maximum free distance $d_{\text{free}} = 10$ among all K=7 codes, yielding best error correction performance.

Binary representations:

- $G_1 = 1111001_2 = 171_8 = 0x79$
- $G_2 = 1011011_2 = 133_8 = 0x5B$

Traceback Depth Rationale

Traceback depth of 5K is industry standard for K=7 codes, providing 99.99% probability of correct path selection while minimizing memory ($5 \times 7 = 35$ symbols = 70 bits = ~ 9 bytes per state).

Field Constraints

Field	Min	Max	Rationale
constraint_length	7	7	Fixed by NASA standard
num_states	64	64	Derived: $2^{(K-1)}$
tail_bits	6	6	Derived: K-1 (flush encoder to zero)
traceback_depth	35	35	5K empirical optimal
num_outputs	2	2	Rate 1/2 code (fixed)

Usage Example

```
// Calculate encoded length
uint32_t input_bits = 800; // 100 bytes
uint32_t total_bits = input_bits + k_fec_tail_bits; // 806 bits
uint32_t output_bits = total_bits * k_fec_num_outputs; // 1612 bits
uint32_t output_bytes = (output_bits + 7) / 8; // 202 bytes

// Verify state count
assert(k_fec_num_states == (1 << (k_fec_constraint_length - 1)));
```

See also

"Error Control Coding" by Lin & Costello, Section 11.3

NASA/JPL standard for deep space communication (K=7, r=1/2)

Since

Version 1.0.0

Enumerator

k_fec_constraint_length	Constraint length K=7 (shift register holds K-1 = 6 bits). Determines number of states ($2^6=64$) and code complexity. Larger K increases coding gain but exponentially increases decoder complexity. K=7 is optimal for embedded systems. Number of encoder states: $2^{(K-1)} = 64$ Each state represents a unique 6-bit shift register configuration. Viterbi decoder maintains path metrics for all 64 states simultaneously.
k_fec_num_states	Tail bits for termination: K-1 = 6 zeros. Appended to input to flush encoder to zero state, enabling traceback to start from known state. Without tail bits, decoder would need to search all final states, reducing performance.
k_fec_tail_bits	Generator polynomial G1 in octal: 171 (binary 1111001). Tap connections: positions 0,3,4,5,6 (MSB is input bit). Combined with G2 to achieve d_free=10 (maximum for K=7).
k_fec_g1_octal	Generator polynomial G2 in octal: 133 (binary 1011011). Tap connections: positions 0,1,2,3,6 (MSB is input bit). Complementary to G1 to maximize minimum Hamming distance.
k_fec_g2_octal	Traceback depth: $5 \times K = 35$ symbols. Industry standard for K=7 codes. Deeper traceback increases latency and memory but negligibly improves error rate. 5K provides >99.99% probability of correct decision. Memory: 35×64 bits = 280 bytes.
k_fec_traceback_depth	Number of possible input values: 2 (binary: 0 or 1). Used in branch table generation to enumerate all transitions. For each state, there are 2 possible next states (input=0 or input=1).
k_fec_num_input_values	Number of output bits per input bit: 2 (rate 1/2). Code rate R = 1/2 means bandwidth expansion by factor of 2. Lower rates (1/3, 1/4) increase coding gain but reduce data throughput. 1/2 is optimal for most communication systems.
k_fec_num_outputs	

Definition at line 242 of file rx_fec.h.

```
00242 : uint8_t {
00243 /**< @brief Constraint length K=7 (shift register holds K-1 = 6 bits)
```

```

00244 * @details Determines number of states ( $2^6=64$ ) and code complexity.
00245 * Larger K increases coding gain but exponentially increases decoder
00246 * complexity. K=7 is optimal for embedded systems. */
00247 k_fec_constraint_length = 7,
00248
00249 /**< @brief Number of encoder states:  $2^{(K-1)} = 64$ 
00250 * @details Each state represents a unique 6-bit shift register configuration.
00251 * Viterbi decoder maintains path metrics for all 64 states simultaneously. */
00252 k_fec_num_states = 64,
00253
00254 /**< @brief Tail bits for termination: K-1 = 6 zeros
00255 * @details Appended to input to flush encoder to zero state, enabling
00256 * traceback to start from known state. Without tail bits, decoder would
00257 * need to search all final states, reducing performance. */
00258 k_fec_tail_bits = 6,
00259
00260 /**< @brief Generator polynomial G1 in octal: 171 (binary 1111001)
00261 * @details Tap connections: positions 0,3,4,5,6 (MSB is input bit).
00262 * Combined with G2 to achieve d_free=10 (maximum for K=7). */
00263 k_fec_g1_octal = 0171,
00264
00265 /**< @brief Generator polynomial G2 in octal: 133 (binary 1011011)
00266 * @details Tap connections: positions 0,1,2,3,6 (MSB is input bit).
00267 * Complementary to G1 to maximize minimum Hamming distance. */
00268 k_fec_g2_octal = 0133,
00269
00270 /**< @brief Traceback depth:  $5 \times K = 35$  symbols
00271 * @details Industry standard for K=7 codes. Deeper traceback increases
00272 * latency and memory but negligibly improves error rate. 5K provides
00273 * >99.99% probability of correct decision. Memory:  $35 \times 64$  bits = 280 bytes. */
00274 k_fec_traceback_depth = 35,
00275
00276 /**< @brief Number of possible input values: 2 (binary: 0 or 1)
00277 * @details Used in branch table generation to enumerate all transitions.
00278 * For each state, there are 2 possible next states (input=0 or input=1). */
00279 k_fec_num_input_values = 2,
00280
00281 /**< @brief Number of output bits per input bit: 2 (rate 1/2)
00282 * @details Code rate  $R = 1/2$  means bandwidth expansion by factor of 2.
00283 * Lower rates (1/3, 1/4) increase coding gain but reduce data throughput.
00284 * 1/2 is optimal for most communication systems. */
00285 k_fec_num_outputs = 2,
00286 } rx_fec_params_t;

```

7.1.4.6 rx_soft_bit_limits_t

```
enum rx_soft_bit_limits_t : int8_t
```

Soft bit value range limits.

Defines the valid range for soft bit values. These limits ensure that soft bits fit in an `int8_t` and provide symmetric positive/negative ranges.

7.1.4.7 Range Justification

- Maximum +127: Highest positive value for `int8_t`. Represents absolute certainty that received bit is 1.
- Minimum -127: Symmetric negative bound. Represents absolute certainty that received bit is 0. Note: -128 is avoided to maintain symmetry ($|+127| = |-127| = 127$).
- Zero: No information. Used for erasures (corrupted symbols that provide no reliable information).

7.1.4.8 Why Not -128?

Using -128 would create asymmetry: $|-128| = 128 \neq 127$. This would complicate branch metric calculations and potentially cause subtle biases in the Viterbi decoder. Symmetric range ensures:

$$|k_{\text{soft_bit_max}}| = |k_{\text{soft_bit_min}}| = 127$$

Usage Example

```

// Validate soft bit range
rx_soft_bit_t s = get_demodulated_soft_bit();
if (s < k_soft_bit_min || s > k_soft_bit_max) {
    // Clamp to valid range
    if (s < k_soft_bit_min) s = k_soft_bit_min;
    if (s > k_soft_bit_max) s = k_soft_bit_max;
}

// Initialize soft bit array to erasures
rx_soft_bit_t soft_bits[100];
for (int i = 0; i < 100; i++) {
    soft_bits[i] = k_soft_bit_zero; // All erasures
}

```

See also

[rx_soft_bit_t](#) Soft bit type definition

Since

Version 1.0.0

Enumerator

k_soft_bit_max	Maximum soft bit value: +127 (confident bit=1). Represents strongest possible belief that received bit is 1. Used when channel has no noise and signal is clearly above threshold. Minimum soft bit value: -127 (confident bit=0) Represents strongest possible belief that received bit is 0. Used when channel has no noise and signal is clearly below threshold. Note: -128 is NOT used to maintain symmetric range.
k_soft_bit_min	Zero confidence: 0 (erasure). Represents complete uncertainty about bit value. Used when: - Signal is at exact threshold (cannot decide 0 vs 1) - Channel reported error/corruption - Demodulator detected ambiguous symbol Viterbi decoder treats erasures as equally likely 0 or 1.
k_soft_bit_zero	

Definition at line 429 of file [rx_fec.h](#).

```

00429     : int8_t {
00430     /**< @brief Maximum soft bit value: +127 (confident bit=1)
00431     * @details Represents strongest possible belief that received bit is 1.
00432     * Used when channel has no noise and signal is clearly above threshold. */
00433     k_soft_bit_max = 127,
00434
00435     /**< @brief Minimum soft bit value: -127 (confident bit=0)
00436     * @details Represents strongest possible belief that received bit is 0.
00437     * Used when channel has no noise and signal is clearly below threshold.
00438     * Note: -128 is NOT used to maintain symmetric range. */
00439     k_soft_bit_min = -127,
00440
00441     /**< @brief Zero confidence: 0 (erasure)
00442     * @details Represents complete uncertainty about bit value. Used when:
00443     * - Signal is at exact threshold (cannot decide 0 vs 1)
00444     * - Channel reported error/corruption
00445     * - Demodulator detected ambiguous symbol
00446     * Viterbi decoder treats erasures as equally likely 0 or 1. */
00447     k_soft_bit_zero = 0,
00448 } rx_soft_bit_limits_t;

```

7.1.5 Function Documentation

7.1.5.1 rx_fec_decode_hard()

```
rx_err_t rx_fec_decode_hard (
    rx_fec_decoder_t * dec,
    const rx_fec_decode_hard_params_t * params) [nodiscard]
```

Decode hard bits using Viterbi algorithm.

Converts hard bits to soft bits internally, then performs Viterbi decoding. Less accurate than soft-decision decoding but useful for testing.

The caller must provide a working buffer for soft bit conversion to ensure thread safety. Size should be at least `k_fec_max_symbols * k_fec_num_outputs`.

Parameters

in	dec	Initialized decoder handle
in,out	params	Decode parameters (input data, buffers)

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if any pointer is nullptr
`k_rx_err_invalid_state` if decoder not initialized
`k_rx_err_invalid_size` if `soft_bits_buffer` is too small

Decode hard bits using Viterbi algorithm.

Convenience wrapper that converts hard-decision bits (0/1) to soft-decision values and delegates to [rx_fec_decode_soft\(\)](#). Accepts raw bit-packed data (MSB-first) as produced by standard digital logic or RSPI transfers.

Algorithm steps:

1. Validate all input pointers and size constraints
2. Convert each hard bit to a soft value via [rx_fec_hard_to_soft\(\)](#) (maps 0 -> negative confidence, 1 -> positive confidence)
3. Build [rx_fec_decode_soft_params_t](#) from the converted soft bits
4. Delegate to [rx_fec_decode_soft\(\)](#) for full Viterbi decode

Precondition

`dec` must be initialized via [rx_fec_decoder_init\(\)](#)
`params->soft_bits_buffer` must be large enough: `soft_buffer_len >= data_len * 8`
`params->data_len` must be `> 0` and `<= k_fec_max_input_bytes`

Postcondition

*params->output_len written with actual decoded byte count on k_rx_ok
 params->output contains decoded data bytes on k_rx_ok
 params->soft_bits_buffer populated with converted soft values (side effect)

Note

Not thread-safe; caller must ensure exclusive access to dec and params
 Hard-decision decoding has ~3 dB lower coding gain than soft-decision

See also

[rx_fec_decode_soft\(\)](#) Preferred: use when soft values are available
[rx_fec_decoder_init\(\)](#) Initialize decoder before calling this function
[rx_fec_hard_to_soft\(\)](#) Conversion function used internally
[rx_fec_encode\(\)](#) Encoder that produces data this function decodes

Since

Version 1.0.0

```
// Example: hard-decision decode of packed FEC-encoded bytes
rx_fec_decoder_t dec;
uint64_t survivors[k_fec_max_symbols];
rx_soft_bit_t soft_scratch[k_fec_max_symbols * k_fec_num_outputs];
uint8_t out_buf[k_fec_max_input_bytes];
uint32_t out_len = k_fec_zero;

(void)rx_fec_decoder_init(&dec, survivors, k_fec_max_symbols);

rx_fec_decode_hard_params_t p = {
    .data = received_bytes,
    .data_len = received_len,
    .soft_bits_buffer = soft_scratch,
    .soft_buffer_len = k_fec_max_symbols * k_fec_num_outputs,
    .expected_output_len = original_data_len,
    .output = out_buf,
    .output_len = &out_len,
};
rx_err_t err = rx_fec_decode_hard(&dec, &p);
```

Definition at line 1052 of file rx_fec.c.

```
01053 {
01054     if (dec == nullptr || params == nullptr) {
01055         return k_rx_err_invalid_arg;
01056     }
01057
01058     if (params->data == nullptr || params->output == nullptr || params->output_len == nullptr ||
01059         params->soft_bits_buffer == nullptr) {
01060         return k_rx_err_invalid_arg;
01061     }
01062
01063     if (!dec->initialized) {
01064         return k_rx_err_invalid_state;
01065     }
01066
01067     if (params->data_len == k_fec_zero) {
01068         return k_rx_err_invalid_arg;
01069     }
01070
01071     if (params->data_len > k_fec_max_input_bytes) {
01072         return k_rx_err_invalid_size;
01073     }
01074
01075     /* Convert hard bits to soft bits */
01076     const uint32_t num_bits = (uint32_t)(params->data_len * k_rx_bits_per_byte);
01077
01078     /* Ensure soft bits buffer is large enough */
```

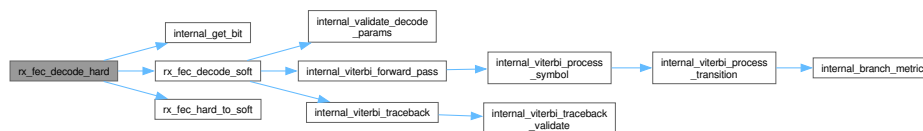
```

01079 if (num_bits > params->soft_buffer_len) {
01080     return k_rx_err_invalid_size;
01081 }
01082
01083 for (uint32_t i = k_fec_zero; i < num_bits; i++) {
01084     const uint8_t bit = internal_get_bit(params->data, i);
01085     params->soft_bits_buffer[i] = rx_fec_hard_to_soft(bit);
01086 }
01087
01088 /* Prepare soft decode parameters */
01089 const rx_fec_decode_soft_params_t soft_params = {
01090     .soft_bits      = params->soft_bits_buffer,
01091     .soft_len       = num_bits,
01092     .expected_output_len = params->expected_output_len,
01093     .output         = params->output,
01094     .output_len     = params->output_len,
01095 };
01096
01097 return rx_fec_decode_soft(dec, &soft_params);
01098 }

```

References [rx_fec_decode_hard_params_t::data](#), [rx_fec_decode_hard_params_t::data_len](#), [rx_fec_decode_hard_params_t::initialized](#), [internal_get_bit\(\)](#), [k_fec_max_input_bytes](#), [k_fec_zero](#), [rx_fec_decode_hard_params_t::output_len](#), [rx_fec_decode_soft\(\)](#), [rx_fec_hard_to_soft\(\)](#), [rx_fec_decode_hard_params_t::soft_buffer_len](#).

Here is the call graph for this function:



7.1.5.2 rx_fec_decode_soft()

```

rx_err_t rx_fec_decode_soft (
    rx_fec_decoder_t * dec,
    const rx_fec_decode_soft_params_t * params) [nodiscard]

```

Decode soft bits using Viterbi algorithm.

Performs soft-decision Viterbi decoding on received soft bits. Soft bits should be in pairs (G1, G2) for each encoded symbol.

Parameters

in	dec	Initialized decoder handle
in,out	params	Decode parameters (input soft bits, output buffer)

Returns

- k_rx_ok on success
- k_rx_err_invalid_arg if any pointer is nullptr or soft_len is odd
- k_rx_err_invalid_state if decoder not initialized
- k_rx_err_invalid_size if output buffer too small

Decode soft bits using Viterbi algorithm.

Implements Rate-1/2 Viterbi soft-decision decoding with constraint length 7. Accepts soft-decision values (signed integers in range [-128, 127]) to achieve approximately 3 dB coding gain over hard-decision decoding.

Algorithm steps:

1. Validate parameters via [internal_validate_decode_params\(\)](#)
2. Forward pass: compute branch and path metrics for all symbols
3. Calculate data bit count (num_symbols - tail bits)
4. Traceback: extract maximum-likelihood decoded bit sequence
5. Pack decoded bits into output byte array

Precondition

dec must be initialized via [rx_fec_decoder_init\(\)](#)
params->soft_len must be divisible by 2 (G1/G2 symbol pairs)
params->expected_output_len must be > 0 and <= k_fec_max_input_bytes

Postcondition

*params->output_len written with actual decoded byte count on k_rx_ok
params->output contains decoded data bytes on k_rx_ok
dec path metric arrays updated (internal state consumed by traceback)

Note

Not thread-safe; caller must ensure exclusive access to dec and params
Soft values: positive = bit-1 likely, negative = bit-0 likely (signed).

See also

[rx_fec_decoder_init\(\)](#) Initialize decoder before calling this function
[rx_fec_decode_hard\(\)](#) Hard-decision wrapper (lower coding gain, simpler input)
[rx_fec_encode\(\)](#) Encoder that produces the data this function decodes

Since

Version 1.0.0

```
// Example: soft-decision decode of a previously FEC-encoded buffer
rx_fec_decoder_t      dec;
uint64_t              survivors[k_fec_max_symbols];
rx_soft_bit_t         soft_buf[k_fec_max_symbols * k_fec_num_outputs];
uint8_t               out_buf[k_fec_max_input_bytes];
uint32_t              out_len = k_fec_zero;

(void)rx_fec_decoder_init(&dec, survivors, k_fec_max_symbols);

// Populate soft_buf from channel (positive = bit-1 likely)
rx_fec_decode_soft_params_t p = {
    .soft_bits      = soft_buf,
    .soft_len       = encoded_len_bits,
    .expected_output_len = original_data_len,
    .output         = out_buf,
    .output_len     = &out_len,
};
rx_err_t err = rx_fec_decode_soft(&dec, &p);
```

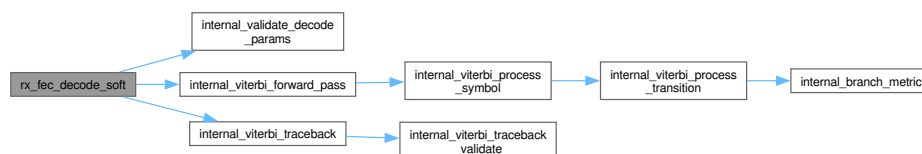
Definition at line 972 of file [rx_fec.c](#).

```
00973 {
00974     uint32_t      num_symbols = k_fec_zero;
00975     const rx_err_t err      = internal_validate_decode_params(dec, params, &num_symbols);
00976     if (err != k_rx_ok) {
00977         return err;
00978     }
00979
00980     /* Forward pass: initialize path metrics and process symbols */
00981     internal_viterbi_forward_pass(dec, params->soft_bits, num_symbols);
00982
00983     /* Calculate data bits and output size */
00984     const uint32_t data_bits = num_symbols - k_fec_tail_bits;
00985
00986     const uint32_t output_bytes = (data_bits + k_fec_msb_bit_position) / k_rx_bits_per_byte;
00987
00988     /* Traceback: extract decoded bits */
00989     internal_viterbi_traceback(dec, num_symbols, data_bits, params->output, output_bytes);
00990
00991     *params->output_len = output_bytes;
00992     return k_rx_ok;
00993 }
```

References [internal_validate_decode_params\(\)](#), [internal_viterbi_forward_pass\(\)](#), [internal_viterbi_traceback\(\)](#), [k_fec_msb_bit_position](#), [k_fec_tail_bits](#), [k_fec_zero](#), [rx_fec_decode_soft_params_t::output](#), [rx_fec_decode_soft_params_t::output_len](#), and [rx_fec_decode_soft_params_t::soft_bits](#).

Referenced by [rx_fec_decode_hard\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.1.5.3 rx_fec_decoder_deinit()

```
rx_err_t rx_fec_decoder_deinit (
    rx_fec_decoder_t * dec) [nodiscard]
```

Deinitialize FEC decoder.

Parameters

in,out	dec	Pointer to decoder handle
--------	-----	---------------------------

Returns

k_rx_ok on success

Definition at line 905 of file [rx_fec.c](#).

```
00906 {
00907     if (dec == nullptr) {
00908         return k_rx_err_invalid_arg;
00909     }
00910
00911     dec->survivors = nullptr;
00912     dec->survivors_len = k_fec_zero;
00913     dec->initialized = false;
00914     return k_rx_ok;
00915 }
```

References [rx_fec_decoder_t::initialized](#), [k_fec_zero](#), [rx_fec_decoder_t::survivors](#), and [rx_fec_decoder_t::survivors_len](#).

7.1.5.4 rx_fec_decoder_init()

```
rx_err_t rx_fec_decoder_init (
    rx_fec_decoder_t * dec,
    uint64_t * survivors_buf,
    const uint32_t survivors_len)
```

Initialize FEC decoder.

The caller must provide a survivors buffer for traceback. Size should be at least $(\text{max_symbols} / 64 + 1) * \text{k_fec_num_states}$ uint64_t elements.

Parameters

out	dec	Pointer to decoder handle
in	survivors_buf	Caller-provided buffer for survivor bits
in	survivors_len	Number of uint64_t elements in survivors_buf

Returns

k_rx_ok on success

k_rx_err_invalid_arg if any pointer is nullptr or buffer too small

Initializes decoder state and associates it with provided survivors buffer. The buffer is used for Viterbi algorithm path storage.

Precondition

dec and survivors_buf must be non-nullptr
 survivors_len must be > 0 (minimum 1 entry per symbol)

Postcondition

dec->initialized set to true
 Branch table initialized

Definition at line 879 of file rx_fec.c.

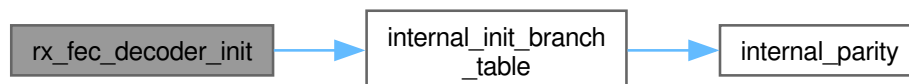
```

00880 {
00881     if (dec == nullptr || survivors_buf == nullptr) {
00882         return k_rx_err_invalid_arg;
00883     }
00884     /* Minimum survivors buffer size: at least 1 entry per symbol */
00885     if (survivors_len == k_fec_zero) {
00886         return k_rx_err_invalid_size;
00887     }
00888     dec->survivors = survivors_buf;
00889     dec->survivors_len = survivors_len;
00890     /* Initialize branch table */
00891     internal_init_branch_table(dec->branch_table);
00892     dec->initialized = true;
00893     return k_rx_ok;
00894 }

```

References [rx_fec_decoder_t::branch_table](#), [rx_fec_decoder_t::initialized](#), [internal_init_branch_table\(\)](#), [k_fec_zero](#), [rx_fec_decoder_t::survivors](#), and [rx_fec_decoder_t::survivors_len](#).

Here is the call graph for this function:



7.1.5.5 rx_fec_encode()

```

rx_err_t rx_fec_encode (
    const rx_fec_encoder_t * enc,
    const uint8_t * input,
    const uint32_t input_len,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]

```

Encode data using K=7, rate 1/2 convolutional code.

Encodes input bytes into FEC-protected output using NASA-standard convolutional code. Each input bit produces 2 output bits (G1 and G2), followed by 6 tail bits to flush encoder to zero state.

7.1.5.6 Algorithm Steps

1. Validation: Check pointers, initialization, input length
2. Calculation: Compute output size = (input_bits + 6) x 2 / 8 bytes
3. Initialization: Clear output buffer, reset shift register to 0
4. Data Encoding: For each input bit (MSB first):
 - Compute G1 = parity(input_bit || state) & 0x79
 - Compute G2 = parity(input_bit || state) & 0x5B
 - Output (G1, G2) bit pair
 - Update state: shift right, insert input_bit
5. Tail Termination: Encode 6 zero bits to flush state to 0
6. Return: Set output_len, return success

7.1.5.7 Mathematical Model

For input sequence $\mathbf{u} = [u_0, u_1, \dots, u_{n-1}]$, the encoder produces output $\mathbf{c} = [c_0^1, c_0^2, c_1^1, c_1^2, \dots]$ where:

$$c_i^1 = u_i \oplus \sum_{j=0}^5 g_1[j] \cdot s_j$$

$$c_i^2 = u_i \oplus \sum_{j=0}^5 g_2[j] \cdot s_j$$

State update: $s_{i+1} = [u_i, s_{i,5}, s_{i,4}, s_{i,3}, s_{i,2}, s_{i,1}]$

7.1.5.8 Output Length Formula

Given n input bytes:

$$L_{\text{out}} = \left\lceil \frac{(8n + 6) \times 2}{8} \right\rceil = 2n + 2 \text{ bytes}$$

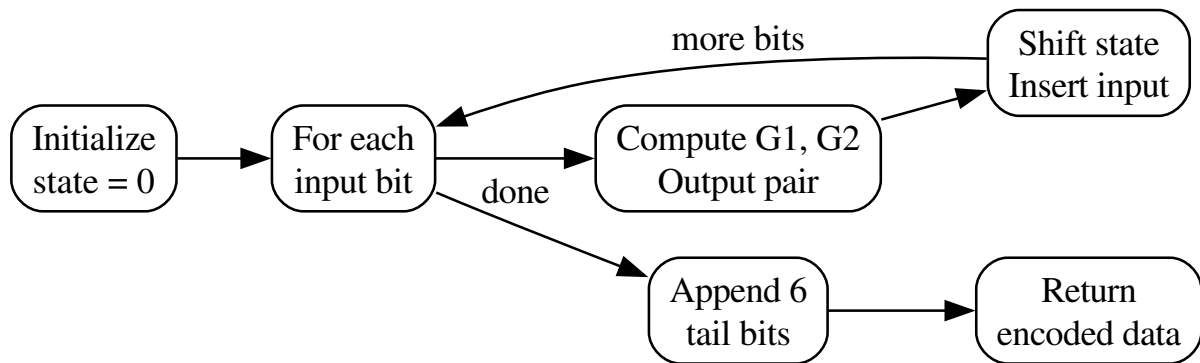
Examples:

- 100 bytes in -> 202 bytes out (expansion factor ~ 2.02)
- 1 byte in -> 4 bytes out (overhead significant for small frames)

7.1.5.9 Performance Analysis

- Time Complexity: $O(8n)$ where $n = \text{input_len}$
- Space Complexity: $O(1)$ (constant stack usage)
- Execution Time: ~ 15 us for 100 bytes @ 240 MHz
- Per-Bit Cost: ~ 19 CPU cycles per input bit

7.1.5.10 State Machine



Parameters

in	enc	Initialized encoder handle • Must be initialized via rx_fec_encoder_init() • Read-only (const): Can share among threads
in	input	Input data bytes to encode • Must not be nullptr • Valid range: [1, k_fec_max_input_bytes] bytes • Read-only: Original data unchanged
in	input_len	Number of input bytes • Valid range: [1, 1024] bytes • Constraint: input_len ≤ k_fec_max_input_bytes
out	output	Output buffer for encoded data • Must not be nullptr • Minimum size: rx_fec_encoded_len(input_len) bytes • Cleared before writing • MSB-first bit packing (network byte order)
out	output_len	Actual number of output bytes written • Must not be nullptr • Set to (input_len × 8 + 6) × 2 / 8 bytes • Always ≤ buffer size if input valid

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Successfully encoded, output contains FEC data
<code>k_rx_err_invalid_arg</code>	Any pointer is nullptr or <code>input_len</code> is 0
<code>k_rx_err_invalid_state</code>	Encoder not initialized
<code>k_rx_err_invalid_size</code>	<code>input_len</code> exceeds <code>k_fec_max_input_bytes</code>

Precondition

`enc->initialized == true` (via `rx_fec_encoder_init()`)
`input != nullptr && output != nullptr && output_len != nullptr`
`1 <= input_len <= k_fec_max_input_bytes`
`output buffer size >= rx_fec_encoded_len(input_len)`

Postcondition

`output` contains FEC-encoded data (big-endian bit packing)
`*output_len == (input_len x 8 + 6) x 2 / 8` (rounded up)
 Last 6 tail bits ensure encoder state returns to zero

Invariant

Encoding is deterministic: same input -> same output
 Output length formula always holds: $2n + 2$ bytes for n input bytes

Note

Thread Safety: Encoder is read-only during encode, safe for concurrent `encode()` calls on same encoder handle.

Bit Ordering: Bits packed MSB-first (network byte order). First output bit goes to `byte[0]` bit 7.

Performance: For critical paths, consider encoding in batches to amortize function call overhead.

Warning

`output` buffer MUST be at least `rx_fec_encoded_len(input_len)` bytes. Buffer overflow protection is caller's responsibility.

Example: Basic Encoding

```

rx_fec_encoder_t enc;
rx_fec_encoder_init(&enc);

uint8_t input[100] = {...}; // 100 bytes input data
uint32_t output_size = rx_fec_encoded_len(100); // 202 bytes
uint8_t output[202];
uint32_t actual_len;

rx_err_t err = rx_fec_encode(&enc, input, 100, output, &actual_len);
if (err == k_rx_ok) {
    // actual_len == 202
    // Transmit output[0..201] over noisy channel
}
```

Example: Error Handling

```

rx_err_t err = rx_fec_encode(&enc, data, len, out, &out_len);
switch (err) {
    case k_rx_ok:
        break; // Success
    case k_rx_err_invalid_arg:
        rx_log_error("FEC", "nullptr or zero length");
        return err;
    case k_rx_err_invalid_state:
        rx_log_error("FEC", "Encoder not initialized");
        return err;
    case k_rx_err_invalid_size:
        rx_log_error("FEC", "Input too large (max 1024 bytes)");
        return err;
}

```

Example: Batch Encoding

```

// Encode multiple frames efficiently
for (int i = 0; i < num_frames; i++) {
    uint32_t len;
    rx_err_t err = rx_fec_encode(&enc, frames[i], frame_sizes[i],
                                encoded[i], &len);
    if (err != k_rx_ok) {
        // Handle error for frame i
        continue;
    }
    // Transmit encoded[i][0..len-1]
}

```

See also

[rx_fec_encoder_init\(\)](#) Initialize encoder first
[rx_fec_encoded_len\(\)](#) Calculate required output buffer size
[rx_fec_decode_soft\(\)](#) Decode with soft decision
[rx_fec_decode_hard\(\)](#) Decode with hard decision

Since

Version 1.0.0

Version

1.0.0

[Test test_rx_fec.c::test_encode_basic\(\)](#) Tests basic encoding
[test_rx_fec.c::test_encode_tail_bits\(\)](#) Verifies tail bit termination

NASA Power of 10 Compliance

- Rule 2: [OK] Loop bounds statically provable (`input_len <= k_fec_max_input_bytes`)
- Rule 5: [OK] 7 precondition checks, 3 postconditions

Encode data using K=7, rate 1/2 convolutional code.

Applies rate-1/2, constraint-length-7 convolutional encoding to input data. Output length is approximately 2x input length.

Precondition

enc must be initialized via [rx_fec_encoder_init\(\)](#)
 All pointers must be non-nullptr
 input_len in range [1, k_fec_max_input_bytes]

Postcondition

output buffer contains FEC-encoded data
 *output_len set to actual encoded length

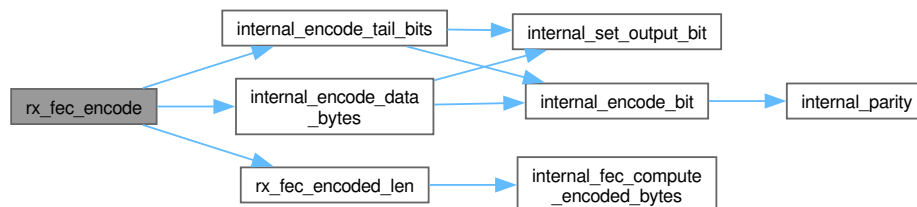
Definition at line 738 of file [rx_fec.c](#).

```

00743 {
00744   if (enc == nullptr || input == nullptr || output == nullptr || output_len == nullptr) {
00745     return k_rx_err_invalid_arg;
00746   }
00747   if (!enc->initialized) {
00748     return k_rx_err_invalid_state;
00749   }
00750 }
00751   if (input_len == k_fec_zero) {
00752     return k_rx_err_invalid_arg;
00753   }
00754 }
00755   if (input_len > k_fec_max_input_bytes) {
00756     return k_rx_err_invalid_size;
00757   }
00758 }
00759   /* Calculate output size and clear output buffer */
00760   const uint32_t expected_output_len = rx_fec_encoded_len(input_len);
00761   for (uint32_t i = k_fec_zero; i < expected_output_len; i++) {
00762     output[i] = (uint8_t)k_fec_zero;
00763   }
00764 }
00765   /* Encode input bytes then flush with tail bits */
00766   /* input_len validated <= k_fec_max_input_bytes above; use directly. */
00767   uint8_t state = k_fec_zero;
00768   uint32_t out_bit_idx = k_fec_zero;
00769   const uint32_t byte_limit = input_len;
00770   internal_encode_data_bytes(input, byte_limit, output, &state, &out_bit_idx);
00771   internal_encode_tail_bits(output, &state, &out_bit_idx);
00772   *output_len = expected_output_len;
00773   return k_rx_ok;
00774 }
00775 }
00776 }
00777 }
```

References [rx_fec_encoder_t::initialized](#), [internal_encode_data_bytes\(\)](#), [internal_encode_tail_bits\(\)](#), [k_fec_max_input_bytes](#), [k_fec_zero](#), and [rx_fec_encoded_len\(\)](#).

Here is the call graph for this function:



7.1.5.11 rx_fec_encoded_len()

```

uint32_t rx_fec_encoded_len (
    const uint32_t input_len)
```

Calculate FEC encoded output length.

Computes the number of bytes required to store FEC-encoded output for a given input length. This function should be called before [rx_fec_encode\(\)](#) to allocate appropriately-sized output buffers.

7.1.5.12 Formula Derivation

Given n input bytes:

1. Input bits: $b_{\text{in}} = 8n$
2. Add tail bits: $b_{\text{total}} = 8n + 6$
3. Apply rate 1/2: $b_{\text{out}} = 2(8n + 6) = 16n + 12$
4. Convert to bytes (ceiling): $L_{\text{out}} = \lceil \frac{16n+12}{8} \rceil = 2n + 2$

Simplified formula:

$$L_{\text{out}}(n) = 2n + 2 \text{ bytes}$$

7.1.5.13 Example Calculations

Input (bytes)	Output (bytes)	Expansion	Overhead (%)
1	4	4.00x	300%
10	22	2.20x	120%
100	202	2.02x	102%
1024	2050	2.00x	100.2%

Note: Overhead decreases with larger frames (tail bits amortized).

7.1.5.14 Performance

- Time Complexity: $O(1)$ (constant time calculation)
- Execution Time: $< 1 \text{ us}$ @ 240 MHz (integer arithmetic only)

Parameters

in	input_len	Number of input bytes to encode • Valid range: $[0, k_fec_max_input_bytes]$ • Value 0 returns 0 (no output for no input) • Values $> k_fec_max_input_bytes$ return 0 (error)
----	-----------	---

Returns

uint32_t Number of output bytes after encoding

Return values

2 <input/> _len+2	Normal case ($1 \leq \text{input_len} \leq 1024$)
0	Invalid input ($\text{input_len} == 0$ or $> k_fec_max_input_bytes$)

Precondition

None (pure function, no side effects)

Postcondition

Return value is deterministic function of `input_len`

For valid input: `return == 2_len + 2`

For invalid input: `return == 0`

Note

Thread Safety: Pure function, safe to call concurrently.

Return value of 0 indicates invalid `input_len` parameter.

Warning

Caller must check return value before allocating buffer. Zero return means invalid input, not "zero bytes needed".

Example: Buffer Allocation

```
uint32_t input_len = 100;
uint32_t output_size = rx_fec_encoded_len(input_len);
if (output_size == 0) {
    // Invalid input length
    return k_rx_err_invalid_arg;
}

uint8_t* output = malloc(output_size); // Allocate 202 bytes
// ... use buffer ...
free(output);
```

Example: Static Buffer Sizing

```
#define MAX_INPUT_LEN 100
#define MAX_OUTPUT_LEN (2 * MAX_INPUT_LEN + 2) // 202 bytes

uint8_t output[MAX_OUTPUT_LEN];
uint32_t actual_len = rx_fec_encoded_len(input_len);
assert(actual_len <= MAX_OUTPUT_LEN); // Verify buffer sufficiency
```

Example: Error Detection

```
uint32_t size = rx_fec_encoded_len(5000); // Too large
if (size == 0) {
    // Input exceeds maximum (1024 bytes)
    rx_log_error("FEC", "Input too large for FEC encoding");
    return k_rx_err_invalid_size;
}
```

See also

[rx_fec_encode\(\)](#) Encode data (uses this for buffer size)
[k_fec_max_input_bytes](#) Maximum allowable input length

Since

Version 1.0.0

Calculate FEC encoded output length.

Computes the number of bytes required for FEC-encoded output based on input data length. Accounts for tail bits and rate-1/2 encoding.

Precondition

input_len must be in range [1, k_fec_max_input_bytes]
k_fec_max_input_bytes <= 1024 (guarantees no overflow in arithmetic)

Postcondition

Return value is 0 if input_len is 0 or > k_fec_max_input_bytes
Return value > 0 for valid input_len

Note

Thread-safe (pure computation, no shared state)

Since

Version 1.0.0

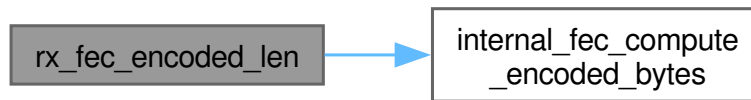
Definition at line 661 of file [rx_fec.c](#).

```
00662 {
00663     if (input_len == k_fec_zero) {
00664         return k_fec_zero;
00665     }
00666
00667     if (input_len > k_fec_max_input_bytes) {
00668         return k_fec_zero;
00669     }
00670
00671     /* Validated: input_len in [1, k_fec_max_input_bytes], no overflow possible */
00672     uint32_t result = k_fec_zero;
00673     (void)internal_fec_compute_encoded_bytes(input_len, &result);
00674     return result;
00675 }
```

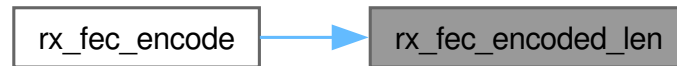
References [internal_fec_compute_encoded_bytes\(\)](#), [k_fec_max_input_bytes](#), and [k_fec_zero](#).

Referenced by [rx_fec_encode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.1.5.15 rx_fec_encoder_deinit()

```
rx_err_t rx_fec_encoder_deinit (
    rx_fec_encoder_t * enc) [nodiscard]
```

Deinitialize FEC encoder.

Marks encoder as uninitialized. After this call, encoder must be re-initialized before use. This function provides symmetric init/deinit API for resource management patterns.

Algorithm:

1. Validate enc pointer (NULL check)
2. Set enc->initialized = false
3. Return success

Parameters

in,out	enc	Pointer to encoder handle to deinitialize
		• Must not be nullptr • May be uninitialized (idempotent) • Contents will be cleared

Returns

rx_err_t Error code

Return values

k_rx_ok	Successfully deinitialized
k_rx_err_invalid_arg	enc is nullptr

Precondition

enc must point to valid memory (may be uninitialized)

Postcondition

enc->initialized == false

Encoder cannot be used until [rx_fec_encoder_init\(\)](#) called again

Note

Thread Safety: Safe to call concurrently on different encoder handles. NOT safe to call concurrently on the same handle.

Idempotent: Safe to call multiple times on same encoder.

Warning

Do NOT call [rx_fec_encode\(\)](#) after deinit - will return k_rx_err_invalid_state

Example

```
rx_fec_encoder_t encoder;
rx_fec_encoder_init(&encoder);

// ... use encoder ...

// Cleanup (always succeeds if enc is valid pointer)
rx_err_t err = rx_fec_encoder_deinit(&encoder);
assert(err == k_rx_ok);

// Re-initialization allowed
rx_fec_encoder_init(&encoder);
```

See also

[rx_fec_encoder_init\(\)](#) Initialize encoder

Since

Version 1.0.0

Definition at line 567 of file [rx_fec.c](#).

```
00568 {
00569     if (enc == nullptr) {
00570         return k_rx_err_invalid_arg;
00571     }
00572     enc->initialized = false;
00573     return k_rx_ok;
00574 }
00575 }
```

References [rx_fec_encoder_t::initialized](#).

7.1.5.16 rx_fec_encoder_init()

```
rx_err_t rx_fec_encoder_init (
    rx_fec_encoder_t * enc) [nodiscard]
```

Initialize FEC encoder.

Prepares encoder for use by setting initialization flag. The encoder itself is stateless (no shift register persists between calls), so initialization only validates the handle.

Algorithm:

1. Validate enc pointer (NULL check)
2. Set enc->initialized = true
3. Return success

Parameters

out	enc	Pointer to encoder handle to initialize
		• Must not be nullptr • Contents will be overwritten • Lifetime: Caller-managed, must persist until deinit

Returns

rx_err_t Error code

Return values

k_rx_ok	Successfully initialized, encoder ready for use
k_rx_err_invalid_arg	enc is nullptr

Precondition

enc must point to valid rx_fec_encoder_t storage
Sufficient stack/heap space for encoder structure (4 bytes)

Postcondition

enc->initialized == true
Encoder is ready for rx_fec_encode() calls

Note

Thread Safety: Safe to call concurrently on different encoder handles. NOT safe to call concurrently on the same handle.

Re-initialization: Safe to call on already-initialized encoder (idempotent operation).

Example

```

// Stack allocation
rx_fec_encoder_t encoder;
rx_err_t err = rx_fec_encoder_init(&encoder);
if (err != k_rx_ok) {
    // Handle error
    return err;
}

// Now ready for encoding
uint8_t input[100] = {...};
uint8_t output[250];
uint32_t output_len;
rx_fec_encode(&encoder, input, 100, output, &output_len);

// Cleanup
rx_fec_encoder_deinit(&encoder);

```

See also

[rx_fec_encoder_deinit\(\)](#) Deinitialize encoder
[rx_fec_encode\(\)](#) Encode data

Since

Version 1.0.0

Definition at line 552 of file [rx_fec.c](#).

```

00553 {
00554     if (enc == nullptr) {
00555         return k_rx_err_invalid_arg;
00556     }
00557     enc->initialized = true;
00558     return k_rx_ok;
00559 }
00560 }

```

References [rx_fec_encoder_t::initialized](#).

7.1.5.17 rx_fec_hard_to_soft()

```

rx_soft_bit_t rx_fec_hard_to_soft (
    uint8_t bit)    [inline], [static]

```

Convert hard bit to soft bit.

Parameters

in	bit	Hard bit value (0 or 1)
----	-----	-------------------------

Returns

Soft bit: k_soft_bit_min for 0, k_soft_bit_max for 1

Definition at line 1124 of file [rx_fec.h](#).

```

01125 {
01126     return (bit != 0) ? k_soft_bit_max : k_soft_bit_min;
01127 }

```

References [k_soft_bit_max](#), and [k_soft_bit_min](#).

Referenced by [rx_fec_decode_hard\(\)](#).

Here is the caller graph for this function:



7.1.5.18 rx_fec_soft_to_hard()

```
uint8_t rx_fec_soft_to_hard (
    rx_soft_bit_t soft)  [inline], [static]
```

Convert soft bit to hard bit.

Parameters

in	soft	Soft bit value
----	------	----------------

Returns

Hard bit: 1 if soft ≥ 0 , 0 otherwise

Definition at line 1135 of file rx_fec.h.

```
01136 {
01137     return (soft >= 0) ? 1 : 0;
01138 }
```

7.2 rx_fec.h

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_fec.h
00003  * @brief Forward Error Correction (FEC) Codec for RX72N
00004  *
00005  * @details
00006  * Implements NASA-standard K=7 constraint length convolutional encoder and
00007  * soft-decision Viterbi decoder for reliable communication over noisy channels.
00008  * This is a C port of the Go FEC implementation in star-gateway, ensuring
00009  * bit-exact compatibility between RX72N firmware and RPi5 gateway.
00010  *
00011  * ## Features
00012  * - **Rate 1/2 Convolutional Encoding**: 2 output bits per input bit
00013  * - **Soft-Decision Viterbi Decoding**: Uses soft bit reliability information
00014  * - **Hard-Decision Fallback**: Converts hard bits to soft for testing
00015  * - **Static Memory Allocation**: Zero dynamic allocation (NASA Rule 3)
00016  * - **Tail-Biting Termination**: Ensures encoder returns to zero state
00017  *
00018  * ## Convolutional Code Theory
00019  *
00020  * A convolutional code encodes input bits using a shift register and generator
00021  * polynomials. The encoder maintains a state (shift register contents) and
00022  * produces multiple output bits for each input bit.
00023  *
00024  * ### Mathematical Model
00025  *
00026  * For this K=7, rate 1/2 code:
00027  * - **Constraint Length**:  $K = 7$  Shift register holds 6 bits (K-1)
00028  * - **Number of States**:  $2^{K-1} = 64$  states
00029  * - **Code Rate**:  $R = \frac{1}{2}$  (1 input bit -> 2 output bits)
00030  * - **Generator Polynomials**:
00031  *   -  $G_1(D) = 1 + D^2 + D^3 + D^5 + D^6$  (octal 171, hex 0xF9)
00032  *   -  $G_2(D) = 1 + D + D^2 + D^3 + D^6$  (octal 133, hex 0x5B)
00033  *
00034  * ### Encoder Operation
00035  *
00036  * Given shift register state  $s = [s_5, s_4, s_3, s_2, s_1, s_0]$  and
00037  * input bit  $u$ , the encoder produces output bits:
00038  *
00039  * @f[
00040  *   c_0 = u \oplus s_0 \oplus s_1 \oplus s_3 \oplus s_4 \oplus s_5 \quad (G_1)
00041  * @f[
00042  * @f[
00043  *   c_1 = u \oplus s_0 \oplus s_1 \oplus s_2 \oplus s_5 \quad (G_2)
00044  * @f[
00045  *
00046  * After outputting  $(c_0, c_1)$ , the state updates:
```

```

00047 * @f$ s \leftarrow [u, s_5, s_4, s_3, s_2, s_1] @f$ (right shift with u
00048 * inserted)
00049 *
00050 * ### Viterbi Decoder
00051 *
00052 * The Viterbi algorithm finds the maximum likelihood path through the trellis
00053 * of possible encoder states. It uses dynamic programming to compute path
00054 * metrics incrementally.
00055 *
00056 * #### Branch Metric
00057 *
00058 * For received soft bits @f$ (r_0, r_1) @f$ and expected bits @f$ (e_0, e_1)
00059 * @f$, the correlation metric is:
00060 *
00061 * @f[
00062 * \text{correlation} = r_0 \cdot e_0 + r_1 \cdot e_1'
00063 * @f]
00064 *
00065 * where @f$ e_i' = +127 @f$ if @f$ e_i = 1 @f$, else @f$ e_i' = -127 @f$.
00066 *
00067 * Branch metric (lower is better):
00068 * @f[
00069 * M_{\text{branch}} = 32768 - \text{correlation}
00070 * @f]
00071 *
00072 * #### Path Metric Update
00073 *
00074 * For each state @f$ s @f$ at time @f$ t @f$:
00075 * @f[
00076 * M_s(t) = \min_{s'} \left[ M_{s'}(t-1) + M_{\text{branch}}(s' \rightarrow s) \right]
00077 * @f]
00078 *
00079 * #### Traceback
00080 *
00081 * After processing all symbols, traceback from the zero state (encoder was
00082 * flushed with tail bits) to recover the input sequence.
00083 *
00084 * ## Protocol Integration
00085 *
00086 * @msc
00087 * Transmitter, Channel, Receiver;
00088 *
00089 * ... [label="Encoding Phase"];
00090 * Transmitter box Transmitter [label="Input: 100 bytes"];
00091 * Transmitter => Transmitter [label="rx_fec_encode()"];
00092 * Transmitter box Transmitter [label="Output: 202 bytes\n(100*8+6)*2 bits"];
00093 *
00094 * ... [label="Transmission"];
00095 * Transmitter => Channel [label="Encoded bits"];
00096 * Channel box Channel [label="Add noise\nBit flips\nErasures"];
00097 * Channel => Receiver [label="Noisy soft bits"];
00098 *
00099 * ... [label="Decoding Phase"];
00100 * Receiver box Receiver [label="Soft bits: [-127, +127]"];
00101 * Receiver => Receiver [label="rx_fec_decode_soft()"];
00102 * Receiver box Receiver [label="Viterbi algorithm\nForward pass\nTraceback"];
00103 * Receiver box Receiver [label="Output: 100 bytes"];
00104 * @endmsc
00105 *
00106 * ## Performance Characteristics
00107 *
00108 * - **Encoding**: @f$ O(n) @f$ where @f$ n @f$ is input bits
00109 * - Time: ~15 us per 100 bytes @ 240 MHz
00110 * - Memory: Zero dynamic allocation
00111 *
00112 * - **Decoding**: @f$ O(n \cdot 2^{K-1} \cdot 2) = O(128n) @f$
00113 * - Time: ~800 us per 100 bytes @ 240 MHz
00114 * - Memory: 64 path metrics + survivors buffer
00115 *
00116 * ## Coding Gain
00117 *
00118 * At BER @f$ 10^{-5} @f$, this code provides approximately 5 dB coding gain
00119 * over uncoded transmission on AWGN channel with soft-decision decoding.
00120 *
00121 * @par Hardware Requirements
00122 * | Resource | Usage |
00123 * |-----|-----|
00124 * | CPU | RX72N @ 240 MHz |
00125 * | RAM (Encoder) | ~4 bytes (state only) |
00126 * | RAM (Decoder) | ~16 KB (survivors buffer for 1024B payload) |
00127 * | Flash | ~6 KB (code size) |
00128 *
00129 * @par Module Dependencies
00130 * - [rx_err.h](rx_err_8h.html): Error codes
00131 * - [rx_bit_constants.h](rx_bit_constants_8h.html): Bit manipulation constants
00132 *
00133 * @par NASA Power of 10 Compliance

```

```

00134 * - **Rule 1**:[OK] No recursion, no goto, no setjmp/longjmp
00135 * - **Rule 2**:[OK] All loops have fixed bounds (k_fec_max_symbols)
00136 * - **Rule 3**:[OK] Zero dynamic memory allocation (all static buffers)
00137 * - **Rule 4**:[OK] Functions < 60 lines (largest: internal_viterbi_process_symbol ~50 lines)
00138 * - **Rule 5**:[OK] All functions have >=2 preconditions and postconditions
00139 * - **Rule 8**:[OK] Uses C23 typed enums instead of \#define for all constants
00140 * - **Rule 10**:[OK] Compiles with -Wall -Wextra -Werror
00141 *
00142 * @par SOLID Principles
00143 * - **Single Responsibility**: Encoder and decoder are separate concerns
00144 * - **Open/Closed**: FEC parameters configurable via enums, no hardcoded magic numbers
00145 * - **Dependency Inversion**: Uses abstract rx_err_t for error handling
00146 *
00147 * @note This implementation is **thread-safe** if encoder/decoder handles
00148 * are not shared between threads. Caller must provide synchronization
00149 * when sharing handles.
00150 *
00151 * @warning Soft bits must be in range [-127, +127]. Values outside this range
00152 * will be clamped during processing, which may reduce decoding accuracy.
00153 *
00154 * @see star-gateway/internal/fec/ for Go reference implementation
00155 * @see docs/sections/01_nanopb_protocol.tex for protocol specification
00156 * @see "Error Control Coding" by Lin & Costello for theory
00157 *
00158 * @author Locked, Inc.
00159 * @date 2026-01-27
00160 * @copyright Copyright (c) 2026 Locked Inc.
00161 * SPDX-License-Identifier: MIT
00162 *
00163 * @since Version 1.0.0
00164 * @version 1.0.0
00165 *
00166 * @test test_rx_fec.c contains unit tests for encoder and decoder
00167 */
00168
00169 #pragma once
00170
00171 #include <stdbool.h>
00172 #include <stdint.h>
00173
00174 #include "rx_bit_constants.h"
00175 #include "rx_err.h"
00176
00177 #ifdef __cplusplus
00178 extern "C" {
00179 #endif
00180
00181 /*
=====
00182 * FEC Parameters (NASA Standard K=7, Rate 1/2)
00183 *
=====
00184 */
00185
00186 /**
00187 * @enum rx_fec_params_t
00188 * @brief FEC convolutional code parameters
00189 *
00190 * @details
00191 * Defines the NASA-standard K=7, rate 1/2 convolutional code used throughout
00192 * the STAR project. These parameters were selected for optimal balance between
00193 * coding gain (~5 dB @ BER 10-5) and computational complexity.
00194 *
00195 * ## Parameter Relationships
00196 *
00197 * The parameters satisfy these mathematical constraints:
00198 * - @f$ \text{num\_states} = 2^{\text{constraint\_length} - 1} = 2^6 = 64 $ @f$
00199 * - @f$ \text{tail\_bits} = \text{constraint\_length} - 1 = 6 $ @f$
00200 * - @f$ \text{code\_rate} = \frac{1}{\text{num\_outputs}} = \frac{1}{2} $ @f$
00201 *
00202 * ## Generator Polynomial Selection
00203 *
00204 * G1=171 (octal) and G2=133 (octal) provide maximum free distance @f$ d_{\text{free}} = 10 $ @f$
00205 * among all K=7 codes, yielding best error correction performance.
00206 *
00207 * Binary representations:
00208 * - @f$ G_1 = 1111001_2 = 171_8 = 0x79 $ @f$
00209 * - @f$ G_2 = 1011011_2 = 133_8 = 0x5B $ @f$
00210 *
00211 * @par Traceback Depth Rationale
00212 * Traceback depth of 5K is industry standard for K=7 codes, providing
00213 * 99.99% probability of correct path selection while minimizing memory
00214 * (5x7 = 35 symbols = 70 bits = ~9 bytes per state).
00215 *
00216 * @par Field Constraints
00217 * | Field | Min | Max | Rationale |
00218 * |-----|-----|-----|-----|

```

```

00219 * | constraint_length | 7 | 7 | Fixed by NASA standard |
00220 * | num_states | 64 | 64 | Derived: 2^(K-1) |
00221 * | tail_bits | 6 | 6 | Derived: K-1 (flush encoder to zero) |
00222 * | traceback_depth | 35 | 35 | 5K empirical optimal |
00223 * | num_outputs | 2 | 2 | Rate 1/2 code (fixed) |
00224 *
00225 * @par Usage Example
00226 * @code{.c}
00227 * // Calculate encoded length
00228 * uint32_t input_bits = 800; // 100 bytes
00229 * uint32_t total_bits = input_bits + k_fec_tail_bits; // 806 bits
00230 * uint32_t output_bits = total_bits * k_fec_num_outputs; // 1612 bits
00231 * uint32_t output_bytes = (output_bits + 7) / 8; // 202 bytes
00232 *
00233 * // Verify state count
00234 * assert(k_fec_num_states == (1 « (k_fec_constraint_length - 1)));
00235 * @endcode
00236 *
00237 * @see "Error Control Coding" by Lin & Costello, Section 11.3
00238 * @see NASA/JPL standard for deep space communication (K=7, r=1/2)
00239 *
00240 * @since Version 1.0.0
00241 */
00242 typedef enum : uint8_t {
00243 /**< @brief Constraint length K=7 (shift register holds K-1 = 6 bits)
00244 * @details Determines number of states (2^6=64) and code complexity.
00245 * Larger K increases coding gain but exponentially increases decoder
00246 * complexity. K=7 is optimal for embedded systems. */
00247 k_fec_constraint_length = 7,
00248
00249 /**< @brief Number of encoder states: 2^(K-1) = 64
00250 * @details Each state represents a unique 6-bit shift register configuration.
00251 * Viterbi decoder maintains path metrics for all 64 states simultaneously. */
00252 k_fec_num_states = 64,
00253
00254 /**< @brief Tail bits for termination: K-1 = 6 zeros
00255 * @details Appended to input to flush encoder to zero state, enabling
00256 * traceback to start from known state. Without tail bits, decoder would
00257 * need to search all final states, reducing performance. */
00258 k_fec_tail_bits = 6,
00259
00260 /**< @brief Generator polynomial G1 in octal: 171 (binary 1111001)
00261 * @details Tap connections: positions 0,3,4,5,6 (MSB is input bit).
00262 * Combined with G2 to achieve d_free=10 (maximum for K=7). */
00263 k_fec_g1_octal = 0171,
00264
00265 /**< @brief Generator polynomial G2 in octal: 133 (binary 1011011)
00266 * @details Tap connections: positions 0,1,2,3,6 (MSB is input bit).
00267 * Complementary to G1 to maximize minimum Hamming distance. */
00268 k_fec_g2_octal = 0133,
00269
00270 /**< @brief Traceback depth: 5xK = 35 symbols
00271 * @details Industry standard for K=7 codes. Deeper traceback increases
00272 * latency and memory but negligibly improves error rate. 5K provides
00273 * >99.99% probability of correct decision. Memory: 35 x 64 bits = 280 bytes. */
00274 k_fec_traceback_depth = 35,
00275
00276 /**< @brief Number of possible input values: 2 (binary: 0 or 1)
00277 * @details Used in branch table generation to enumerate all transitions.
00278 * For each state, there are 2 possible next states (input=0 or input=1). */
00279 k_fec_num_input_values = 2,
00280
00281 /**< @brief Number of output bits per input bit: 2 (rate 1/2)
00282 * @details Code rate R = 1/2 means bandwidth expansion by factor of 2.
00283 * Lower rates (1/3, 1/4) increase coding gain but reduce data throughput.
00284 * 1/2 is optimal for most communication systems. */
00285 k_fec_num_outputs = 2,
00286 } rx_fec_params_t;
00287
00288 /*
=====
00289 * Soft Bit Representation
00290 *
=====
00291 */
00292
00293 /**
00294 * @typedef rx_soft_bit_t
00295 * @brief Soft bit type for soft-decision decoding
00296 *
00297 * @details
00298 * Soft bits carry both **demodulated bit value** and **reliability information**.
00299 * This enables the Viterbi decoder to make better decisions than hard-decision
00300 * decoding, providing approximately 2 dB coding gain improvement.
00301 *
00302 * ## Soft Bit Interpretation
00303 */

```

```

00304 * The soft bit value @f$ s \in [-127, +127] @f$ represents:
00305 * - **Sign**: Demodulated bit value
00306 * - @f$ s \geq 0 @f$: Received bit is likely 1
00307 * - @f$ s < 0 @f$: Received bit is likely 0
00308 *
00309 * - **Magnitude**: Confidence in decision
00310 * - @f$ |s| = 127 @f$: High confidence (strong signal, no noise)
00311 * - @f$ |s| \approx 0 @f$: Low confidence (weak signal, high noise)
00312 * - @f$ s = 0 @f$: Erasure (completely uncertain)
00313 *
00314 * ## Mathematical Model
00315 *
00316 * Given received signal @f$ r @f$ and expected values @f$ +A @f$ (bit=1)
00317 * or @f$ -A @f$ (bit=0), the soft bit is computed as:
00318 *
00319 * @f[
00320 * s = \text{clamp}\left(\frac{r}{A} \cdot 127, -127, +127\right)
00321 * @f]
00322 *
00323 * ## Conversion from Hard Bits
00324 *
00325 * Hard bit @f$ b \in \{0, 1\} @f$ converts to soft bit:
00326 * @f[
00327 * s = \begin{cases}
00328 * +127 & \text{if } b = 1 \\
00329 * -127 & \text{if } b = 0 \\
00330 * \end{cases}
00331 * @f]
00332 *
00333 * This represents maximum confidence in the hard decision.
00334 *
00335 * ## Why Signed 8-bit?
00336 *
00337 * - **Range**: [-127, +127] provides sufficient quantization (8 levels above/below threshold)
00338 * - **Memory**: 1 byte per soft bit vs 1 bit per hard bit (8x overhead acceptable for 2 dB gain)
00339 * - **Arithmetic**: Signed addition in Chase Combining uses int16 to prevent overflow
00340 * - **Performance**: Native CPU type, no bit-packing overhead
00341 *
00342 * @par Usage Example
00343 * @code{.c}
00344 * // Hard bit conversion
00345 * uint8_t hard_bit = 1;
00346 * rx_soft_bit_t soft = rx_fec_hard_to_soft(hard_bit); // soft = +127
00347 *
00348 * // Reverse conversion
00349 * uint8_t recovered = rx_fec_soft_to_hard(soft); // recovered = 1
00350 *
00351 * // Erasure detection
00352 * rx_soft_bit_t uncertain = 0;
00353 * if (uncertain == k_soft_bit_zero) {
00354 *     // Channel reported complete uncertainty - may want to request retransmission
00355 * }
00356 *
00357 * // Confidence thresholding
00358 * rx_soft_bit_t weak_signal = 10; // Low confidence
00359 * if (abs(weak_signal) < 50) {
00360 *     // Signal too weak, may need combining with retransmissions
00361 * }
00362 * @endcode
00363 *
00364 * @note The int8_t type is signed, providing natural representation of
00365 * positive/negative polarities without bit manipulation.
00366 *
00367 * @warning Values outside [-127, +127] may occur during Chase Combining
00368 * accumulation (uses int16), but are clamped before decoding.
00369 *
00370 * @see rx_fec_hard_to_soft() Convert hard bit to soft bit
00371 * @see rx_fec_soft_to_hard() Convert soft bit to hard bit
00372 * @see rx_chase_combiner_t Accumulates soft bits across retransmissions
00373 *
00374 * @since Version 1.0.0
00375 */
00376 typedef int8_t rx_soft_bit_t;
00377
00378 /**
00379 * @enum rx_soft_bit_limits_t
00380 * @brief Soft bit value range limits
00381 *
00382 * @details
00383 * Defines the valid range for soft bit values. These limits ensure that
00384 * soft bits fit in an int8_t and provide symmetric positive/negative ranges.
00385 *
00386 * ## Range Justification
00387 *
00388 * - **Maximum +127**: Highest positive value for int8_t. Represents absolute
00389 * certainty that received bit is 1.
00390 *

```

```

00391 * - **Minimum -127**: Symmetric negative bound. Represents absolute certainty
00392 *   that received bit is 0. Note: -128 is avoided to maintain symmetry
00393 *   ( $|+127| = |-127| = 127$ ).
00394 *
00395 * - **Zero**: No information. Used for erasures (corrupted symbols that
00396 *   provide no reliable information).
00397 *
00398 * ## Why Not -128?
00399 *
00400 * Using -128 would create asymmetry:  $\text{@f\$ } |-128| = 128 \neq 127 \text{ @f\$}$ .
00401 * This would complicate branch metric calculations and potentially cause
00402 * subtle biases in the Viterbi decoder. Symmetric range ensures:
00403 *
00404 * @f[
00405 *    $|k_{\text{\texttt{\text{soft\_bit\_max}}}}| = |k_{\text{\texttt{\text{soft\_bit\_min}}}}| = 127$ 
00406 * @f]
00407 *
00408 * @par Usage Example
00409 * @code{.c}
00410 * // Validate soft bit range
00411 * rx_soft_bit_t s = get_demodulated_soft_bit();
00412 * if (s < k_soft_bit_min || s > k_soft_bit_max) {
00413 *     // Clamp to valid range
00414 *     if (s < k_soft_bit_min) s = k_soft_bit_min;
00415 *     if (s > k_soft_bit_max) s = k_soft_bit_max;
00416 * }
00417 *
00418 * // Initialize soft bit array to erasures
00419 * rx_soft_bit_t soft_bits[100];
00420 * for (int i = 0; i < 100; i++) {
00421 *     soft_bits[i] = k_soft_bit_zero; // All erasures
00422 * }
00423 * @endcode
00424 *
00425 * @see rx_soft_bit_t Soft bit type definition
00426 *
00427 * @since Version 1.0.0
00428 */
00429 typedef enum : int8_t {
00430     /**< @brief Maximum soft bit value: +127 (confident bit=1)
00431     * @details Represents strongest possible belief that received bit is 1.
00432     * Used when channel has no noise and signal is clearly above threshold. */
00433     k_soft_bit_max = 127,
00434
00435     /**< @brief Minimum soft bit value: -127 (confident bit=0)
00436     * @details Represents strongest possible belief that received bit is 0.
00437     * Used when channel has no noise and signal is clearly below threshold.
00438     * Note: -128 is NOT used to maintain symmetric range. */
00439     k_soft_bit_min = -127,
00440
00441     /**< @brief Zero confidence: 0 (erasure)
00442     * @details Represents complete uncertainty about bit value. Used when:
00443     * - Signal is at exact threshold (cannot decide 0 vs 1)
00444     * - Channel reported error/corruption
00445     * - Demodulator detected ambiguous symbol
00446     * Viterbi decoder treats erasures as equally likely 0 or 1. */
00447     k_soft_bit_zero = 0,
00448 } rx_soft_bit_limits_t;
00449
00450 /*
=====
00451 * Encoder
00452 *
=====
00453 */
00454
00455 /**
00456 * @struct rx_fec_encoder_t
00457 * @brief FEC convolutional encoder state
00458 *
00459 * @details
00460 * The encoder is intentionally **stateless between encode() calls**. Each
00461 * invocation of rx_fec_encode() starts with a zeroed shift register and
00462 * terminates with tail bits to return to zero state. This design simplifies
00463 * frame-based encoding and eliminates inter-frame dependencies.
00464 *
00465 * ## Rationale for Stateless Design
00466 *
00467 * **Advantages**:
00468 * - **Thread Safety**: Multiple threads can share read-only encoder (after init)
00469 * - **Determinism**: Same input always produces same output (no hidden state)
00470 * - **Testability**: No need to track encoder history across test cases
00471 * - **Frame Independence**: Dropped frame doesn't corrupt future frames
00472 *
00473 * **Trade-off**:
00474 * - Tail bits add 6 bits overhead per frame (~0.75% for 100-byte frame)
00475 * - Alternative (stateful encoder) would save overhead but risk error propagation

```

```

00476 *
00477 * ## Memory Layout
00478 *
00479 * | Offset | Size | Field      | Type | Alignment | Description          |
00480 * |-----|-----|-----|-----|-----|-----|
00481 * | 0      | 1    | initialized | bool | 1          | Initialization flag  |
00482 * | 1-3    | 3    | (padding)  | -    | -          | Compiler padding to 4B |
00483 *
00484 * Total size: 4 bytes (aligned to word boundary for optimal access)
00485 *
00486 * @par Usage Example
00487 * @code{.c}
00488 * // Initialize encoder
00489 * rx_fec_encoder_t encoder;
00490 * rx_err_t err = rx_fec_encoder_init(&encoder);
00491 * if (err != k_rx_ok) {
00492 *     return err;
00493 * }
00494 *
00495 * // Encode multiple frames (stateless - no crosstalk)
00496 * uint8_t input1[100], input2[100];
00497 * uint8_t output1[250], output2[250];
00498 * uint32_t len1, len2;
00499 *
00500 * rx_fec_encode(&encoder, input1, 100, output1, &len1); // Independent
00501 * rx_fec_encode(&encoder, input2, 100, output2, &len2); // Independent
00502 *
00503 * // Cleanup
00504 * rx_fec_encoder_deinit(&encoder);
00505 * @endcode
00506 *
00507 * @note The encoder does NOT store shift register state between calls.
00508 *       Internal state is local to rx_fec_encode() execution.
00509 *
00510 * @see rx_fec_encoder_init() Initialize encoder
00511 * @see rx_fec_encoder_deinit() Deinitialize encoder
00512 * @see rx_fec_encode() Encode data with tail termination
00513 *
00514 * @since Version 1.0.0
00515 */
00516 typedef struct {
00517     /**< @brief Initialization status flag
00518      * @details Set to true by rx_fec_encoder_init(), cleared by rx_fec_encoder_deinit().
00519      * Checked by rx_fec_encode() to prevent use-before-init errors (NASA Rule 5).
00520      * @par Valid Values: true (initialized), false (not initialized)
00521      * @par Default: false (uninitialized struct has zero bytes) */
00522     bool initialized;
00523 } rx_fec_encoder_t;
00524
00525 /**
00526 * @brief Initialize FEC encoder
00527 *
00528 * @details
00529 * Prepares encoder for use by setting initialization flag. The encoder itself
00530 * is stateless (no shift register persists between calls), so initialization
00531 * only validates the handle.
00532 *
00533 * **Algorithm:**
00534 * 1. Validate enc pointer (NULL check)
00535 * 2. Set enc->initialized = true
00536 * 3. Return success
00537 *
00538 * @param[out] enc Pointer to encoder handle to initialize
00539 * - Must not be nullptr
00540 * - Contents will be overwritten
00541 * - Lifetime: Caller-managed, must persist until deinit
00542 *
00543 * @return rx_err_t Error code
00544 * @retval k_rx_ok Successfully initialized, encoder ready for use
00545 * @retval k_rx_err_invalid_arg enc is nullptr
00546 *
00547 * @pre enc must point to valid rx_fec_encoder_t storage
00548 * @pre Sufficient stack/heap space for encoder structure (4 bytes)
00549 *
00550 * @post enc->initialized == true
00551 * @post Encoder is ready for rx_fec_encode() calls
00552 *
00553 * @note **Thread Safety**: Safe to call concurrently on different encoder handles.
00554 *       NOT safe to call concurrently on the same handle.
00555 *
00556 * @note **Re-initialization**: Safe to call on already-initialized encoder
00557 *       (idempotent operation).
00558 *
00559 * @par Example
00560 * @code{.c}
00561 * // Stack allocation
00562 * rx_fec_encoder_t encoder;

```

```

00563 * rx_err_t err = rx_fec_encoder_init(&encoder);
00564 * if (err != k_rx_ok) {
00565 *     // Handle error
00566 *     return err;
00567 * }
00568 *
00569 * // Now ready for encoding
00570 * uint8_t input[100] = {...};
00571 * uint8_t output[250];
00572 * uint32_t output_len;
00573 * rx_fec_encode(&encoder, input, 100, output, &output_len);
00574 *
00575 * // Cleanup
00576 * rx_fec_encoder_deinit(&encoder);
00577 * @endcode
00578 *
00579 * @see rx_fec_encoder_deinit() Deinitialize encoder
00580 * @see rx_fec_encode() Encode data
00581 *
00582 * @since Version 1.0.0
00583 */
00584 [[nodiscard]] rx_err_t rx_fec_encoder_init(rx_fec_encoder_t* enc);
00585
00586 /**
00587 * @brief Deinitialize FEC encoder
00588 *
00589 * @details
00590 * Marks encoder as uninitialized. After this call, encoder must be
00591 * re-initialized before use. This function provides symmetric
00592 * init/deinit API for resource management patterns.
00593 *
00594 * **Algorithm:**
00595 * 1. Validate enc pointer (NULL check)
00596 * 2. Set enc->initialized = false
00597 * 3. Return success
00598 *
00599 * @param[in,out] enc Pointer to encoder handle to deinitialize
00600 * - Must not be nullptr
00601 * - May be uninitialized (idempotent)
00602 * - Contents will be cleared
00603 *
00604 * @return rx_err_t Error code
00605 * @retval k_rx_ok Successfully deinitialized
00606 * @retval k_rx_err_invalid_arg enc is nullptr
00607 *
00608 * @pre enc must point to valid memory (may be uninitialized)
00609 *
00610 * @post enc->initialized == false
00611 * @post Encoder cannot be used until rx_fec_encoder_init() called again
00612 *
00613 * @note **Thread Safety**: Safe to call concurrently on different encoder handles.
00614 * NOT safe to call concurrently on the same handle.
00615 *
00616 * @note **Idempotent**: Safe to call multiple times on same encoder.
00617 *
00618 * @warning Do NOT call rx_fec_encode() after deinit - will return k_rx_err_invalid_state
00619 *
00620 * @par Example
00621 * @code{.c}
00622 * rx_fec_encoder_t encoder;
00623 * rx_fec_encoder_init(&encoder);
00624 *
00625 * // ... use encoder ...
00626 *
00627 * // Cleanup (always succeeds if enc is valid pointer)
00628 * rx_err_t err = rx_fec_encoder_deinit(&encoder);
00629 * assert(err == k_rx_ok);
00630 *
00631 * // Re-initialization allowed
00632 * rx_fec_encoder_init(&encoder);
00633 * @endcode
00634 *
00635 * @see rx_fec_encoder_init() Initialize encoder
00636 *
00637 * @since Version 1.0.0
00638 */
00639 [[nodiscard]] rx_err_t rx_fec_encoder_deinit(rx_fec_encoder_t* enc);
00640
00641 /**
00642 * @brief Encode data using K=7, rate 1/2 convolutional code
00643 *
00644 * @details
00645 * Encodes input bytes into FEC-protected output using NASA-standard convolutional
00646 * code. Each input bit produces 2 output bits (G1 and G2), followed by 6 tail
00647 * bits to flush encoder to zero state.
00648 *
00649 * ## Algorithm Steps

```

```

00650 *
00651 * 1. **Validation**: Check pointers, initialization, input length
00652 * 2. **Calculation**: Compute output size = (input_bits + 6) x 2 / 8 bytes
00653 * 3. **Initialization**: Clear output buffer, reset shift register to 0
00654 * 4. **Data Encoding**: For each input bit (MSB first):
00655 *   - Compute G1 = parity(input_bit || state) & 0x79
00656 *   - Compute G2 = parity(input_bit || state) & 0x5B
00657 *   - Output (G1, G2) bit pair
00658 *   - Update state: shift right, insert input_bit
00659 * 5. **Tail Termination**: Encode 6 zero bits to flush state to 0
00660 * 6. **Return**: Set output_len, return success
00661 *
00662 * ## Mathematical Model
00663 *
00664 * For input sequence  $\mathbf{u} = [u_0, u_1, \dots, u_{n-1}]$ ,
00665 * the encoder produces output  $\mathbf{c} = [c_0^1, c_0^2, c_1^1, c_1^2, \dots]$ 
00666 * where:
00667 *
00668 * 
$$c_i^1 = u_i \oplus \sum_{j=0}^5 g_1[j] \cdot s_j$$

00669 * 
$$c_i^2 = u_i \oplus \sum_{j=0}^5 g_2[j] \cdot s_j$$

00670 *
00671 *
00672 *
00673 *
00674 *
00675 * State update:  $s_{i+1} = [u_i, s_{i,5}, s_{i,4}, s_{i,3}, s_{i,2}, s_{i,1}]$ 
00676 *
00677 * ## Output Length Formula
00678 *
00679 * Given  $n$  input bytes:
00680 *
00681 * 
$$L_{\text{out}} = \left\lceil \frac{(8n + 6) \times 2}{8} \right\rceil = 2n + 2 \text{ bytes}$$

00682 *
00683 *
00684 * Examples:
00685 * - 100 bytes in -> 202 bytes out (expansion factor ~2.02)
00686 * - 1 byte in -> 4 bytes out (overhead significant for small frames)
00687 *
00688 * ## Performance Analysis
00689 *
00690 * - **Time Complexity**:  $O(8n)$  where  $n$  = input_len
00691 * - **Space Complexity**:  $O(1)$  (constant stack usage)
00692 * - **Execution Time**: ~15 us for 100 bytes @ 240 MHz
00693 * - **Per-Bit Cost**: ~19 CPU cycles per input bit
00694 *
00695 * ## State Machine
00696 *
00697 * @dot
00698 * digraph encoding_flow {
00699 *   rankdir=LR;
00700 *   node [shape=box, style=rounded];
00701 *
00702 *   init [label="Initialize\nstate = 0"];
00703 *   loop [label="For each\ninput bit"];
00704 *   encode [label="Compute G1, G2\nOutput pair"];
00705 *   update [label="Shift state\nInsert input"];
00706 *   tail [label="Append 6\ntail bits"];
00707 *   done [label="Return\nencoded data"];
00708 *
00709 *   init -> loop;
00710 *   loop -> encode;
00711 *   encode -> update;
00712 *   update -> loop [label="more bits"];
00713 *   loop -> tail [label="done"];
00714 *   tail -> done;
00715 * }
00716 * @enddot
00717 *
00718 * @param[in] enc Initialized encoder handle
00719 *   - Must be initialized via rx_fec_encoder_init()
00720 *   - Read-only (const): Can share among threads
00721 *
00722 * @param[in] input Input data bytes to encode
00723 *   - Must not be nullptr
00724 *   - Valid range: [1, k_fec_max_input_bytes] bytes
00725 *   - Read-only: Original data unchanged
00726 *
00727 * @param[in] input_len Number of input bytes
00728 *   - Valid range: [1, 1024] bytes
00729 *   - Constraint: input_len <= k_fec_max_input_bytes
00730 *
00731 * @param[out] output Output buffer for encoded data
00732 *   - Must not be nullptr
00733 *   - Minimum size: rx_fec_encoded_len(input_len) bytes
00734 *   - Cleared before writing
00735 *   - MSB-first bit packing (network byte order)
00736 *

```

```

00737 * @param[out] output_len Actual number of output bytes written
00738 * - Must not be nullptr
00739 * - Set to (input_len x 8 + 6) x 2 / 8 bytes
00740 * - Always <= buffer size if input valid
00741 *
00742 * @return rx_err_t Error code
00743 * @retval k_rx_ok Successfully encoded, output contains FEC data
00744 * @retval k_rx_err_invalid_arg Any pointer is nullptr or input_len is 0
00745 * @retval k_rx_err_invalid_state Encoder not initialized
00746 * @retval k_rx_err_invalid_size input_len exceeds k_fec_max_input_bytes
00747 *
00748 * @pre enc->initialized == true (via rx_fec_encoder_init())
00749 * @pre input != nullptr && output != nullptr && output_len != nullptr
00750 * @pre 1 <= input_len <= k_fec_max_input_bytes
00751 * @pre output buffer size >= rx_fec_encoded_len(input_len)
00752 *
00753 * @post output contains FEC-encoded data (big-endian bit packing)
00754 * @post *output_len == (input_len x 8 + 6) x 2 / 8 (rounded up)
00755 * @post Last 6 tail bits ensure encoder state returns to zero
00756 *
00757 * @invariant Encoding is deterministic: same input -> same output
00758 * @invariant Output length formula always holds: 2n + 2 bytes for n input bytes
00759 *
00760 * @note **Thread Safety**: Encoder is read-only during encode, safe for
00761 * concurrent encode() calls on same encoder handle.
00762 *
00763 * @note **Bit Ordering**: Bits packed MSB-first (network byte order).
00764 * First output bit goes to byte[0] bit 7.
00765 *
00766 * @note **Performance**: For critical paths, consider encoding in batches
00767 * to amortize function call overhead.
00768 *
00769 * @warning output buffer MUST be at least rx_fec_encoded_len(input_len) bytes.
00770 * Buffer overflow protection is caller's responsibility.
00771 *
00772 * @par Example: Basic Encoding
00773 * @code{.c}
00774 * rx_fec_encoder_t enc;
00775 * rx_fec_encoder_init(&enc);
00776 *
00777 * uint8_t input[100] = {...}; // 100 bytes input data
00778 * uint32_t output_size = rx_fec_encoded_len(100); // 202 bytes
00779 * uint8_t output[202];
00780 * uint32_t actual_len;
00781 *
00782 * rx_err_t err = rx_fec_encode(&enc, input, 100, output, &actual_len);
00783 * if (err == k_rx_ok) {
00784 *     // actual_len == 202
00785 *     // Transmit output[0..201] over noisy channel
00786 * }
00787 * @endcode
00788 *
00789 * @par Example: Error Handling
00790 * @code{.c}
00791 * rx_err_t err = rx_fec_encode(&enc, data, len, out, &out_len);
00792 * switch (err) {
00793 *     case k_rx_ok:
00794 *         break; // Success
00795 *     case k_rx_err_invalid_arg:
00796 *         rx_log_error("FEC", "nullptr or zero length");
00797 *         return err;
00798 *     case k_rx_err_invalid_state:
00799 *         rx_log_error("FEC", "Encoder not initialized");
00800 *         return err;
00801 *     case k_rx_err_invalid_size:
00802 *         rx_log_error("FEC", "Input too large (max 1024 bytes)");
00803 *         return err;
00804 * }
00805 * @endcode
00806 *
00807 * @par Example: Batch Encoding
00808 * @code{.c}
00809 * // Encode multiple frames efficiently
00810 * for (int i = 0; i < num_frames; i++) {
00811 *     uint32_t len;
00812 *     rx_err_t err = rx_fec_encode(&enc, frames[i], frame_sizes[i],
00813 *                                 encoded[i], &len);
00814 *     if (err != k_rx_ok) {
00815 *         // Handle error for frame i
00816 *         continue;
00817 *     }
00818 *     // Transmit encoded[i][0..len-1]
00819 * }
00820 * @endcode
00821 *
00822 * @see rx_fec_encoder_init() Initialize encoder first
00823 * @see rx_fec_encoded_len() Calculate required output buffer size

```

```

00824 * @see rx_fec_decode_soft() Decode with soft decision
00825 * @see rx_fec_decode_hard() Decode with hard decision
00826 *
00827 * @since Version 1.0.0
00828 * @version 1.0.0
00829 *
00830 * @test test_rx_fec.c::test_encode_basic() Tests basic encoding
00831 * @test test_rx_fec.c::test_encode_tail_bits() Verifies tail bit termination
00832 *
00833 * @par NASA Power of 10 Compliance
00834 * - **Rule 2**: [OK] Loop bounds statically provable (input_len <= k_fec_max_input_bytes)
00835 * - **Rule 5**: [OK] 7 precondition checks, 3 postconditions
00836 */
00837 [[nodiscard]] rx_err_t rx_fec_encode(const rx_fec_encoder_t* enc,
00838                                     const uint8_t* input,
00839                                     uint32_t input_len,
00840                                     uint8_t* output,
00841                                     uint32_t* output_len);
00842
00843 /**
00844 * @brief Calculate FEC encoded output length
00845 *
00846 * @details
00847 * Computes the number of bytes required to store FEC-encoded output for
00848 * a given input length. This function should be called before rx_fec_encode()
00849 * to allocate appropriately-sized output buffers.
00850 *
00851 * ## Formula Derivation
00852 *
00853 * Given @f$ n @f$ input bytes:
00854 *
00855 * 1. **Input bits**: @f$ b_{\text{in}} = 8n @f$
00856 * 2. **Add tail bits**: @f$ b_{\text{total}} = 8n + 6 @f$
00857 * 3. **Apply rate 1/2**: @f$ b_{\text{out}} = 2(8n + 6) = 16n + 12 @f$
00858 * 4. **Convert to bytes (ceiling)**: @f$ L_{\text{out}} = \lceil \frac{16n + 12}{8} \rceil = 2n + 2 @f$
00859 *
00860 * Simplified formula:
00861 * @f[
00862 * L_{\text{out}}(n) = 2n + 2 \text{ bytes}
00863 * @f]
00864 *
00865 * ## Example Calculations
00866 *
00867 * | Input (bytes) | Output (bytes) | Expansion | Overhead (%) |
00868 * |-----|-----|-----|-----|
00869 * | 1 | 4 | 4.00x | 300% |
00870 * | 10 | 22 | 2.20x | 120% |
00871 * | 100 | 202 | 2.02x | 102% |
00872 * | 1024 | 2050 | 2.00x | 100.2% |
00873 *
00874 * Note: Overhead decreases with larger frames (tail bits amortized).
00875 *
00876 * ## Performance
00877 *
00878 * - **Time Complexity**: @f$ O(1) @f$ (constant time calculation)
00879 * - **Execution Time**: < 1 us @ 240 MHz (integer arithmetic only)
00880 *
00881 * @param[in] input_len Number of input bytes to encode
00882 * - Valid range: [0, k_fec_max_input_bytes]
00883 * - Value 0 returns 0 (no output for no input)
00884 * - Values > k_fec_max_input_bytes return 0 (error)
00885 *
00886 * @return uint32_t Number of output bytes after encoding
00887 * @retval 2*input_len+2 Normal case (1 <= input_len <= 1024)
00888 * @retval 0 Invalid input (input_len == 0 or > k_fec_max_input_bytes)
00889 *
00890 * @pre None (pure function, no side effects)
00891 *
00892 * @post Return value is deterministic function of input_len
00893 * @post For valid input: return == 2*input_len + 2
00894 * @post For invalid input: return == 0
00895 *
00896 * @note **Thread Safety**: Pure function, safe to call concurrently.
00897 *
00898 * @note Return value of 0 indicates invalid input_len parameter.
00899 *
00900 * @warning Caller must check return value before allocating buffer.
00901 * Zero return means invalid input, not "zero bytes needed".
00902 *
00903 * @par Example: Buffer Allocation
00904 * @code{.c}
00905 * uint32_t input_len = 100;
00906 * uint32_t output_size = rx_fec_encoded_len(input_len);
00907 * if (output_size == 0) {
00908 *     // Invalid input length
00909 *     return k_rx_err_invalid_arg;
00910 * }

```

```

00911 *
00912 * uint8_t* output = malloc(output_size); // Allocate 202 bytes
00913 * // ... use buffer ...
00914 * free(output);
00915 * @endcode
00916 *
00917 * @par Example: Static Buffer Sizing
00918 * @code{.c}
00919 * \#define MAX_INPUT_LEN 100
00920 * \#define MAX_OUTPUT_LEN (2 * MAX_INPUT_LEN + 2) // 202 bytes
00921 *
00922 * uint8_t output[MAX_OUTPUT_LEN];
00923 * uint32_t actual_len = rx_fec_encoded_len(input_len);
00924 * assert(actual_len <= MAX_OUTPUT_LEN); // Verify buffer sufficiency
00925 * @endcode
00926 *
00927 * @par Example: Error Detection
00928 * @code{.c}
00929 * uint32_t size = rx_fec_encoded_len(5000); // Too large
00930 * if (size == 0) {
00931 *     // Input exceeds maximum (1024 bytes)
00932 *     rx_log_error("FEC", "Input too large for FEC encoding");
00933 *     return k_rx_err_invalid_size;
00934 * }
00935 * @endcode
00936 *
00937 * @see rx_fec_encode() Encode data (uses this for buffer size)
00938 * @see k_fec_max_input_bytes Maximum allowable input length
00939 *
00940 * @since Version 1.0.0
00941 */
00942 uint32_t rx_fec_encoded_len(uint32_t input_len);
00943
00944 /*
=====
00945 * Decoder
00946 *
=====
00947 */
00948 /**
00949  * @enum rx_fec_buffer_limits_t
00950  * @brief Maximum buffer sizes for static FEC decoder allocation
00951  *
00952  * @details
00953  * Defines compile-time buffer limits to enable zero dynamic allocation (NASA Rule 3).
00954  * Values calculated from maximum payload size (1024 bytes) and FEC parameters.
00955  *
00956  * ## Size Calculations
00957  *
00958  * For maximum 1024-byte payload:
00959  * - Input bits: @f$ 1024 \times 8 = 8192 @f$
00960  * - With tail: @f$ 8192 + 6 = 8198 @f$ bits
00961  * - Encoded symbols (rate 1/2): @f$ 8198 @f$ symbol pairs
00962  * - Safety margin: Round to 8200 symbols
00963  *
00964  * @par Memory Usage
00965  * | Buffer | Size | Bytes |
00966  * |-----|-----|-----|
00967  * | Soft bits | 8200 x 2 x 1 byte | 16400 |
00968  * | Survivors | 8200 x 8 bytes (uint64_t) | 65600 |
00969  * | Path metrics | 64 x 4 bytes (int32_t) | 256 |
00970  * | Branch table | 64 x 2 x 2 x 1 byte | 256 |
00971  * | **Total per decoder** | | **~82 KB** |
00972  *
00973  * @note Large memory footprint limits number of concurrent decoders.
00974  * RX72N has 512 KB SRAM, allowing ~6 decoders maximum.
00975  *
00976  * @see rx_fec_decoder_t Decoder structure using these limits
00977  *
00978  * @since Version 1.0.0
00979  */
00980
00981 typedef enum : uint16_t {
00982     /**< @brief Maximum symbols for Viterbi trellis: 8200
00983     * @details Supports up to 1024-byte payloads (8192 bits + 6 tail bits + 2 margin).
00984     * Each symbol is a pair of soft bits (G1, G2). */
00985     k_fec_max_symbols = 8200,
00986
00987     /**< @brief Maximum input bytes before FEC encoding: 1024
00988     * @details Derived from k_fec_max_symbols: (8200 - 6) / 8 = 1024.25 -> 1024 bytes.
00989     * Larger payloads must be fragmented. */
00990     k_fec_max_input_bytes = (uint16_t)((k_fec_max_symbols - k_fec_tail_bits) / k_rx_bits_per_byte),
00991 } rx_fec_buffer_limits_t;
00992
00993 static_assert((bool)(k_fec_max_input_bytes ==
00994     (uint16_t)((k_fec_max_symbols - k_fec_tail_bits) / k_rx_bits_per_byte)),
00995     "k_fec_max_input_bytes mismatch with k_fec_max_symbols");

```

```

00996
00997 /**
00998  * @brief FEC decoder state
00999  *
01000  * Contains all working buffers for Viterbi decoding. The survivors buffer
01001  * must be provided by the caller for static allocation.
01002  */
01003 typedef struct {
01004     int32_t path_metrics[k_fec_num_states]; /**< Current path metrics */
01005     int32_t new_path_metrics[k_fec_num_states]; /**< Next path metrics */
01006     uint64_t* survivors; /**< Survivor bits (caller-provided) */
01007     uint32_t survivors_len; /**< Length of survivors buffer */
01008     uint8_t branch_table[k_fec_num_states][k_fec_num_input_values]
01009                [k_fec_num_outputs]; /**< Precomputed outputs */
01010     bool initialized; /**< True if initialized */
01011 } rx_fec_decoder_t;
01012
01013 /**
01014  * @brief Initialize FEC decoder
01015  *
01016  * The caller must provide a survivors buffer for traceback. Size should be
01017  * at least (max_symbols / 64 + 1) * k_fec_num_states uint64_t elements.
01018  *
01019  * @param[out] dec Pointer to decoder handle
01020  * @param[in] survivors_buf Caller-provided buffer for survivor bits
01021  * @param[in] survivors_len Number of uint64_t elements in survivors_buf
01022  *
01023  * @return k_rx_ok on success
01024  * @return k_rx_err_invalid_arg if any pointer is nullptr or buffer too small
01025  */
01026 rx_err_t
01027 rx_fec_decoder_init(rx_fec_decoder_t* dec, uint64_t* survivors_buf, uint32_t survivors_len);
01028
01029 /**
01030  * @brief Deinitialize FEC decoder
01031  *
01032  * @param[in,out] dec Pointer to decoder handle
01033  * @return k_rx_ok on success
01034  */
01035 [[nodiscard]] rx_err_t rx_fec_decoder_deinit(rx_fec_decoder_t* dec);
01036
01037 /**
01038  * @brief Soft-decision decode parameters
01039  *
01040  * Groups parameters for soft-decision Viterbi decoding to simplify API.
01041  */
01042 typedef struct {
01043     const rx_soft_bit_t* soft_bits; /**< Received soft bits (pairs) */
01044     uint32_t soft_len; /**< Number of soft bits (must be even) */
01045     uint32_t expected_output_len; /**< Expected decoded length in bytes */
01046     uint8_t* output; /**< Decoded output buffer */
01047     uint32_t* output_len; /**< Actual decoded length */
01048 } rx_fec_decode_soft_params_t;
01049
01050 /**
01051  * @brief Decode soft bits using Viterbi algorithm
01052  *
01053  * Performs soft-decision Viterbi decoding on received soft bits.
01054  * Soft bits should be in pairs (G1, G2) for each encoded symbol.
01055  *
01056  * @param[in] dec Initialized decoder handle
01057  * @param[in,out] params Decode parameters (input soft bits, output buffer)
01058  *
01059  * @return k_rx_ok on success
01060  * @return k_rx_err_invalid_arg if any pointer is nullptr or soft_len is odd
01061  * @return k_rx_err_invalid_state if decoder not initialized
01062  * @return k_rx_err_invalid_size if output buffer too small
01063  */
01064 [[nodiscard]] rx_err_t rx_fec_decode_soft(rx_fec_decoder_t* dec,
01065     const rx_fec_decode_soft_params_t* params);
01066
01067 /**
01068  * @brief Hard-decision decode parameters
01069  *
01070  * Groups parameters for hard-decision Viterbi decoding to simplify API.
01071  */
01072 typedef struct {
01073     const uint8_t* data; /**< Received hard bits (packed bytes) */
01074     uint32_t data_len; /**< Number of bytes */
01075     uint32_t expected_output_len; /**< Expected decoded length in bytes */
01076     uint8_t* output; /**< Decoded output buffer */
01077     uint32_t* output_len; /**< Actual decoded length */
01078     rx_soft_bit_t* soft_bits_buffer; /**< Working buffer for soft bit conversion */
01079     uint32_t soft_buffer_len; /**< Length of soft_bits_buffer */
01080 } rx_fec_decode_hard_params_t;
01081
01082 /**

```

```

01083 * @brief Decode hard bits using Viterbi algorithm
01084 *
01085 * Converts hard bits to soft bits internally, then performs Viterbi decoding.
01086 * Less accurate than soft-decision decoding but useful for testing.
01087 *
01088 * The caller must provide a working buffer for soft bit conversion to ensure
01089 * thread safety. Size should be at least k_fec_max_symbols * k_fec_num_outputs.
01090 *
01091 * @param[in] dec Initialized decoder handle
01092 * @param[in,out] params Decode parameters (input data, buffers)
01093 *
01094 * @return k_rx_ok on success
01095 * @return k_rx_err_invalid_arg if any pointer is nullptr
01096 * @return k_rx_err_invalid_state if decoder not initialized
01097 * @return k_rx_err_invalid_size if soft_bits_buffer is too small
01098 */
01099 [[nodiscard]] rx_err_t rx_fec_decode_hard(rx_fec_decoder_t* dec,
01100                                           const rx_fec_decode_hard_params_t* params);
01101
01102 /*
=====
01103 * Utility Functions
01104 *
01105 * NOTE: These are intentionally static inline in the header because:
01106 * 1. They are trivial one-liner conversions (single ternary operation)
01107 * 2. They are called in tight loops during encoding/decoding
01108 * 3. They need to be available across multiple translation units (rx_fec.c,
01109 * rx_harq.c, tests)
01110 * 4. Function call overhead would exceed the actual computation cost
01111 *
01112 * For such trivial operations, static inline is the standard C pattern and
01113 * results in better performance with no binary size increase (the function
01114 * body is smaller than the call setup/teardown).
01115 *
=====
01116 */
01117 /**
01118 * @brief Convert hard bit to soft bit
01119 *
01120 * @param[in] bit Hard bit value (0 or 1)
01121 * @return Soft bit: k_soft_bit_min for 0, k_soft_bit_max for 1
01122 */
01123 static inline rx_soft_bit_t rx_fec_hard_to_soft(uint8_t bit)
01124 {
01125     return (bit != 0) ? k_soft_bit_max : k_soft_bit_min;
01126 }
01127
01128 /**
01129 * @brief Convert soft bit to hard bit
01130 *
01131 * @param[in] soft Soft bit value
01132 * @return Hard bit: 1 if soft >= 0, 0 otherwise
01133 */
01134 static inline uint8_t rx_fec_soft_to_hard(rx_soft_bit_t soft)
01135 {
01136     return (soft >= 0) ? 1 : 0;
01137 }
01138
01139 #ifdef __cplusplus
01140 }
01141 #endif

```

7.3 libs/rx_fec/src/rx_fec.c File Reference

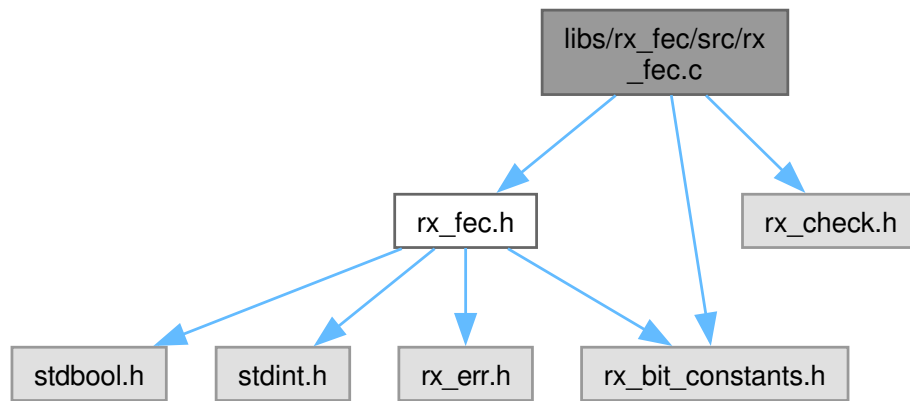
Forward Error Correction (FEC) Codec Implementation.

```

#include "rx_fec.h"
#include "rx_bit_constants.h"
#include "rx_check.h"

```

Include dependency graph for rx_fec.c:



Macros

- `#define RX_HAS_STDCKDINT (0)`
- `#define RX_HAS_STDBIT (0)`

Enumerations

- `enum rx_fec_impl_constants_t : uint16_t {`
`k_fec_zero = 0, k_fec_msb_bit_position = 7, k_fec_shift_register_bits = 6, k_fec_correlation_offset = 32768,`
`k_fec_state_shift_amount = 5, k_fec_bit_mask = 1, k_fec_parity_shift_nibble = 4,`
`k_fec_parity_shift_pair = 2 }`
 FEC implementation constants.
- `enum rx_fec_output_index_t : uint8_t {`
`k_fec_output_g1 = 0, k_fec_output_g2 = 1 }`
 FEC output indices for G1 and G2 generator polynomials.

Functions

- `RX_STATIC_TESTABLE uint8_t internal_parity (uint8_t x)`
 Calculate parity (XOR of all bits) of a byte.
- `RX_STATIC_TESTABLE void internal_set_output_bit (uint8_t *output, const uint32_t bit_idx, uint8_t value)`
 Set a bit at specified index in output buffer (MSB first).
- `RX_STATIC_TESTABLE uint8_t internal_get_bit (const uint8_t *data, uint32_t bit_idx)`
 Get a bit at specified index from buffer (MSB first).
- `RX_STATIC_TESTABLE void internal_encode_bit (uint8_t *state, const uint8_t input_bit, uint8_t *out0, uint8_t *out1)`
 Encode a single bit and update encoder state.
- `static void internal_init_branch_table (uint8_t branch_table[k_fec_num_states][k_fec_num_input_values][k_fec_num_output_values])`
 Initialize the branch table for Viterbi decoder.
- `RX_STATIC_TESTABLE int32_t internal_branch_metric (const rx_soft_bit_t soft0, const rx_soft_bit_t soft1, const uint8_t exp0, const uint8_t exp1)`
 Compute branch metric for soft decoding.
- `static void internal_viterbi_process_transition (rx_fec_decoder_t *dec, const rx_soft_bit_t soft0, const rx_soft_bit_t soft1, const uint32_t symbol_idx, const uint8_t state, const uint8_t input)`
 Process one transition (state + input combination) in the Viterbi trellis.

- RX_STATIC_TESTABLE void [internal_viterbi_process_symbol](#) ([rx_fec_decoder_t](#) *dec, const [rx_soft_bit_t](#) soft0, const [rx_soft_bit_t](#) soft1, const [uint32_t](#) symbol_idx)
Process one symbol pair through the Viterbi trellis.
- RX_STATIC_TESTABLE void [internal_viterbi_forward_pass](#) ([rx_fec_decoder_t](#) *dec, const [rx_soft_bit_t](#) *soft_bits, const [uint32_t](#) num_symbols)
Run Viterbi forward pass through trellis.
- RX_STATIC_TESTABLE bool [internal_viterbi_traceback_validate](#) (const [rx_fec_decoder_t](#) *dec, const [uint32_t](#) num_symbols, const [uint8_t](#) *output, const [uint32_t](#) output_bytes)
Validate traceback preconditions.
- RX_STATIC_TESTABLE void [internal_viterbi_traceback](#) (const [rx_fec_decoder_t](#) *dec, const [uint32_t](#) num_symbols, const [uint32_t](#) data_bits, [uint8_t](#) *output, const [uint32_t](#) output_bytes)
Perform Viterbi traceback to extract decoded bits.
- [rx_err_t](#) [rx_fec_encoder_init](#) ([rx_fec_encoder_t](#) *enc)
Initialize FEC encoder.
- [rx_err_t](#) [rx_fec_encoder_deinit](#) ([rx_fec_encoder_t](#) *enc)
Deinitialize FEC encoder.
- RX_STATIC_TESTABLE bool [internal_fec_compute_encoded_bytes](#) ([uint32_t](#) input_len, [uint32_t](#) *out_bytes)
Compute encoded bit count from raw input length (unchecked).
- [uint32_t](#) [rx_fec_encoded_len](#) (const [uint32_t](#) input_len)
Calculate encoded output length for given input.
- static void [internal_encode_data_bytes](#) (const [uint8_t](#) *input, const [uint32_t](#) byte_limit, [uint8_t](#) *output, [uint8_t](#) *state, [uint32_t](#) *out_bit_idx)
Encode all data bytes into output bit stream.
- static void [internal_encode_tail_bits](#) ([uint8_t](#) *output, [uint8_t](#) *state, [uint32_t](#) *out_bit_idx)
Append tail bits to flush encoder shift register to zero state.
- [rx_err_t](#) [rx_fec_encode](#) (const [rx_fec_encoder_t](#) *enc, const [uint8_t](#) *input, const [uint32_t](#) input_len, [uint8_t](#) *output, [uint32_t](#) *output_len)
Encode data using convolutional FEC.
- static [rx_err_t](#) [internal_validate_decode_params](#) ([rx_fec_decoder_t](#) *dec, const [rx_fec_decode_soft_params_t](#) *params, [uint32_t](#) *num_symbols_out)
Validate decode parameters before Viterbi decoding.
- [rx_err_t](#) [rx_fec_decoder_init](#) ([rx_fec_decoder_t](#) *dec, [uint64_t](#) *survivors_buf, const [uint32_t](#) survivors_len)
Initialize FEC decoder.
- [rx_err_t](#) [rx_fec_decoder_deinit](#) ([rx_fec_decoder_t](#) *dec)
Deinitialize FEC decoder.
- [rx_err_t](#) [rx_fec_decode_soft](#) ([rx_fec_decoder_t](#) *dec, const [rx_fec_decode_soft_params_t](#) *params)
Decode FEC-encoded data using soft-decision Viterbi algorithm.
- [rx_err_t](#) [rx_fec_decode_hard](#) ([rx_fec_decoder_t](#) *dec, const [rx_fec_decode_hard_params_t](#) *params)
Decode FEC-encoded data using hard-decision bits (converts to soft internally).

7.3.1 Detailed Description

Forward Error Correction (FEC) Codec Implementation.

Implements NASA-standard K=7, rate 1/2 convolutional encoder and soft-decision Viterbi decoder. This is a C port of star-gateway/internal/fec/ ensuring bit-exact compatibility between RX72N firmware and RPi5 gateway.

7.3.1.1 Implementation Architecture

7.3.1.1.1 Encoder (Stateless)

- Each encode() call starts from zero state
- Processes input MSB-first (network byte order)
- Appends 6 tail bits to flush to zero
- Output packed MSB-first into bytes

7.3.1.1.2 Decoder (Viterbi Algorithm)

- Forward Pass: Compute path metrics for all 64 states at each time step
- Traceback: Reconstruct most-likely input sequence from survivors
- Soft Decision: Uses branch metrics based on soft bit correlation
- Memory: Survivors buffer stores predecessor bits for traceback

7.3.1.2 Algorithm Complexity

Operation	Time Complexity	Space Complexity
Encoding	$O(n)$	$O(1)$
Decoding (Viterbi)	$O(n \times 64 \times 2)$	$O(n \times 64 \text{ bits})$
Traceback	$O(n)$	$O(1)$

where n = number of input bits.

7.3.1.3 Viterbi Algorithm Overview

The Viterbi algorithm finds the maximum likelihood path through the trellis of encoder states using dynamic programming:

1. Initialization: Set path metric for state 0 to 0, all others to inf
2. Forward Pass: For each received symbol pair (r_0 , r_1):
 - For each current state with finite metric:
 - Compute metrics for both possible transitions (input=0, input=1)
 - Update next state metrics (keep minimum)
 - Store survivor path (which previous state was best)
3. Traceback: Starting from final state 0 (encoder flushed), work backwards through survivors to recover input sequence

7.3.1.4 Branch Metric Calculation

Uses correlation metric for soft-decision decoding:

$$M_{\text{branch}} = 32768 - (r_0 \cdot e_0' + r_1 \cdot e_1')$$

where:

- r_i = received soft bit (-127 to +127)
- e_i' = +127 if expected bit is 1, -127 if 0
- Lower metric = better match

7.3.1.5 Bit Ordering Convention

Throughout this implementation: MSB-first (network byte order)

- Bit index 0 -> byte[0] bit 7 (MSB)
- Bit index 7 -> byte[0] bit 0 (LSB)
- Bit index 8 -> byte[1] bit 7 (MSB)

This matches wire protocols and simplifies interoperability.

Module Dependencies

- [rx_fec.h](#): Public API declarations
- [rx_check.h](#): Assertion macros (RX_ASSERT)
- [rx_bit_constants.h](#): Bit manipulation constants
- No standard library headers required (memset replaced with for-loop)

NASA Power of 10 Compliance

- Rule 1: [OK] No recursion, goto, or setjmp/longjmp
- Rule 2: [OK] All loops bounded (k_fec_max_symbols, k_fec_num_states)
- Rule 3: [OK] Zero dynamic allocation
- Rule 4: [OK] All functions <= 60 lines
- Rule 5: [OK] All functions have >=2 assertions
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror

Author

Locked, Inc.

Date

2026-01-27

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

Version

1.0.0

Definition in file [rx_fec.c](#).

7.3.2 Macro Definition Documentation

7.3.2.1 RX_HAS_STDBIT

```
#define RX_HAS_STDBIT (0)
```

Definition at line 113 of file [rx_fec.c](#).

7.3.2.2 RX_HAS_STDCKDINT

```
#define RX_HAS_STDCKDINT (0)
```

Definition at line 106 of file [rx_fec.c](#).

7.3.3 Enumeration Type Documentation

7.3.3.1 rx_fec_impl_constants_t

```
enum rx_fec_impl_constants_t : uint16_t
```

FEC implementation constants.

Enumerator

k_fec_zero	Zero constant
k_fec_msb_bit_position	MSB position in byte (0-indexed)
k_fec_shift_register_bits	K-1 shift register size
k_fec_correlation_offset	Correlation metric offset (2^{15})
k_fec_state_shift_amount	k_fec_constraint_length - 2
k_fec_bit_mask	Mask for extracting single bit
k_fec_parity_shift_nibble	Shift amount for nibble XOR in parity computation
k_fec_parity_shift_pair	Shift amount for bit-pair XOR in parity computation

Definition at line 127 of file [rx_fec.c](#).

```
00127     : uint16_t {
00128     k_fec_zero          = 0,    /**< Zero constant */
00129     k_fec_msb_bit_position = 7,  /**< MSB position in byte (0-indexed) */
00130     k_fec_shift_register_bits = 6, /**< K-1 shift register size */
00131     k_fec_correlation_offset = 32768, /**< Correlation metric offset ( $2^{15}$ ) */
00132     k_fec_state_shift_amount = 5,  /**< k_fec_constraint_length - 2 */
00133     k_fec_bit_mask       = 1,    /**< Mask for extracting single bit */
00134     k_fec_parity_shift_nibble = 4, /**< Shift amount for nibble XOR in parity computation */
00135     k_fec_parity_shift_pair = 2,  /**< Shift amount for bit-pair XOR in parity computation */
00136 } rx_fec_impl_constants_t;
```

7.3.3.2 rx_fec_output_index_t

enum [rx_fec_output_index_t](#) : uint8_t

FEC output indices for G1 and G2 generator polynomials.

Enumerator

k_fec_output_g1	G1 generator output (first encoded bit)
k_fec_output_g2	G2 generator output (second encoded bit)

Definition at line 141 of file [rx_fec.c](#).

```
00141     : uint8_t {
00142   k_fec_output_g1 = 0, /**< G1 generator output (first encoded bit) */
00143   k_fec_output_g2 = 1, /**< G2 generator output (second encoded bit) */
00144 } rx_fec_output_index_t;
```

7.3.4 Function Documentation

7.3.4.1 internal_branch_metric()

```
RX_STATIC_TESTABLE int32_t internal_branch_metric (
    const rx\_soft\_bit\_t soft0,
    const rx\_soft\_bit\_t soft1,
    const uint8_t exp0,
    const uint8_t exp1)
```

Compute branch metric for soft decoding.

Uses correlation metric: higher correlation = lower metric (better). Formula: $32768 - (\text{soft0} * \text{exp0_soft} + \text{soft1} * \text{exp1_soft})$

Definition at line 312 of file [rx_fec.c](#).

```
00316 {
00317   /* Pre-conditions: expected bits must be 0 or 1 */
00318   RX_ASSERT_PRE((exp0 == k_fec_zero) || (exp0 == k_fec_bit_mask), "Expected bit 0 must be 0 or 1");
00319   RX_ASSERT_PRE((exp1 == k_fec_zero) || (exp1 == k_fec_bit_mask), "Expected bit 1 must be 0 or 1");
00320
00321   #ifdef UNIT_TEST
00322   if (((exp0 != k_fec_zero) && (exp0 != k_fec_bit_mask)) ||
00323       ((exp1 != k_fec_zero) && (exp1 != k_fec_bit_mask))) {
00324     return 0;
00325   }
00326   #endif
00327
00328   /* Convert expected bits to soft values: 0 -> -127, 1 -> +127 */
00329   const int32_t exp0_soft = (exp0 != 0) ? k_soft_bit_max : k_soft_bit_min;
00330   const int32_t exp1_soft = (exp1 != 0) ? k_soft_bit_max : k_soft_bit_min;
00331
00332   /* Correlation = soft0*exp0_soft + soft1*exp1_soft */
00333   const int32_t correlation = ((int32_t)soft0 * exp0_soft) + ((int32_t)soft1 * exp1_soft);
00334
00335   /* Negate and offset to keep metrics positive (lower is better) */
00336   return k_fec_correlation_offset - correlation;
00337 }
```

References [k_fec_bit_mask](#), [k_fec_correlation_offset](#), [k_fec_zero](#), [k_soft_bit_max](#), and [k_soft_bit_min](#).

Referenced by [internal_viterbi_process_transition\(\)](#).

Here is the caller graph for this function:



7.3.4.2 internal_encode_bit()

```
RX_STATIC_TESTABLE void internal_encode_bit (
    uint8_t * state,
    const uint8_t input_bit,
    uint8_t * out0,
    uint8_t * out1)
```

Encode a single bit and update encoder state.

Definition at line 256 of file [rx_fec.c](#).

```
00257 {
00258     /* Pre-condition 1: All pointers must be valid */
00259     RX_ASSERT_PRE(state != nullptr, "State pointer must not be nullptr");
00260     RX_ASSERT_PRE(out0 != nullptr, "Output 0 pointer must not be nullptr");
00261     RX_ASSERT_PRE(out1 != nullptr, "Output 1 pointer must not be nullptr");
00262
00263     /* Pre-condition 2: input_bit must be 0 or 1 */
00264     RX_ASSERT_PRE((input_bit == k_fec_zero) || (input_bit == k_fec_bit_mask),
00265         "Input bit must be 0 or 1");
00266
00267     #ifdef UNIT_TEST
00268     if ((state == nullptr) || (out0 == nullptr) || (out1 == nullptr)) {
00269         return;
00270     }
00271     #endif
00272
00273     /* Shift in the new bit (input is MSB of the combined state) */
00274     const uint8_t combined = (uint8_t)((input_bit « k_fec_shift_register_bits) | *state);
00275
00276     /* Calculate output bits using generator polynomials */
00277     *out0 = internal_parity(combined & k_fec_g1_octal);
00278     *out1 = internal_parity(combined & k_fec_g2_octal);
00279
00280     /* Update state (shift right, new bit enters from left) */
00281     *state = combined » k_fec_bit_mask;
00282 }
```

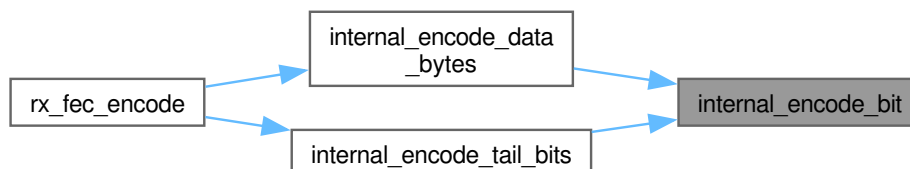
References [internal_parity\(\)](#), [k_fec_bit_mask](#), [k_fec_g1_octal](#), [k_fec_g2_octal](#), [k_fec_shift_register_bits](#), and [k_fec_zero](#).

Referenced by [internal_encode_data_bytes\(\)](#), and [internal_encode_tail_bits\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.3 internal_encode_data_bytes()

```
void internal_encode_data_bytes (
    const uint8_t * input,
    const uint32_t byte_limit,
    uint8_t * output,
    uint8_t * state,
    uint32_t * out_bit_idx) [static]
```

Encode all data bytes into output bit stream.

Processes each byte of input MSB-first, producing two output bits per input bit via the convolutional encoder.

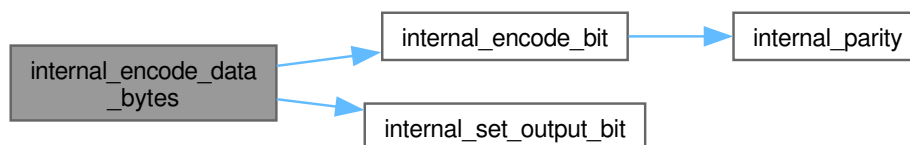
Definition at line 684 of file [rx_fec.c](#).

```
00689 {
00690     for (uint32_t byte_idx = k_fec_zero; byte_idx < byte_limit; byte_idx++) {
00691         const uint8_t b = input[byte_idx];
00692         for (int8_t i = (int8_t)k_fec_msb_bit_position; i >= (int8_t)k_fec_zero; i--) {
00693             const uint8_t input_bit = (b » i) & k_fec_bit_mask;
00694             uint8_t out0;
00695             uint8_t out1;
00696
00697             internal_encode_bit(state, input_bit, &out0, &out1);
00698
00699             internal_set_output_bit(output, (*out_bit_idx)++, out0);
00700             internal_set_output_bit(output, (*out_bit_idx)++, out1);
00701         }
00702     }
00703 }
```

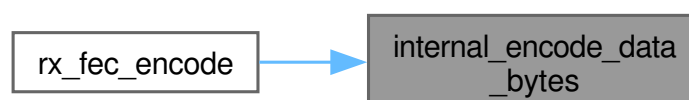
References [internal_encode_bit\(\)](#), [internal_set_output_bit\(\)](#), [k_fec_bit_mask](#), [k_fec_msb_bit_position](#), and [k_fec_zero](#).

Referenced by [rx_fec_encode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.4 internal_encode_tail_bits()

```
void internal_encode_tail_bits (
    uint8_t * output,
    uint8_t * state,
    uint32_t * out_bit_idx) [static]
```

Append tail bits to flush encoder shift register to zero state.

Appends `k_fec_tail_bits` zero-input bits to the output stream, ensuring the decoder traceback terminates at state 0.

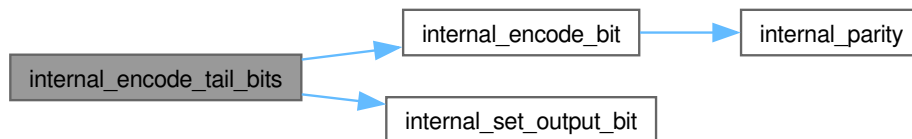
Definition at line 712 of file `rx_fec.c`.

```
00713 {
00714     for (uint8_t i = k_fec_zero; i < k_fec_tail_bits; i++) {
00715         uint8_t out0;
00716         uint8_t out1;
00717         internal_encode_bit(state, k_fec_zero, &out0, &out1);
00718         internal_set_output_bit(output, (*out_bit_idx)++, out0);
00719         internal_set_output_bit(output, (*out_bit_idx)++, out1);
00720     }
00721 }
```

References `internal_encode_bit()`, `internal_set_output_bit()`, `k_fec_tail_bits`, and `k_fec_zero`.

Referenced by `rx_fec_encode()`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.5 internal_fec_compute_encoded_bytes()

```
RX_STATIC_TESTABLE bool internal_fec_compute_encoded_bytes (
    uint32_t input_len,
    uint32_t * out_bytes)
```

Compute encoded bit count from raw input length (unchecked).

Computes the number of output bits for FEC encoding using the formula: `output_bits = round_up_8((input_len * 8 + tail_bits) * 2 + 7)`

With C23 checked arithmetic (`RX_HAS_STDCKDINT`), any overflow returns 0. Without it, unchecked arithmetic is used (caller must ensure no overflow).

This function does NOT validate `input_len`. It is exposed as `RX_STATIC_TESTABLE` so that unit tests can exercise the overflow paths with values larger than `k_fec_max_input_bytes` (which the public `rx_fec_encoded_len()` rejects first).

Precondition

`out_bytes` must not be `nullptr`

Postcondition

`*out_bytes == 0` if overflow detected; otherwise rounded-up byte count

Note

Not thread-safe (pure computation, no shared state)

Since

Version 1.0.0

Definition at line 599 of file [rx_fec.c](#).

```

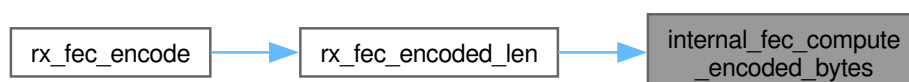
00600 {
00601     RX_ASSERT_PRE(out_bytes != nullptr, "out_bytes must not be nullptr");
00602     #ifdef UNIT_TEST
00603     if (out_bytes == nullptr) {
00604         return false;
00605     }
00606     #endif
00607
00608     /* Output bits = (input bits + tail bits) * 2, rounded up to whole bytes */
00609     #if RX_HAS_STDCKDINT
00610     uint32_t input_bits = 0;
00611     if (ckd_mul(&input_bits, input_len, (uint32_t)k_rx_bits_per_byte)) {
00612         *out_bytes = k_fec_zero;
00613         return false;
00614     }
00615
00616     /* Step-2 ckd_add not needed: if step-1 passes, input_bits <= 536870911*8 = 4294967288,
00617      * so input_bits + k_fec_tail_bits (6) <= 4294967294 <= UINT32_MAX, no overflow. */
00618     const uint32_t total_input_bits = input_bits + (uint32_t)k_fec_tail_bits;
00619
00620     uint32_t total_output_bits = 0;
00621     if (ckd_mul(&total_output_bits, total_input_bits, (uint32_t)k_fec_num_outputs)) {
00622         *out_bytes = k_fec_zero;
00623         return false;
00624     }
00625
00626     uint32_t rounded = 0;
00627     if (ckd_add(&rounded, total_output_bits, (uint32_t)k_fec_msb_bit_position)) {
00628         *out_bytes = k_fec_zero;
00629         return false;
00630     }
00631
00632     *out_bytes = rounded / (uint32_t)k_rx_bits_per_byte;
00633     return true;
00634     #else
00635     /* Fallback: caller must ensure input_len <= k_fec_max_input_bytes (1024) */
00636     const uint32_t input_bits = input_len * (uint32_t)k_rx_bits_per_byte;
00637     const uint32_t total_input_bits = input_bits + (uint32_t)k_fec_tail_bits;
00638     const uint32_t total_output_bits = total_input_bits * (uint32_t)k_fec_num_outputs;
00639     const uint32_t rounded = total_output_bits + (uint32_t)k_fec_msb_bit_position;
00640     *out_bytes = rounded / (uint32_t)k_rx_bits_per_byte;
00641     return true;
00642     #endif
00643 }

```

References [k_fec_msb_bit_position](#), [k_fec_num_outputs](#), [k_fec_tail_bits](#), and [k_fec_zero](#).

Referenced by [rx_fec_encoded_len\(\)](#).

Here is the caller graph for this function:



7.3.4.6 internal_get_bit()

```
RX_STATIC_TESTABLE uint8_t internal_get_bit (
    const uint8_t * data,
    uint32_t bit_idx)
```

Get a bit at specified index from buffer (MSB first).

Bit ordering (MSB first, network byte order): bit_idx=0 -> byte 0, bit 7 (MSB) bit_idx=7 -> byte 0, bit 0 (LSB) bit_idx=8 -> byte 1, bit 7 (MSB)

Definition at line 230 of file [rx_fec.c](#).

```
00231 {
00232     /* Pre-condition: data must be valid */
00233     RX_ASSERT_PRE(data != nullptr, "Data buffer must not be nullptr");
00234
00235     #ifndef UNIT_TEST
00236     if (data == nullptr) {
00237         return 0;
00238     }
00239     #endif
00240
00241     const uint32_t byte_idx = bit_idx / k_rx_bits_per_byte;
00242     const uint32_t bit_pos = k_fec_msb_bit_position - (bit_idx % k_rx_bits_per_byte); /* MSB first */
00243
00244     const uint8_t result = (data[byte_idx] >> bit_pos) & k_fec_bit_mask;
00245
00246     /* Invariant: bitwise AND with 1 always yields 0 or 1; no runtime check needed */
00247
00248     return result;
00249 }
```

References [k_fec_bit_mask](#), and [k_fec_msb_bit_position](#).

Referenced by [rx_fec_decode_hard\(\)](#).

Here is the caller graph for this function:



7.3.4.7 internal_init_branch_table()

```
void internal_init_branch_table (
    uint8_t branch_table[k_fec_num_states][k_fec_num_input_values][k_fec_num_outputs]) [static]
```

Initialize the branch table for Viterbi decoder.

Precomputes expected output bits for each state and input combination.

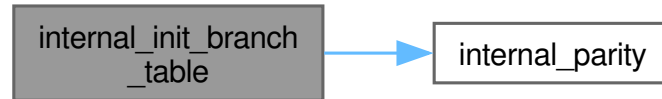
Definition at line 290 of file [rx_fec.c](#).

```
00292 {
00293     /* Pre-condition: branch_table array parameter is always valid in C */
00294     /* Loop bounds guarantee state < k_fec_num_states and input < k_fec_num_input_values */
00295
00296     for (uint8_t state = k_fec_zero; state < k_fec_num_states; state++) {
00297         for (uint8_t input = k_fec_zero; input < k_fec_num_input_values; input++) {
00298             const uint8_t combined = (uint8_t)((input << k_fec_shift_register_bits) | state);
00299             branch_table[state][input][k_fec_output_g1] = internal_parity(combined & k_fec_g1_octal);
00300             branch_table[state][input][k_fec_output_g2] = internal_parity(combined & k_fec_g2_octal);
00301         }
00302     }
00303 }
```

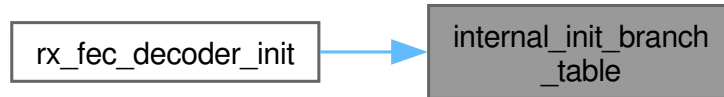
References [internal_parity\(\)](#), [k_fec_g1_octal](#), [k_fec_g2_octal](#), [k_fec_num_input_values](#), [k_fec_num_outputs](#), [k_fec_num_states](#), [k_fec_output_g1](#), [k_fec_output_g2](#), [k_fec_shift_register_bits](#), and [k_fec_zero](#).

Referenced by [rx_fec_decoder_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.8 internal_parity()

```

RX_STATIC_TESTABLE uint8_t internal_parity (
    uint8_t x)
  
```

Calculate parity (XOR of all bits) of a byte.

Uses parallel XOR reduction to compute parity in $O(\log n)$ operations. This is more efficient than looping through each bit.

Algorithm:

- Step 1 ($x \wedge= x \gg 4$): XOR upper nibble with lower nibble Original: [b7 b6 b5 b4][b3 b2 b1 b0]
Result: [x x x x][b7^b3 b6^b2 b5^b1 b4^b0]
- Step 2 ($x \wedge= x \gg 2$): XOR pairs of bits Result bits [1:0] now contain XOR of all 8 original bits
- Step 3 ($x \wedge= x \gg 1$): Final XOR to get single parity bit Result bit [0] = XOR of all original bits

Definition at line 169 of file [rx_fec.c](#).

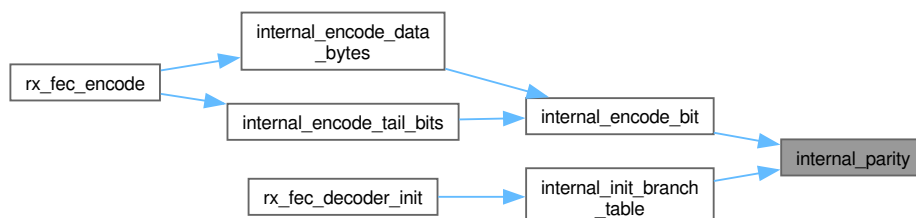
```

00170 {
00171 #if RX_HAS_STDBIT
00172 /* C23 stdc_count_ones_uc: UB-free popcount; parity = LSB of popcount */
00173 const uint8_t result = (uint8_t)(stdc_count_ones_uc(x) & k_fec_bit_mask);
00174 #else
00175 /* Fallback: parallel XOR reduction (GNURX lacks <stdbit.h>) */
00176 x ^= x » k_fec_parity_shift_nibble;
00177 x ^= x » k_fec_parity_shift_pair;
00178 x ^= x » k_fec_bit_mask;
00179 const uint8_t result = x & k_fec_bit_mask;
00180 #endif
00181
00182 /* Invariant: bitwise AND with 1 always yields 0 or 1; no runtime check needed */
00183
00184 return result;
00185 }
  
```

References [k_fec_bit_mask](#), [k_fec_parity_shift_nibble](#), and [k_fec_parity_shift_pair](#).

Referenced by [internal_encode_bit\(\)](#), and [internal_init_branch_table\(\)](#).

Here is the caller graph for this function:



7.3.4.9 internal_set_output_bit()

```
RX_STATIC_TESTABLE void internal_set_output_bit (
    uint8_t * output,
    const uint32_t bit_idx,
    uint8_t value)
```

Set a bit at specified index in output buffer (MSB first).

Bit ordering (MSB first, network byte order): bit_idx=0 -> byte 0, bit 7 (MSB) bit_idx=7 -> byte 0, bit 0 (LSB) bit_idx=8 -> byte 1, bit 7 (MSB)

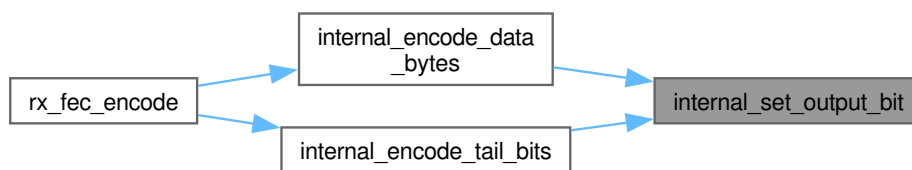
Definition at line 197 of file [rx_fec.c](#).

```
00198 {
00199     /* Pre-condition 1: output must be valid */
00200     RX_ASSERT_PRE(output != nullptr, "Output buffer must not be nullptr");
00201
00202     /* Pre-condition 2: bit index must be within maximum symbol range */
00203     RX_ASSERT_PRE(bit_idx < (k_fec_max_symbols * k_rx_bits_per_byte), "Bit index out of range");
00204
00205     #ifdef UNIT_TEST
00206     if ((output == nullptr) || (bit_idx >= (k_fec_max_symbols * k_rx_bits_per_byte))) {
00207         return;
00208     }
00209     #endif
00210
00211     value = (value != 0) ? k_fec_bit_mask : 0;
00212
00213     const uint32_t byte_idx = bit_idx / k_rx_bits_per_byte;
00214     const uint32_t bit_pos = k_fec_msb_bit_position - (bit_idx % k_rx_bits_per_byte); /* MSB first */
00215
00216     if (value != 0) {
00217         output[byte_idx] |= (uint8_t)(1U << bit_pos);
00218     }
00219 }
```

References [k_fec_bit_mask](#), [k_fec_max_symbols](#), and [k_fec_msb_bit_position](#).

Referenced by [internal_encode_data_bytes\(\)](#), and [internal_encode_tail_bits\(\)](#).

Here is the caller graph for this function:



7.3.4.10 internal_validate_decode_params()

```
rx_err_t internal_validate_decode_params (
    rx_fec_decoder_t * dec,
    const rx_fec_decode_soft_params_t * params,
    uint32_t * num_symbols_out) [static]
```

Validate decode parameters before Viterbi decoding.

Pre-conditions:

- `dec`, `params`, `num_symbols_out` must be non-nullptr

- dec must be initialized
- params->soft_bits, output, output_len must be non-nullptr
- params->soft_len must be non-zero and divisible by 2 (G1, G2 pairs)

Validation checks:

- Calculated num_symbols from expected_output_len must be valid and within bounds
- Sufficient soft bits must be available for calculated num_symbols (NOT silently reduced - caller error if params are inconsistent)
- Survivors buffer must be large enough for num_symbols

Post-condition:

- If successful, num_symbols_out contains the number of symbols to decode
- If parameters are invalid or inconsistent, returns appropriate error code

Definition at line 805 of file rx_fec.c.

```

00808 {
00809     const bool dec_null      = (bool)(dec == nullptr);
00810     const bool params_null   = (bool)(params == nullptr);
00811     const bool num_symbols_null = (bool)(num_symbols_out == nullptr);
00812     if ((bool)((int)dec_null | (int)params_null | (int)num_symbols_null)) {
00813         return k_rx_err_invalid_arg;
00814     }
00815
00816     const bool soft_bits_null = (bool)(params->soft_bits == nullptr);
00817     const bool output_null    = (bool)(params->output == nullptr);
00818     const bool output_len_null = (bool)(params->output_len == nullptr);
00819     if ((bool)((int)soft_bits_null | (int)output_null | (int)output_len_null)) {
00820         return k_rx_err_invalid_arg;
00821     }
00822
00823     if (!dec->initialized) {
00824         return k_rx_err_invalid_state;
00825     }
00826
00827     /* Soft bits must come in pairs (G1, G2) */
00828     if (params->soft_len == k_fec_zero || (params->soft_len % k_fec_num_outputs) != k_fec_zero) {
00829         return k_rx_err_invalid_arg;
00830     }
00831
00832     const uint32_t num_symbols =
00833         (params->expected_output_len > k_fec_zero)
00834         ? (uint32_t)((params->expected_output_len * k_rx_bits_per_byte) + k_fec_tail_bits)
00835         : k_fec_zero;
00836
00837     if (num_symbols < k_fec_tail_bits) {
00838         return k_rx_err_invalid_size;
00839     }
00840
00841     if (num_symbols > k_fec_max_symbols) {
00842         return k_rx_err_invalid_size;
00843     }
00844
00845     /* Verify sufficient soft bits are available for the calculated num_symbols.
00846     * This is NOT a fallback condition - if the caller provides inconsistent
00847     * expected_output_len and soft_len parameters (e.g., expects to decode 100
00848     * bytes but only provides soft bits for 50 bytes), that is a programming error
00849     * that must be caught and reported, not silently degraded. Graceful degradation
00850     * would hide bugs in parameter preparation. */
00851     if (num_symbols * k_fec_num_outputs > params->soft_len) {
00852         return k_rx_err_invalid_size;
00853     }
00854
00855     /* Check survivors buffer is large enough */
00856     if (num_symbols > dec->survivors_len) {
00857         return k_rx_err_invalid_size;
00858     }
00859
00860     *num_symbols_out = num_symbols;

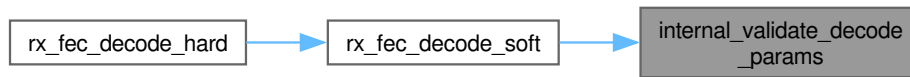
```

```
00861  return k_rx_ok;
00862 }
```

References [rx_fec_decode_soft_params_t::expected_output_len](#), [rx_fec_decoder_t::initialized](#), [k_fec_max_symbols](#), [k_fec_num_outputs](#), [k_fec_tail_bits](#), [k_fec_zero](#), [rx_fec_decode_soft_params_t::output](#), [rx_fec_decode_soft_params_t::output_len](#), [rx_fec_decode_soft_params_t::soft_bits](#), [rx_fec_decode_soft_params_t::survivors_len](#), and [rx_fec_decoder_t::survivors_len](#).

Referenced by [rx_fec_decode_soft\(\)](#).

Here is the caller graph for this function:



7.3.4.11 internal_viterbi_forward_pass()

```
RX_STATIC_TESTABLE void internal_viterbi_forward_pass (
    rx_fec_decoder_t * dec,
    const rx_soft_bit_t * soft_bits,
    const uint32_t num_symbols)
```

Run Viterbi forward pass through trellis.

Initializes path metrics and processes all symbols through the trellis.

Definition at line 428 of file [rx_fec.c](#).

```
00431 {
00432  /* Pre-conditions */
00433  RX_ASSERT_PRE(dec != nullptr, "Decoder handle must not be nullptr");
00434  RX_ASSERT_PRE(soft_bits != nullptr, "Soft bits buffer must not be nullptr");
00435
00436  #ifdef UNIT_TEST
00437  if ((dec == nullptr) || (soft_bits == nullptr)) {
00438    return;
00439  }
00440  #endif
00441
00442  /* Initialize path metrics: state 0 = 0, others = MAX */
00443  for (uint8_t i = k_fec_zero; i < k_fec_num_states; i++) {
00444    dec->path_metrics[i] = INT32_MAX;
00445  }
00446  dec->path_metrics[k_fec_zero] = k_fec_zero;
00447
00448  /* Process each symbol pair through the trellis */
00449  uint32_t limit = num_symbols;
00450  if (num_symbols > k_fec_max_symbols) {
00451    limit = k_fec_max_symbols;
00452  }
00453  for (uint32_t t = k_fec_zero; t < limit; t++) {
00454    const rx_soft_bit_t soft0 = soft_bits[t * k_fec_num_outputs + k_fec_output_g1];
00455    const rx_soft_bit_t soft1 = soft_bits[t * k_fec_num_outputs + k_fec_output_g2];
00456    internal_viterbi_process_symbol(dec, soft0, soft1, t);
00457  }
00458 }
```

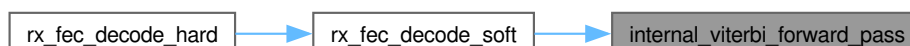
References [internal_viterbi_process_symbol\(\)](#), [k_fec_max_symbols](#), [k_fec_num_outputs](#), [k_fec_num_states](#), [k_fec_output_g1](#), [k_fec_output_g2](#), [k_fec_zero](#), and [rx_fec_decoder_t::path_metrics](#).

Referenced by [rx_fec_decode_soft\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.12 `internal_viterbi_process_symbol()`

```
RX_STATIC_TESTABLE void internal_viterbi_process_symbol (
    rx_fec_decoder_t * dec,
    const rx_soft_bit_t soft0,
    const rx_soft_bit_t soft1,
    const uint32_t symbol_idx)
```

Process one symbol pair through the Viterbi trellis.

Updates path metrics and survivors for one time step.

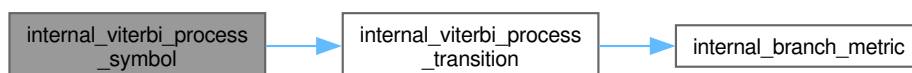
Definition at line 379 of file `rx_fec.c`.

```
00383 {
00384     /* Pre-conditions: valid decoder handle, symbol index in bounds */
00385     RX_ASSERT_PRE(dec != nullptr, "Decoder handle must not be nullptr");
00386     #ifdef UNIT_TEST
00387     if (dec == nullptr) {
00388         return;
00389     }
00390     #endif
00391     RX_ASSERT_PRE(symbol_idx < dec->survivors_len, "Symbol index exceeds survivors buffer length");
00392     #ifdef UNIT_TEST
00393     if (symbol_idx >= dec->survivors_len) {
00394         return;
00395     }
00396     #endif
00397     /* Reset new path metrics and clear survivors for this time step */
00398     for (uint8_t i = k_fec_zero; i < k_fec_num_states; i++) {
00399         dec->new_path_metrics[i] = INT32_MAX;
00400     }
00401     dec->survivors[symbol_idx] = k_fec_zero;
00402     /* For each current state, compute transitions */
00403     for (uint8_t state = k_fec_zero; state < k_fec_num_states; state++) {
00404         if (dec->path_metrics[state] == INT32_MAX) {
00405             continue;
00406         }
00407         for (uint8_t input = k_fec_zero; input < k_fec_num_input_values; input++) {
00408             internal_viterbi_process_transition(dec, soft0, soft1, symbol_idx, state, input);
00409         }
00410     }
00411     /* Swap path metrics */
00412     for (uint8_t i = k_fec_zero; i < k_fec_num_states; i++) {
00413         dec->path_metrics[i] = dec->new_path_metrics[i];
00414         dec->new_path_metrics[i] = INT32_MAX;
00415     }
00416 }
```

References `internal_viterbi_process_transition()`, `k_fec_num_input_values`, `k_fec_num_states`, `k_fec_zero`, `rx_fec_decoder_t::new_path_metrics`, `rx_fec_decoder_t::path_metrics`, `rx_fec_decoder_t::survivors`, and `rx_fec_decoder_t::survivors_len`.

Referenced by `internal_viterbi_forward_pass()`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.13 `internal_viterbi_process_transition()`

```
void internal_viterbi_process_transition (
    rx_fec_decoder_t * dec,
    const rx_soft_bit_t soft0,
    const rx_soft_bit_t soft1,
    const uint32_t symbol_idx,
    const uint8_t state,
    const uint8_t input) [static]
```

Process one transition (state + input combination) in the Viterbi trellis.

Computes the branch metric for the given state/input pair and updates the next-state path metric and survivor bit if the new metric is better.

Definition at line 346 of file `rx_fec.c`.

```
00352 {
00353     const uint8_t exp0 = dec->branch_table[state][input][k_fec_output_g1];
00354     const uint8_t exp1 = dec->branch_table[state][input][k_fec_output_g2];
00355
00356     const int32_t branch_metric = internal_branch_metric(soft0, soft1, exp0, exp1);
00357
00358     const uint8_t next_state =
00359         (state » k_fec_bit_mask) | ((uint8_t)input « k_fec_state_shift_amount);
00360
00361     const int32_t new_metric = dec->path_metrics[state] + branch_metric;
00362
00363     if (new_metric < dec->new_path_metrics[next_state]) {
00364         dec->new_path_metrics[next_state] = new_metric;
00365
00366         dec->survivors[symbol_idx] &= ~((uint64_t)k_fec_bit_mask « (uint8_t)next_state);
00367         if ((state & k_fec_bit_mask) == k_fec_bit_mask) {
00368             dec->survivors[symbol_idx] |= ((uint64_t)k_fec_bit_mask « (uint8_t)next_state);
00369         }
00370     }
00371 }
```

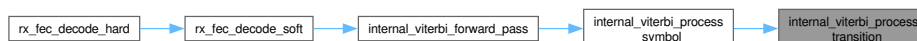
References `rx_fec_decoder_t::branch_table`, `internal_branch_metric()`, `k_fec_bit_mask`, `k_fec_output_g1`, `k_fec_output_g2`, `k_fec_state_shift_amount`, `rx_fec_decoder_t::new_path_metrics`, `rx_fec_decoder_t::path_metrics` and `rx_fec_decoder_t::survivors`.

Referenced by `internal_viterbi_process_symbol()`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.14 internal_viterbi_traceback()

```
RX_STATIC_TESTABLE void internal_viterbi_traceback (
    const rx_fec_decoder_t * dec,
    const uint32_t num_symbols,
    const uint32_t data_bits,
    uint8_t * output,
    const uint32_t output_bytes)
```

Perform Viterbi traceback to extract decoded bits.

Works backwards through the trellis from the terminal state to recover the maximum likelihood input sequence.

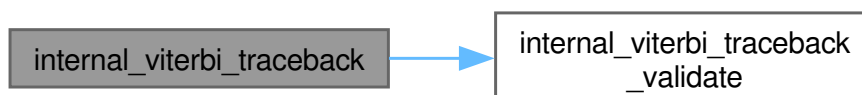
Definition at line 496 of file [rx_fec.c](#).

```
00501 {
00502     if (internal_viterbi_traceback_validate(dec, num_symbols, output, output_bytes)) {
00503         return;
00504     }
00505
00506     /* Clear output buffer */
00507     for (uint32_t i = k_fec_zero; i < output_bytes; i++) {
00508         output[i] = (uint8_t)k_fec_zero;
00509     }
00510
00511     /* Start from state 0 (encoder is flushed to zero by tail bits) */
00512     uint8_t state = k_fec_zero;
00513
00514     /* Traceback: work backwards through the trellis */
00515     uint32_t limit = num_symbols;
00516     if (num_symbols > k_fec_max_symbols) {
00517         limit = k_fec_max_symbols;
00518     }
00519     for (uint32_t t = limit; t > k_fec_zero; t--) {
00520         const uint32_t t_idx = t - k_fec_bit_mask;
00521
00522         /* The input bit is the MSB of the current state */
00523         const uint8_t input_bit = (uint8_t)((state » k_fec_state_shift_amount) & k_fec_bit_mask);
00524
00525         /* Store decoded bit if it's a data bit (not tail bit) */
00526         if (t_idx < data_bits) {
00527             if (input_bit != k_fec_zero) {
00528                 const uint32_t byte_idx = t_idx / k_rx_bits_per_byte;
00529                 const uint32_t bit_pos = k_fec_msb_bit_position - (t_idx % k_rx_bits_per_byte);
00530                 output[byte_idx] |= (uint8_t)(k_fec_bit_mask « bit_pos);
00531             }
00532         }
00533
00534         /* Get predecessor's LSB from survivors */
00535         const uint8_t predecessor_lsb = (uint8_t)((dec->survivors[t_idx] » state) & k_fec_bit_mask);
00536
00537         /* Compute predecessor state: shift left and insert the LSB */
00538         state = ((state « k_fec_bit_mask) & (k_fec_num_states - k_fec_bit_mask)) | predecessor_lsb;
00539     }
00540 }
```

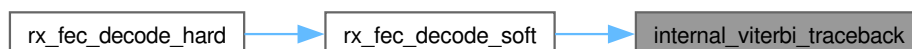
References [internal_viterbi_traceback_validate\(\)](#), [k_fec_bit_mask](#), [k_fec_max_symbols](#), [k_fec_msb_bit_position](#), [k_fec_num_states](#), [k_fec_state_shift_amount](#), [k_fec_zero](#), and [rx_fec_decoder_t::survivors](#).

Referenced by [rx_fec_decode_soft\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.15 `internal_viterbi_traceback_validate()`

```
RX_STATIC_TESTABLE bool internal_viterbi_traceback_validate (
    const rx_fec_decoder_t * dec,
    const uint32_t num_symbols,
    const uint8_t * output,
    const uint32_t output_bytes)
```

Validate traceback preconditions.

Definition at line 464 of file `rx_fec.c`.

```
00468 {
00469     RX_ASSERT_PRE(dec != nullptr, "Decoder handle must not be nullptr");
00470     RX_ASSERT_PRE(output != nullptr, "Output buffer must not be nullptr");
00471     RX_ASSERT_PRE(output_bytes > k_fec_zero, "Output bytes must be greater than zero");
00472     bool valid = true;
00473     #ifdef UNIT_TEST
00474     if ((dec == nullptr) || (output == nullptr) || (output_bytes == k_fec_zero)) {
00475         valid = false;
00476     }
00477     #endif
00478     if (valid) {
00479         RX_ASSERT_PRE(num_symbols <= dec->survivors_len, "Number of symbols exceeds survivors buffer");
00480         #ifdef UNIT_TEST
00481         if (num_symbols > dec->survivors_len) {
00482             valid = false;
00483         }
00484         #endif
00485     }
00486     return valid;
00487 }
```

References `k_fec_zero`, and `rx_fec_decoder_t::survivors_len`.

Referenced by `internal_viterbi_traceback()`.

Here is the caller graph for this function:

7.3.4.16 `rx_fec_decode_hard()`

```
rx_err_t rx_fec_decode_hard (
    rx_fec_decoder_t * dec,
    const rx_fec_decode_hard_params_t * params) [nodiscard]
```

Decode FEC-encoded data using hard-decision bits (converts to soft internally).

Decode hard bits using Viterbi algorithm.

Convenience wrapper that converts hard-decision bits (0/1) to soft-decision values and delegates to `rx_fec_decode_soft()`. Accepts raw bit-packed data (MSB-first) as produced by standard digital logic or RSPI transfers.

Algorithm steps:

1. Validate all input pointers and size constraints
2. Convert each hard bit to a soft value via `rx_fec_hard_to_soft()` (maps 0 -> negative confidence, 1 -> positive confidence)
3. Build `rx_fec_decode_soft_params_t` from the converted soft bits
4. Delegate to `rx_fec_decode_soft()` for full Viterbi decode

Precondition

dec must be initialized via [rx_fec_decoder_init\(\)](#)
 params->soft_bits_buffer must be large enough: soft_buffer_len >= data_len * 8
 params->data_len must be > 0 and <= k_fec_max_input_bytes

Postcondition

*params->output_len written with actual decoded byte count on k_rx_ok
 params->output contains decoded data bytes on k_rx_ok
 params->soft_bits_buffer populated with converted soft values (side effect)

Note

Not thread-safe; caller must ensure exclusive access to dec and params
 Hard-decision decoding has ~3 dB lower coding gain than soft-decision

See also

[rx_fec_decode_soft\(\)](#) Preferred: use when soft values are available
[rx_fec_decoder_init\(\)](#) Initialize decoder before calling this function
[rx_fec_hard_to_soft\(\)](#) Conversion function used internally
[rx_fec_encode\(\)](#) Encoder that produces data this function decodes

Since

Version 1.0.0

```
// Example: hard-decision decode of packed FEC-encoded bytes
rx_fec_decoder_t      dec;
uint64_t              survivors[k_fec_max_symbols];
rx_soft_bit_t          soft_scratch[k_fec_max_symbols * k_fec_num_outputs];
uint8_t               out_buf[k_fec_max_input_bytes];
uint32_t               out_len = k_fec_zero;

(void)rx_fec_decoder_init(&dec, survivors, k_fec_max_symbols);

rx_fec_decode_hard_params_t p = {
    .data          = received_bytes,
    .data_len      = received_len,
    .soft_bits_buffer = soft_scratch,
    .soft_buffer_len = k_fec_max_symbols * k_fec_num_outputs,
    .expected_output_len = original_data_len,
    .output         = out_buf,
    .output_len     = &out_len,
};
rx_err_t err = rx_fec_decode_hard(&dec, &p);
```

Definition at line 1052 of file [rx_fec.c](#).

```
01053 {
01054     if (dec == nullptr || params == nullptr) {
01055         return k_rx_err_invalid_arg;
01056     }
01057
01058     if (params->data == nullptr || params->output == nullptr || params->output_len == nullptr ||
01059         params->soft_bits_buffer == nullptr) {
01060         return k_rx_err_invalid_arg;
01061     }
01062
01063     if (!dec->initialized) {
01064         return k_rx_err_invalid_state;
01065     }
01066
01067     if (params->data_len == k_fec_zero) {
```

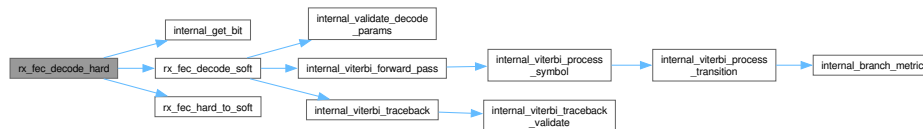
```

01068     return k_rx_err_invalid_arg;
01069 }
01070
01071 if (params->data_len > k_fec_max_input_bytes) {
01072     return k_rx_err_invalid_size;
01073 }
01074
01075 /* Convert hard bits to soft bits */
01076 const uint32_t num_bits = (uint32_t)(params->data_len * k_rx_bits_per_byte);
01077
01078 /* Ensure soft bits buffer is large enough */
01079 if (num_bits > params->soft_buffer_len) {
01080     return k_rx_err_invalid_size;
01081 }
01082
01083 for (uint32_t i = k_fec_zero; i < num_bits; i++) {
01084     const uint8_t bit = internal_get_bit(params->data, i);
01085     params->soft_bits_buffer[i] = rx_fec_hard_to_soft(bit);
01086 }
01087
01088 /* Prepare soft decode parameters */
01089 const rx_fec_decode_soft_params_t soft_params = {
01090     .soft_bits = params->soft_bits_buffer,
01091     .soft_len = num_bits,
01092     .expected_output_len = params->expected_output_len,
01093     .output = params->output,
01094     .output_len = params->output_len,
01095 };
01096
01097 return rx_fec_decode_soft(dec, &soft_params);
01098 }

```

References [rx_fec_decode_hard_params_t::data](#), [rx_fec_decode_hard_params_t::data_len](#), [rx_fec_decode_hard_params_t::initialized](#), [internal_get_bit\(\)](#), [k_fec_max_input_bytes](#), [k_fec_zero](#), [rx_fec_decode_hard_params_t::output_len](#), [rx_fec_decode_soft\(\)](#), [rx_fec_hard_to_soft\(\)](#), [rx_fec_decode_hard_params_t::soft_buffer_len](#).

Here is the call graph for this function:



7.3.4.17 rx_fec_decode_soft()

```

rx_err_t rx_fec_decode_soft (
    rx_fec_decoder_t * dec,
    const rx_fec_decode_soft_params_t * params) [nodiscard]

```

Decode FEC-encoded data using soft-decision Viterbi algorithm.

Decode soft bits using Viterbi algorithm.

Implements Rate-1/2 Viterbi soft-decision decoding with constraint length 7. Accepts soft-decision values (signed integers in range [-128, 127]) to achieve approximately 3 dB coding gain over hard-decision decoding.

Algorithm steps:

1. Validate parameters via [internal_validate_decode_params\(\)](#)
2. Forward pass: compute branch and path metrics for all symbols
3. Calculate data bit count (num_symbols - tail bits)
4. Traceback: extract maximum-likelihood decoded bit sequence
5. Pack decoded bits into output byte array

Precondition

dec must be initialized via [rx_fec_decoder_init\(\)](#)
 params->soft_len must be divisible by 2 (G1/G2 symbol pairs)
 params->expected_output_len must be > 0 and <= k_fec_max_input_bytes

Postcondition

*params->output_len written with actual decoded byte count on k_rx_ok
 params->output contains decoded data bytes on k_rx_ok
 dec path metric arrays updated (internal state consumed by traceback)

Note

Not thread-safe; caller must ensure exclusive access to dec and params
 Soft values: positive = bit-1 likely, negative = bit-0 likely (signed).

See also

[rx_fec_decoder_init\(\)](#) Initialize decoder before calling this function
[rx_fec_decode_hard\(\)](#) Hard-decision wrapper (lower coding gain, simpler input)
[rx_fec_encode\(\)](#) Encoder that produces the data this function decodes

Since

Version 1.0.0

```
// Example: soft-decision decode of a previously FEC-encoded buffer
rx_fec_decoder_t dec;
uint64_t survivors[k_fec_max_symbols];
rx_soft_bit_t soft_buf[k_fec_max_symbols * k_fec_num_outputs];
uint8_t out_buf[k_fec_max_input_bytes];
uint32_t out_len = k_fec_zero;

(void)rx_fec_decoder_init(&dec, survivors, k_fec_max_symbols);

// Populate soft_buf from channel (positive = bit-1 likely)
rx_fec_decode_soft_params_t p = {
    .soft_bits = soft_buf,
    .soft_len = encoded_len_bits,
    .expected_output_len = original_data_len,
    .output = out_buf,
    .output_len = &out_len,
};
rx_err_t err = rx_fec_decode_soft(&dec, &p);
```

Definition at line 972 of file [rx_fec.c](#).

```
00973 {
00974     uint32_t num_symbols = k_fec_zero;
00975     const rx_err_t err = internal_validate_decode_params(dec, params, &num_symbols);
00976     if (err != k_rx_ok) {
00977         return err;
00978     }
00979
00980     /* Forward pass: initialize path metrics and process symbols */
00981     internal_viterbi_forward_pass(dec, params->soft_bits, num_symbols);
00982
00983     /* Calculate data bits and output size */
00984     const uint32_t data_bits = num_symbols - k_fec_tail_bits;
00985
00986     const uint32_t output_bytes = (data_bits + k_fec_msb_bit_position) / k_rx_bits_per_byte;
00987
00988     /* Traceback: extract decoded bits */
00989     internal_viterbi_traceback(dec, num_symbols, data_bits, params->output, output_bytes);
00990 }
```

```

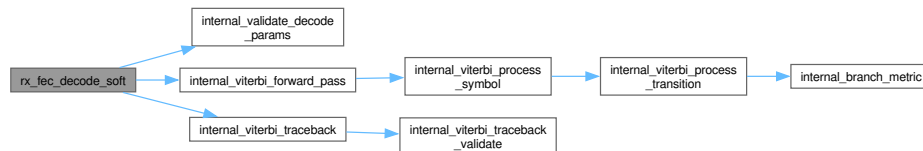
00991 *params->output_len = output_bytes;
00992 return k_rx_ok;
00993 }

```

References [internal_validate_decode_params\(\)](#), [internal_viterbi_forward_pass\(\)](#), [internal_viterbi_traceback\(\)](#), [k_fec_msb_bit_position](#), [k_fec_tail_bits](#), [k_fec_zero](#), [rx_fec_decode_soft_params_t::output](#), [rx_fec_decode_soft_params_t::output_len](#), and [rx_fec_decode_soft_params_t::soft_bits](#).

Referenced by [rx_fec_decode_hard\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.18 rx_fec_decoder_deinit()

```

rx_err_t rx_fec_decoder_deinit (
    rx_fec_decoder_t * dec) [nodiscard]

```

Deinitialize FEC decoder.

Parameters

in,out	dec	Pointer to decoder handle
--------	-----	---------------------------

Returns

[k_rx_ok](#) on success

Definition at line 905 of file [rx_fec.c](#).

```

00906 {
00907     if (dec == nullptr) {
00908         return k_rx_err_invalid_arg;
00909     }
00910
00911     dec->survivors = nullptr;
00912     dec->survivors_len = k_fec_zero;
00913     dec->initialized = false;
00914     return k_rx_ok;
00915 }

```

References [rx_fec_decoder_t::initialized](#), [k_fec_zero](#), [rx_fec_decoder_t::survivors](#), and [rx_fec_decoder_t::survivors_len](#).

7.3.4.19 rx_fec_decoder_init()

```
rx_err_t rx_fec_decoder_init (
    rx_fec_decoder_t * dec,
    uint64_t * survivors_buf,
    const uint32_t survivors_len)
```

Initialize FEC decoder.

Initializes decoder state and associates it with provided survivors buffer. The buffer is used for Viterbi algorithm path storage.

Precondition

dec and survivors_buf must be non-nullptr
 survivors_len must be > 0 (minimum 1 entry per symbol)

Postcondition

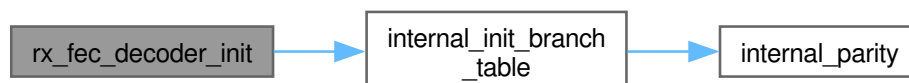
dec->initialized set to true
 Branch table initialized

Definition at line 879 of file rx_fec.c.

```
00880 {
00881     if (dec == nullptr || survivors_buf == nullptr) {
00882         return k_rx_err_invalid_arg;
00883     }
00884
00885     /* Minimum survivors buffer size: at least 1 entry per symbol */
00886     if (survivors_len == k_fec_zero) {
00887         return k_rx_err_invalid_size;
00888     }
00889
00890     dec->survivors = survivors_buf;
00891     dec->survivors_len = survivors_len;
00892
00893     /* Initialize branch table */
00894     internal_init_branch_table(dec->branch_table);
00895
00896     dec->initialized = true;
00897     return k_rx_ok;
00898 }
```

References [rx_fec_decoder_t::branch_table](#), [rx_fec_decoder_t::initialized](#), [internal_init_branch_table\(\)](#), [k_fec_zero](#), [rx_fec_decoder_t::survivors](#), and [rx_fec_decoder_t::survivors_len](#).

Here is the call graph for this function:



7.3.4.20 rx_fec_encode()

```
rx_err_t rx_fec_encode (
    const rx_fec_encoder_t * enc,
    const uint8_t * input,
    const uint32_t input_len,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]
```

Encode data using convolutional FEC.

Encode data using K=7, rate 1/2 convolutional code.

Applies rate-1/2, constraint-length-7 convolutional encoding to input data. Output length is approximately 2x input length.

Precondition

- enc must be initialized via [rx_fec_encoder_init\(\)](#)
- All pointers must be non-nullptr
- input_len in range [1, k_fec_max_input_bytes]

Postcondition

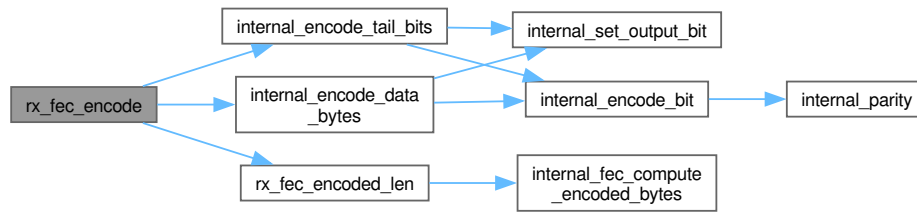
- output buffer contains FEC-encoded data
- *output_len set to actual encoded length

Definition at line 738 of file [rx_fec.c](#).

```
00743 {
00744     if (enc == nullptr || input == nullptr || output == nullptr || output_len == nullptr) {
00745         return k_rx_err_invalid_arg;
00746     }
00747     if (!enc->initialized) {
00748         return k_rx_err_invalid_state;
00749     }
00750 }
00751
00752 if (input_len == k_fec_zero) {
00753     return k_rx_err_invalid_arg;
00754 }
00755
00756 if (input_len > k_fec_max_input_bytes) {
00757     return k_rx_err_invalid_size;
00758 }
00759
00760 /* Calculate output size and clear output buffer */
00761 const uint32_t expected_output_len = rx_fec_encoded_len(input_len);
00762 for (uint32_t i = k_fec_zero; i < expected_output_len; i++) {
00763     output[i] = (uint8_t)k_fec_zero;
00764 }
00765
00766 /* Encode input bytes then flush with tail bits */
00767 /* input_len validated <= k_fec_max_input_bytes above; use directly. */
00768 uint8_t state = k_fec_zero;
00769 uint32_t out_bit_idx = k_fec_zero;
00770 const uint32_t byte_limit = input_len;
00771
00772 internal_encode_data_bytes(input, byte_limit, output, &state, &out_bit_idx);
00773 internal_encode_tail_bits(output, &state, &out_bit_idx);
00774
00775 *output_len = expected_output_len;
00776 return k_rx_ok;
00777 }
```

References [rx_fec_encoder_t::initialized](#), [internal_encode_data_bytes\(\)](#), [internal_encode_tail_bits\(\)](#), [k_fec_max_input_bytes](#), [k_fec_zero](#), and [rx_fec_encoded_len\(\)](#).

Here is the call graph for this function:



7.3.4.21 rx_fec_encoded_len()

```
uint32_t rx_fec_encoded_len (
    const uint32_t input_len)
```

Calculate encoded output length for given input.

Calculate FEC encoded output length.

Computes the number of bytes required for FEC-encoded output based on input data length. Accounts for tail bits and rate-1/2 encoding.

Precondition

input_len must be in range [1, k_fec_max_input_bytes]
 k_fec_max_input_bytes <= 1024 (guarantees no overflow in arithmetic)

Postcondition

Return value is 0 if input_len is 0 or > k_fec_max_input_bytes
 Return value > 0 for valid input_len

Note

Thread-safe (pure computation, no shared state)

Since

Version 1.0.0

Definition at line 661 of file rx_fec.c.

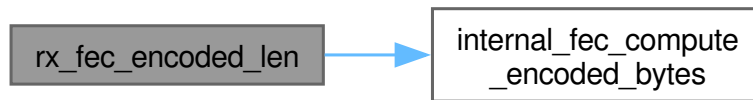
```

00662 {
00663     if (input_len == k_fec_zero) {
00664         return k_fec_zero;
00665     }
00666
00667     if (input_len > k_fec_max_input_bytes) {
00668         return k_fec_zero;
00669     }
00670
00671     /* Validated: input_len in [1, k_fec_max_input_bytes], no overflow possible */
00672     uint32_t result = k_fec_zero;
00673     (void)internal_fec_compute_encoded_bytes(input_len, &result);
00674     return result;
00675 }
```

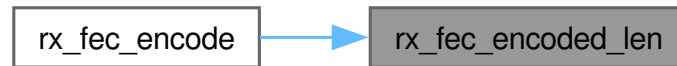
References [internal_fec_compute_encoded_bytes\(\)](#), [k_fec_max_input_bytes](#), and [k_fec_zero](#).

Referenced by [rx_fec_encode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.4.22 rx_fec_encoder_deinit()

```
rx_err_t rx_fec_encoder_deinit (
    rx_fec_encoder_t * enc) [nodiscard]
```

Deinitialize FEC encoder.

Marks encoder as uninitialized. After this call, encoder must be re-initialized before use. This function provides symmetric init/deinit API for resource management patterns.

Algorithm:

1. Validate enc pointer (NULL check)
2. Set `enc->initialized = false`
3. Return success

Parameters

in,out	enc	Pointer to encoder handle to deinitialize
		• Must not be nullptr • May be uninitialized (idempotent) • Contents will be cleared

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Successfully deinitialized
<code>k_rx_err_invalid_arg</code>	enc is nullptr

Precondition

enc must point to valid memory (may be uninitialized)

Postcondition

enc->initialized == false

Encoder cannot be used until [rx_fec_encoder_init\(\)](#) called again

Note

Thread Safety: Safe to call concurrently on different encoder handles. NOT safe to call concurrently on the same handle.

Idempotent: Safe to call multiple times on same encoder.

Warning

Do NOT call [rx_fec_encode\(\)](#) after deinit - will return k_rx_err_invalid_state

Example

```
rx_fec_encoder_t encoder;
rx_fec_encoder_init(&encoder);

// ... use encoder ...

// Cleanup (always succeeds if enc is valid pointer)
rx_err_t err = rx_fec_encoder_deinit(&encoder);
assert(err == k_rx_ok);

// Re-initialization allowed
rx_fec_encoder_init(&encoder);
```

See also

[rx_fec_encoder_init\(\)](#) Initialize encoder

Since

Version 1.0.0

Definition at line 567 of file [rx_fec.c](#).

```
00568 {
00569     if (enc == nullptr) {
00570         return k_rx_err_invalid_arg;
00571     }
00572
00573     enc->initialized = false;
00574     return k_rx_ok;
00575 }
```

References [rx_fec_encoder_t::initialized](#).

7.3.4.23 rx_fec_encoder_init()

```
rx_err_t rx_fec_encoder_init (
    rx_fec_encoder_t * enc) [nodiscard]
```

Initialize FEC encoder.

Prepares encoder for use by setting initialization flag. The encoder itself is stateless (no shift register persists between calls), so initialization only validates the handle.

Algorithm:

1. Validate enc pointer (NULL check)
2. Set enc->initialized = true
3. Return success

Parameters

out	enc	Pointer to encoder handle to initialize
		• Must not be nullptr • Contents will be overwritten • Lifetime: Caller-managed, must persist until deinit

Returns

rx_err_t Error code

Return values

k_rx_ok	Successfully initialized, encoder ready for use
k_rx_err_invalid_arg	enc is nullptr

Precondition

enc must point to valid rx_fec_encoder_t storage
Sufficient stack/heap space for encoder structure (4 bytes)

Postcondition

enc->initialized == true
Encoder is ready for rx_fec_encode() calls

Note

Thread Safety: Safe to call concurrently on different encoder handles. NOT safe to call concurrently on the same handle.

Re-initialization: Safe to call on already-initialized encoder (idempotent operation).

Example

```

// Stack allocation
rx_fec_encoder_t encoder;
rx_err_t err = rx_fec_encoder_init(&encoder);
if (err != k_rx_ok) {
    // Handle error
    return err;
}

// Now ready for encoding
uint8_t input[100] = {...};
uint8_t output[250];
uint32_t output_len;
rx_fec_encode(&encoder, input, 100, output, &output_len);

// Cleanup
rx_fec_encoder_deinit(&encoder);

```

See also

[rx_fec_encoder_deinit\(\)](#) Deinitialize encoder
[rx_fec_encode\(\)](#) Encode data

Since

Version 1.0.0

Definition at line 552 of file [rx_fec.c](#).

```

00553 {
00554     if (enc == nullptr) {
00555         return k_rx_err_invalid_arg;
00556     }
00557     enc->initialized = true;
00558     return k_rx_ok;
00559 }
00560 }

```

References [rx_fec_encoder_t::initialized](#).

7.4 rx_fec.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_fec.c
00003  * @brief Forward Error Correction (FEC) Codec Implementation
00004  *
00005  * @details
00006  * Implements NASA-standard K=7, rate 1/2 convolutional encoder and soft-decision
00007  * Viterbi decoder. This is a C port of star-gateway/internal/fec/ ensuring
00008  * bit-exact compatibility between RX72N firmware and RPi5 gateway.
00009  *
00010  * ## Implementation Architecture
00011  *
00012  * ### Encoder (Stateless)
00013  * - Each encode() call starts from zero state
00014  * - Processes input MSB-first (network byte order)
00015  * - Appends 6 tail bits to flush to zero
00016  * - Output packed MSB-first into bytes
00017  *
00018  * ### Decoder (Viterbi Algorithm)
00019  * - **Forward Pass**: Compute path metrics for all 64 states at each time step
00020  * - **Traceback**: Reconstruct most-likely input sequence from survivors
00021  * - **Soft Decision**: Uses branch metrics based on soft bit correlation
00022  * - **Memory**: Survivors buffer stores predecessor bits for traceback
00023  *
00024  * ## Algorithm Complexity
00025  *
00026  * | Operation | Time Complexity | Space Complexity |
00027  * |-----|-----|-----|

```

```

00028 * | Encoding          | O(n)          | O(1)          |
00029 * | Decoding (Viterbi)| O(n x 64 x 2) | O(n x 64 bits) |
00030 * | Traceback        | O(n)          | O(1)          |
00031 *
00032 * where n = number of input bits.
00033 *
00034 * ## Viterbi Algorithm Overview
00035 *
00036 * The Viterbi algorithm finds the maximum likelihood path through the trellis
00037 * of encoder states using dynamic programming:
00038 *
00039 * 1. **Initialization**: Set path metric for state 0 to 0, all others to inf
00040 * 2. **Forward Pass**: For each received symbol pair (r_0, r_1):
00041 *   - For each current state with finite metric:
00042 *     - Compute metrics for both possible transitions (input=0, input=1)
00043 *     - Update next state metrics (keep minimum)
00044 *     - Store survivor path (which previous state was best)
00045 * 3. **Traceback**: Starting from final state 0 (encoder flushed), work backwards
00046 *   through survivors to recover input sequence
00047 *
00048 * ## Branch Metric Calculation
00049 *
00050 * Uses correlation metric for soft-decision decoding:
00051 *
00052 * @f[
00053 *   M_{\text{branch}} = 32768 - (r_0 \cdot e_0' + r_1 \cdot e_1')
00054 * @f]
00055 *
00056 * where:
00057 * - @f$ r_i @f$ = received soft bit (-127 to +127)
00058 * - @f$ e_i' = +127 @f$ if expected bit is 1, @f$ -127 @f$ if 0
00059 * - Lower metric = better match
00060 *
00061 * ## Bit Ordering Convention
00062 *
00063 * **Throughout this implementation: MSB-first (network byte order)**
00064 * - Bit index 0 -> byte[0] bit 7 (MSB)
00065 * - Bit index 7 -> byte[0] bit 0 (LSB)
00066 * - Bit index 8 -> byte[1] bit 7 (MSB)
00067 *
00068 * This matches wire protocols and simplifies interoperability.
00069 *
00070 * @par Module Dependencies
00071 * - [rx_fec.h](rx_fec_8h.html): Public API declarations
00072 * - [rx_check.h](rx_check_8h.html): Assertion macros (RX_ASSERT)
00073 * - [rx_bit_constants.h](rx_bit_constants_8h.html): Bit manipulation constants
00074 * - No standard library headers required (memset replaced with for-loop)
00075 *
00076 * @par NASA Power of 10 Compliance
00077 * - **Rule 1**: [OK] No recursion, goto, or setjmp/longjmp
00078 * - **Rule 2**: [OK] All loops bounded (k_fec_max_symbols, k_fec_num_states)
00079 * - **Rule 3**: [OK] Zero dynamic allocation
00080 * - **Rule 4**: [OK] All functions <= 60 lines
00081 * - **Rule 5**: [OK] All functions have >=2 assertions
00082 * - **Rule 10**: [OK] Compiles with -Wall -Wextra -Werror
00083 *
00084 * @author Locked, Inc.
00085 * @date 2026-01-27
00086 * @copyright Copyright (c) 2026 Locked Inc.
00087 * SPDX-License-Identifier: MIT
00088 *
00089 * @since Version 1.0.0
00090 * @version 1.0.0
00091 */
00092
00093 #include "rx_fec.h"
00094
00095 /*
00096 * C23 <stdckdint.h> / <stdbit.h>: GNURX GCC 14.2's newlib sysroot predates
00097 * C23 library support, so not all headers are available. We gate each behind
00098 * __has_include and provide manual fallbacks. clang-18 has both; GNURX has
00099 * <stdckdint.h> but not <stdbit.h>. Once Renesas ships an updated newlib,
00100 * remove the #if guards and the fallback code paths.
00101 */
00102 #if __has_include(<stdckdint.h>)
00103 #include <stdckdint.h>
00104 #define RX_HAS_STDCKDINT (1)
00105 #else
00106 #define RX_HAS_STDCKDINT (0)
00107 #endif
00108
00109 #if __has_include(<stdbit.h>)
00110 #include <stdbit.h>
00111 #define RX_HAS_STDBIT (1)
00112 #else
00113 #define RX_HAS_STDBIT (0)
00114 #endif

```

```

00115
00116 #include "rx_bit_constants.h"
00117 #include "rx_check.h"
00118
00119 /*
=====
00120 * Private Constants
00121 *
=====
00122 */
00123
00124 /**
00125 * @brief FEC implementation constants
00126 */
00127 typedef enum : uint16_t {
00128     k_fec_zero = 0, /**< Zero constant */
00129     k_fec_msb_bit_position = 7, /**< MSB position in byte (0-indexed) */
00130     k_fec_shift_register_bits = 6, /**< K-1 shift register size */
00131     k_fec_correlation_offset = 32768, /**< Correlation metric offset (2^15) */
00132     k_fec_state_shift_amount = 5, /**< k_fec_constraint_length - 2 */
00133     k_fec_bit_mask = 1, /**< Mask for extracting single bit */
00134     k_fec_parity_shift_nibble = 4, /**< Shift amount for nibble XOR in parity computation */
00135     k_fec_parity_shift_pair = 2, /**< Shift amount for bit-pair XOR in parity computation */
00136 } rx_fec_impl_constants_t;
00137
00138 /**
00139 * @brief FEC output indices for G1 and G2 generator polynomials
00140 */
00141 typedef enum : uint8_t {
00142     k_fec_output_g1 = 0, /**< G1 generator output (first encoded bit) */
00143     k_fec_output_g2 = 1, /**< G2 generator output (second encoded bit) */
00144 } rx_fec_output_index_t;
00145
00146 /*
=====
00147 * Private Helper Functions
00148 *
=====
00149 */
00150
00151 /**
00152 * @brief Calculate parity (XOR of all bits) of a byte
00153 *
00154 * Uses parallel XOR reduction to compute parity in O(log n) operations.
00155 * This is more efficient than looping through each bit.
00156 *
00157 * Algorithm:
00158 * - Step 1 (x ^= x >> 4): XOR upper nibble with lower nibble
00159 *   Original: [b7 b6 b5 b4][b3 b2 b1 b0]
00160 *   Result:  [x x x x][b7^b3 b6^b2 b5^b1 b4^b0]
00161 *
00162 * - Step 2 (x ^= x >> 2): XOR pairs of bits
00163 *   Result bits [1:0] now contain XOR of all 8 original bits
00164 *
00165 * - Step 3 (x ^= x >> 1): Final XOR to get single parity bit
00166 *   Result bit [0] = XOR of all original bits
00167 *
00168 */
00169 RX_STATIC_TESTABLE uint8_t internal_parity(uint8_t x)
00170 {
00171     #if RX_HAS_STDBIT
00172         /* C23 stdc_count_ones_uc: UB-free popcount; parity = LSB of popcount */
00173         const uint8_t result = (uint8_t)(stdc_count_ones_uc(x) & k_fec_bit_mask);
00174     #else
00175         /* Fallback: parallel XOR reduction (GNURX lacks <stdbit.h>) */
00176         x ^= x >> k_fec_parity_shift_nibble;
00177         x ^= x >> k_fec_parity_shift_pair;
00178         x ^= x >> k_fec_bit_mask;
00179         const uint8_t result = x & k_fec_bit_mask;
00180     #endif
00181
00182     /* Invariant: bitwise AND with 1 always yields 0 or 1; no runtime check needed */
00183
00184     return result;
00185 }
00186
00187 /**
00188 * @brief Set a bit at specified index in output buffer (MSB first)
00189 *
00190 * Bit ordering (MSB first, network byte order):
00191 *   bit_idx=0 -> byte 0, bit 7 (MSB)
00192 *   bit_idx=7 -> byte 0, bit 0 (LSB)
00193 *   bit_idx=8 -> byte 1, bit 7 (MSB)
00194 *
00195 */
00196 RX_STATIC_TESTABLE void
00197 internal_set_output_bit(uint8_t* output, const uint32_t bit_idx, uint8_t value)

```

```

00198 {
00199     /* Pre-condition 1: output must be valid */
00200     RX_ASSERT_PRE(output != nullptr, "Output buffer must not be nullptr");
00201
00202     /* Pre-condition 2: bit index must be within maximum symbol range */
00203     RX_ASSERT_PRE(bit_idx < (k_fec_max_symbols * k_rx_bits_per_byte), "Bit index out of range");
00204
00205     #ifndef UNIT_TEST
00206     if ((output == nullptr) || (bit_idx >= (k_fec_max_symbols * k_rx_bits_per_byte))) {
00207         return;
00208     }
00209     #endif
00210
00211     value = (value != 0) ? k_fec_bit_mask : 0;
00212
00213     const uint32_t byte_idx = bit_idx / k_rx_bits_per_byte;
00214     const uint32_t bit_pos = k_fec_msb_bit_position - (bit_idx % k_rx_bits_per_byte); /* MSB first */
00215
00216     if (value != 0) {
00217         output[byte_idx] |= (uint8_t)(1U « bit_pos);
00218     }
00219 }
00220
00221 /**
00222  * @brief Get a bit at specified index from buffer (MSB first)
00223  *
00224  * Bit ordering (MSB first, network byte order):
00225  *   bit_idx=0 -> byte 0, bit 7 (MSB)
00226  *   bit_idx=7 -> byte 0, bit 0 (LSB)
00227  *   bit_idx=8 -> byte 1, bit 7 (MSB)
00228  *
00229  */
00230 RX_STATIC_TESTABLE uint8_t internal_get_bit(const uint8_t* data, uint32_t bit_idx)
00231 {
00232     /* Pre-condition: data must be valid */
00233     RX_ASSERT_PRE(data != nullptr, "Data buffer must not be nullptr");
00234
00235     #ifndef UNIT_TEST
00236     if (data == nullptr) {
00237         return 0;
00238     }
00239     #endif
00240
00241     const uint32_t byte_idx = bit_idx / k_rx_bits_per_byte;
00242     const uint32_t bit_pos = k_fec_msb_bit_position - (bit_idx % k_rx_bits_per_byte); /* MSB first */
00243
00244     const uint8_t result = (data[byte_idx] » bit_pos) & k_fec_bit_mask;
00245
00246     /* Invariant: bitwise AND with 1 always yields 0 or 1; no runtime check needed */
00247
00248     return result;
00249 }
00250
00251 /**
00252  * @brief Encode a single bit and update encoder state
00253  *
00254  */
00255 RX_STATIC_TESTABLE void
00256 internal_encode_bit(uint8_t* state, const uint8_t input_bit, uint8_t* out0, uint8_t* out1)
00257 {
00258     /* Pre-condition 1: All pointers must be valid */
00259     RX_ASSERT_PRE(state != nullptr, "State pointer must not be nullptr");
00260     RX_ASSERT_PRE(out0 != nullptr, "Output 0 pointer must not be nullptr");
00261     RX_ASSERT_PRE(out1 != nullptr, "Output 1 pointer must not be nullptr");
00262
00263     /* Pre-condition 2: input_bit must be 0 or 1 */
00264     RX_ASSERT_PRE((input_bit == k_fec_zero) || (input_bit == k_fec_bit_mask),
00265         "Input bit must be 0 or 1");
00266
00267     #ifndef UNIT_TEST
00268     if ((state == nullptr) || (out0 == nullptr) || (out1 == nullptr)) {
00269         return;
00270     }
00271     #endif
00272
00273     /* Shift in the new bit (input is MSB of the combined state) */
00274     const uint8_t combined = (uint8_t)((input_bit « k_fec_shift_register_bits) | *state);
00275
00276     /* Calculate output bits using generator polynomials */
00277     *out0 = internal_parity(combined & k_fec_g1_octal);
00278     *out1 = internal_parity(combined & k_fec_g2_octal);
00279
00280     /* Update state (shift right, new bit enters from left) */
00281     *state = combined » k_fec_bit_mask;
00282 }
00283
00284 /**

```

```

00285 * @brief Initialize the branch table for Viterbi decoder
00286 *
00287 * Precomputes expected output bits for each state and input combination.
00288 *
00289 */
00290 static void internal_init_branch_table(
00291     uint8_t branch_table[k_fec_num_states][k_fec_num_input_values][k_fec_num_outputs])
00292 {
00293     /* Pre-condition: branch_table array parameter is always valid in C */
00294     /* Loop bounds guarantee state < k_fec_num_states and input < k_fec_num_input_values */
00295     for (uint8_t state = k_fec_zero; state < k_fec_num_states; state++) {
00296         for (uint8_t input = k_fec_zero; input < k_fec_num_input_values; input++) {
00297             const uint8_t combined = (uint8_t)((input « k_fec_shift_register_bits) | state);
00298             branch_table[state][input][k_fec_output_g1] = internal_parity(combined & k_fec_g1_octal);
00299             branch_table[state][input][k_fec_output_g2] = internal_parity(combined & k_fec_g2_octal);
00300         }
00301     }
00302 }
00303 }
00304
00305 /**
00306 * @brief Compute branch metric for soft decoding
00307 *
00308 * Uses correlation metric: higher correlation = lower metric (better).
00309 * Formula: 32768 - (soft0 * exp0_soft + soft1 * exp1_soft)
00310 *
00311 */
00312 RX_STATIC_TESTABLE int32_t internal_branch_metric(const rx_soft_bit_t soft0,
00313     const rx_soft_bit_t soft1,
00314     const uint8_t exp0,
00315     const uint8_t exp1)
00316 {
00317     /* Pre-conditions: expected bits must be 0 or 1 */
00318     RX_ASSERT_PRE((exp0 == k_fec_zero) || (exp0 == k_fec_bit_mask), "Expected bit 0 must be 0 or 1");
00319     RX_ASSERT_PRE((exp1 == k_fec_zero) || (exp1 == k_fec_bit_mask), "Expected bit 1 must be 0 or 1");
00320
00321 #ifdef UNIT_TEST
00322     if (((exp0 != k_fec_zero) && (exp0 != k_fec_bit_mask)) ||
00323         ((exp1 != k_fec_zero) && (exp1 != k_fec_bit_mask))) {
00324         return 0;
00325     }
00326 #endif
00327
00328     /* Convert expected bits to soft values: 0 -> -127, 1 -> +127 */
00329     const int32_t exp0_soft = (exp0 != 0) ? k_soft_bit_max : k_soft_bit_min;
00330     const int32_t exp1_soft = (exp1 != 0) ? k_soft_bit_max : k_soft_bit_min;
00331
00332     /* Correlation = soft0*exp0_soft + soft1*exp1_soft */
00333     const int32_t correlation = ((int32_t)soft0 * exp0_soft) + ((int32_t)soft1 * exp1_soft);
00334
00335     /* Negate and offset to keep metrics positive (lower is better) */
00336     return k_fec_correlation_offset - correlation;
00337 }
00338
00339 /**
00340 * @brief Process one transition (state + input combination) in the Viterbi trellis
00341 *
00342 * Computes the branch metric for the given state/input pair and updates the
00343 * next-state path metric and survivor bit if the new metric is better.
00344 *
00345 */
00346 static void internal_viterbi_process_transition(rx_fec_decoder_t* dec,
00347     const rx_soft_bit_t soft0,
00348     const rx_soft_bit_t soft1,
00349     const uint32_t symbol_idx,
00350     const uint8_t state,
00351     const uint8_t input)
00352 {
00353     const uint8_t exp0 = dec->branch_table[state][input][k_fec_output_g1];
00354     const uint8_t exp1 = dec->branch_table[state][input][k_fec_output_g2];
00355
00356     const int32_t branch_metric = internal_branch_metric(soft0, soft1, exp0, exp1);
00357
00358     const uint8_t next_state =
00359         (state » k_fec_bit_mask) | ((uint8_t)input « k_fec_state_shift_amount);
00360
00361     const int32_t new_metric = dec->path_metrics[state] + branch_metric;
00362
00363     if (new_metric < dec->new_path_metrics[next_state]) {
00364         dec->new_path_metrics[next_state] = new_metric;
00365
00366         dec->survivors[symbol_idx] &= ~((uint64_t)k_fec_bit_mask « (uint8_t)next_state);
00367         if ((state & k_fec_bit_mask) == k_fec_bit_mask) {
00368             dec->survivors[symbol_idx] |= ((uint64_t)k_fec_bit_mask « (uint8_t)next_state);
00369         }
00370     }
00371 }

```

```

00372
00373 /**
00374  * @brief Process one symbol pair through the Viterbi trellis
00375  *
00376  * Updates path metrics and survivors for one time step.
00377  *
00378  */
00379 RX_STATIC_TESTABLE void internal_viterbi_process_symbol(rx_fec_decoder_t* dec,
00380                                                         const rx_soft_bit_t soft0,
00381                                                         const rx_soft_bit_t soft1,
00382                                                         const uint32_t symbol_idx)
00383 {
00384     /* Pre-conditions: valid decoder handle, symbol index in bounds */
00385     RX_ASSERT_PRE(dec != nullptr, "Decoder handle must not be nullptr");
00386     #ifndef UNIT_TEST
00387     if (dec == nullptr) {
00388         return;
00389     }
00390     #endif
00391     RX_ASSERT_PRE(symbol_idx < dec->survivors_len, "Symbol index exceeds survivors buffer length");
00392     #ifndef UNIT_TEST
00393     if (symbol_idx >= dec->survivors_len) {
00394         return;
00395     }
00396     #endif
00397
00398     /* Reset new path metrics and clear survivors for this time step */
00399     for (uint8_t i = k_fec_zero; i < k_fec_num_states; i++) {
00400         dec->new_path_metrics[i] = INT32_MAX;
00401     }
00402     dec->survivors[symbol_idx] = k_fec_zero;
00403
00404     /* For each current state, compute transitions */
00405     for (uint8_t state = k_fec_zero; state < k_fec_num_states; state++) {
00406         if (dec->path_metrics[state] == INT32_MAX) {
00407             continue;
00408         }
00409
00410         for (uint8_t input = k_fec_zero; input < k_fec_num_input_values; input++) {
00411             internal_viterbi_process_transition(dec, soft0, soft1, symbol_idx, state, input);
00412         }
00413     }
00414
00415     /* Swap path metrics */
00416     for (uint8_t i = k_fec_zero; i < k_fec_num_states; i++) {
00417         dec->path_metrics[i] = dec->new_path_metrics[i];
00418         dec->new_path_metrics[i] = INT32_MAX;
00419     }
00420 }
00421
00422 /**
00423  * @brief Run Viterbi forward pass through trellis
00424  *
00425  * Initializes path metrics and processes all symbols through the trellis.
00426  *
00427  */
00428 RX_STATIC_TESTABLE void internal_viterbi_forward_pass(rx_fec_decoder_t* dec,
00429                                                         const rx_soft_bit_t* soft_bits,
00430                                                         const uint32_t num_symbols)
00431 {
00432     /* Pre-conditions */
00433     RX_ASSERT_PRE(dec != nullptr, "Decoder handle must not be nullptr");
00434     RX_ASSERT_PRE(soft_bits != nullptr, "Soft bits buffer must not be nullptr");
00435
00436     #ifndef UNIT_TEST
00437     if ((dec == nullptr) || (soft_bits == nullptr)) {
00438         return;
00439     }
00440     #endif
00441
00442     /* Initialize path metrics: state 0 = 0, others = MAX */
00443     for (uint8_t i = k_fec_zero; i < k_fec_num_states; i++) {
00444         dec->path_metrics[i] = INT32_MAX;
00445     }
00446     dec->path_metrics[k_fec_zero] = k_fec_zero;
00447
00448     /* Process each symbol pair through the trellis */
00449     uint32_t limit = num_symbols;
00450     if (num_symbols > k_fec_max_symbols) {
00451         limit = k_fec_max_symbols;
00452     }
00453     for (uint32_t t = k_fec_zero; t < limit; t++) {
00454         const rx_soft_bit_t soft0 = soft_bits[t * k_fec_num_outputs + k_fec_output_g1];
00455         const rx_soft_bit_t soft1 = soft_bits[t * k_fec_num_outputs + k_fec_output_g2];
00456         internal_viterbi_process_symbol(dec, soft0, soft1, t);
00457     }
00458 }

```

```

00459
00460 /**
00461  * @brief Validate traceback preconditions
00462  *
00463  */
00464 RX_STATIC_TESTABLE bool internal_viterbi_traceback_validate(const rx_fec_decoder_t* dec,
00465                                                             const uint32_t    num_symbols,
00466                                                             const uint8_t*    output,
00467                                                             const uint32_t    output_bytes)
00468 {
00469     RX_ASSERT_PRE(dec != nullptr, "Decoder handle must not be nullptr");
00470     RX_ASSERT_PRE(output != nullptr, "Output buffer must not be nullptr");
00471     RX_ASSERT_PRE(output_bytes > k_fec_zero, "Output bytes must be greater than zero");
00472     bool valid = true;
00473     #ifdef UNIT_TEST
00474     if ((dec == nullptr) || (output == nullptr) || (output_bytes == k_fec_zero)) {
00475         valid = false;
00476     }
00477     #endif
00478     if (valid) {
00479         RX_ASSERT_PRE(num_symbols <= dec->survivors_len, "Number of symbols exceeds survivors buffer");
00480     }
00481     #ifdef UNIT_TEST
00482     if (num_symbols > dec->survivors_len) {
00483         valid = false;
00484     }
00485     #endif
00486     return valid;
00487 }
00488
00489 /**
00490  * @brief Perform Viterbi traceback to extract decoded bits
00491  *
00492  * Works backwards through the trellis from the terminal state to recover
00493  * the maximum likelihood input sequence.
00494  *
00495  */
00496 RX_STATIC_TESTABLE void internal_viterbi_traceback(const rx_fec_decoder_t* dec,
00497                                                    const uint32_t    num_symbols,
00498                                                    const uint32_t    data_bits,
00499                                                    uint8_t*         output,
00500                                                    const uint32_t    output_bytes)
00501 {
00502     if (!internal_viterbi_traceback_validate(dec, num_symbols, output, output_bytes)) {
00503         return;
00504     }
00505
00506     /* Clear output buffer */
00507     for (uint32_t i = k_fec_zero; i < output_bytes; i++) {
00508         output[i] = (uint8_t)k_fec_zero;
00509     }
00510
00511     /* Start from state 0 (encoder is flushed to zero by tail bits) */
00512     uint8_t state = k_fec_zero;
00513
00514     /* Traceback: work backwards through the trellis */
00515     uint32_t limit = num_symbols;
00516     if (num_symbols > k_fec_max_symbols) {
00517         limit = k_fec_max_symbols;
00518     }
00519     for (uint32_t t = limit; t > k_fec_zero; t--) {
00520         const uint32_t t_idx = t - k_fec_bit_mask;
00521
00522         /* The input bit is the MSB of the current state */
00523         const uint8_t input_bit = (uint8_t)((state » k_fec_state_shift_amount) & k_fec_bit_mask);
00524
00525         /* Store decoded bit if it's a data bit (not tail bit) */
00526         if (t_idx < data_bits) {
00527             if (input_bit != k_fec_zero) {
00528                 const uint32_t byte_idx = t_idx / k_rx_bits_per_byte;
00529                 const uint32_t bit_pos = k_fec_msb_bit_position - (t_idx % k_rx_bits_per_byte);
00530                 output[byte_idx] |= (uint8_t)(k_fec_bit_mask « bit_pos);
00531             }
00532         }
00533
00534         /* Get predecessor's LSB from survivors */
00535         const uint8_t predecessor_lsb = (uint8_t)((dec->survivors[t_idx] » state) & k_fec_bit_mask);
00536
00537         /* Compute predecessor state: shift left and insert the LSB */
00538         state = ((state « k_fec_bit_mask) & (k_fec_num_states - k_fec_bit_mask)) | predecessor_lsb;
00539     }
00540 }
00541
00542 /*
=====
00543  * Encoder API Implementation
00544  */

```

```

=====
00545 */
00546
00547 /**
00548  * @brief Initialize FEC encoder
00549  *
00550  *
00551  */
00552 rx_err_t rx_fec_encoder_init(rx_fec_encoder_t* enc)
00553 {
00554     if (enc == nullptr) {
00555         return k_rx_err_invalid_arg;
00556     }
00557     enc->initialized = true;
00558     return k_rx_ok;
00559 }
00560
00561
00562 /**
00563  * @brief Deinitialize FEC encoder
00564  *
00565  *
00566  */
00567 rx_err_t rx_fec_encoder_deinit(rx_fec_encoder_t* enc)
00568 {
00569     if (enc == nullptr) {
00570         return k_rx_err_invalid_arg;
00571     }
00572     enc->initialized = false;
00573     return k_rx_ok;
00574 }
00575
00576
00577 /**
00578  * @brief Compute encoded bit count from raw input length (unchecked)
00579  *
00580  * @details
00581  * Computes the number of output bits for FEC encoding using the formula:
00582  * output_bits = round_up_8((input_len * 8 + tail_bits) * 2 + 7)
00583  *
00584  * With C23 checked arithmetic (RX_HAS_STDCKDINT), any overflow returns 0.
00585  * Without it, unchecked arithmetic is used (caller must ensure no overflow).
00586  *
00587  * This function does NOT validate input_len. It is exposed as RX_STATIC_TESTABLE
00588  * so that unit tests can exercise the overflow paths with values larger than
00589  * k_fec_max_input_bytes (which the public rx_fec_encoded_len() rejects first).
00590  *
00591  *
00592  *
00593  * @pre out_bytes must not be nullptr
00594  * @post *out_bytes == 0 if overflow detected; otherwise rounded-up byte count
00595  *
00596  * @note Not thread-safe (pure computation, no shared state)
00597  * @since Version 1.0.0
00598  */
00599 RX_STATIC_TESTABLE bool internal_fec_compute_encoded_bytes(uint32_t input_len, uint32_t* out_bytes)
00600 {
00601     RX_ASSERT_PRE(out_bytes != nullptr, "out_bytes must not be nullptr");
00602     #ifdef UNIT_TEST
00603     if (out_bytes == nullptr) {
00604         return false;
00605     }
00606     #endif
00607
00608     /* Output bits = (input bits + tail bits) * 2, rounded up to whole bytes */
00609     #if RX_HAS_STDCKDINT
00610     uint32_t input_bits = 0;
00611     if (ckd_mul(&input_bits, input_len, (uint32_t)k_rx_bits_per_byte)) {
00612         *out_bytes = k_fec_zero;
00613         return false;
00614     }
00615
00616     /* Step-2 ckd_add not needed: if step-1 passes, input_bits <= 536870911*8 = 4294967288,
00617      * so input_bits + k_fec_tail_bits (6) <= 4294967294 <= UINT32_MAX, no overflow. */
00618     const uint32_t total_input_bits = input_bits + (uint32_t)k_fec_tail_bits;
00619
00620     uint32_t total_output_bits = 0;
00621     if (ckd_mul(&total_output_bits, total_input_bits, (uint32_t)k_fec_num_outputs)) {
00622         *out_bytes = k_fec_zero;
00623         return false;
00624     }
00625
00626     uint32_t rounded = 0;
00627     if (ckd_add(&rounded, total_output_bits, (uint32_t)k_fec_msb_bit_position)) {
00628         *out_bytes = k_fec_zero;
00629         return false;
00630     }
00631 }

```

```

00631
00632 *out_bytes = rounded / (uint32_t)k_rx_bits_per_byte;
00633 return true;
00634 #else
00635 /* Fallback: caller must ensure input_len <= k_fec_max_input_bytes (1024) */
00636 const uint32_t input_bits = input_len * (uint32_t)k_rx_bits_per_byte;
00637 const uint32_t total_input_bits = input_bits + (uint32_t)k_fec_tail_bits;
00638 const uint32_t total_output_bits = total_input_bits * (uint32_t)k_fec_num_outputs;
00639 const uint32_t rounded = total_output_bits + (uint32_t)k_fec_msb_bit_position;
00640 *out_bytes = rounded / (uint32_t)k_rx_bits_per_byte;
00641 return true;
00642 #endif
00643 }
00644
00645 /**
00646 * @brief Calculate encoded output length for given input
00647 *
00648 * Computes the number of bytes required for FEC-encoded output
00649 * based on input data length. Accounts for tail bits and rate-1/2 encoding.
00650 *
00651 *
00652 *
00653 * @pre input_len must be in range [1, k_fec_max_input_bytes]
00654 * @pre k_fec_max_input_bytes <= 1024 (guarantees no overflow in arithmetic)
00655 * @post Return value is 0 if input_len is 0 or > k_fec_max_input_bytes
00656 * @post Return value > 0 for valid input_len
00657 *
00658 * @note Thread-safe (pure computation, no shared state)
00659 * @since Version 1.0.0
00660 */
00661 uint32_t rx_fec_encoded_len(const uint32_t input_len)
00662 {
00663     if (input_len == k_fec_zero) {
00664         return k_fec_zero;
00665     }
00666     if (input_len > k_fec_max_input_bytes) {
00667         return k_fec_zero;
00668     }
00669 }
00670
00671 /* Validated: input_len in [1, k_fec_max_input_bytes], no overflow possible */
00672 uint32_t result = k_fec_zero;
00673 (void)internal_fec_compute_encoded_bytes(input_len, &result);
00674 return result;
00675 }
00676
00677 /**
00678 * @brief Encode all data bytes into output bit stream
00679 *
00680 * Processes each byte of input MSB-first, producing two output bits per input bit
00681 * via the convolutional encoder.
00682 *
00683 */
00684 static void internal_encode_data_bytes(const uint8_t* input,
00685                                       const uint32_t byte_limit,
00686                                       uint8_t* output,
00687                                       uint8_t* state,
00688                                       uint32_t* out_bit_idx)
00689 {
00690     for (uint32_t byte_idx = k_fec_zero; byte_idx < byte_limit; byte_idx++) {
00691         const uint8_t b = input[byte_idx];
00692         for (int8_t i = (int8_t)k_fec_msb_bit_position; i >= (int8_t)k_fec_zero; i--) {
00693             const uint8_t input_bit = (b >> i) & k_fec_bit_mask;
00694             uint8_t out0;
00695             uint8_t out1;
00696
00697             internal_encode_bit(state, input_bit, &out0, &out1);
00698
00699             internal_set_output_bit(output, (*out_bit_idx)++, out0);
00700             internal_set_output_bit(output, (*out_bit_idx)++, out1);
00701         }
00702     }
00703 }
00704
00705 /**
00706 * @brief Append tail bits to flush encoder shift register to zero state
00707 *
00708 * Appends k_fec_tail_bits zero-input bits to the output stream, ensuring
00709 * the decoder traceback terminates at state 0.
00710 *
00711 */
00712 static void internal_encode_tail_bits(uint8_t* output, uint8_t* state, uint32_t* out_bit_idx)
00713 {
00714     for (uint8_t i = k_fec_zero; i < k_fec_tail_bits; i++) {
00715         uint8_t out0;
00716         uint8_t out1;
00717         internal_encode_bit(state, k_fec_zero, &out0, &out1);

```

```

00718     internal_set_output_bit(output, (*out_bit_idx)++, out0);
00719     internal_set_output_bit(output, (*out_bit_idx)++, out1);
00720 }
00721 }
00722
00723 /**
00724  * @brief Encode data using convolutional FEC
00725  *
00726  * Applies rate-1/2, constraint-length-7 convolutional encoding
00727  * to input data. Output length is approximately 2x input length.
00728  *
00729  *
00730  *
00731  * @pre enc must be initialized via rx_fec_encoder_init()
00732  * @pre All pointers must be non-nullptr
00733  * @pre input_len in range [1, k_fec_max_input_bytes]
00734  *
00735  * @post output buffer contains FEC-encoded data
00736  * @post *output_len set to actual encoded length
00737  */
00738 rx_err_t rx_fec_encode(const rx_fec_encoder_t* enc,
00739                       const uint8_t* input,
00740                       const uint32_t input_len,
00741                       uint8_t* output,
00742                       uint32_t* output_len)
00743 {
00744     if (enc == nullptr || input == nullptr || output == nullptr || output_len == nullptr) {
00745         return k_rx_err_invalid_arg;
00746     }
00747
00748     if (!enc->initialized) {
00749         return k_rx_err_invalid_state;
00750     }
00751
00752     if (input_len == k_fec_zero) {
00753         return k_rx_err_invalid_arg;
00754     }
00755
00756     if (input_len > k_fec_max_input_bytes) {
00757         return k_rx_err_invalid_size;
00758     }
00759
00760     /* Calculate output size and clear output buffer */
00761     const uint32_t expected_output_len = rx_fec_encoded_len(input_len);
00762     for (uint32_t i = k_fec_zero; i < expected_output_len; i++) {
00763         output[i] = (uint8_t)k_fec_zero;
00764     }
00765
00766     /* Encode input bytes then flush with tail bits */
00767     /* input_len validated <= k_fec_max_input_bytes above; use directly. */
00768     uint8_t state = k_fec_zero;
00769     uint32_t out_bit_idx = k_fec_zero;
00770     const uint32_t byte_limit = input_len;
00771
00772     internal_encode_data_bytes(input, byte_limit, output, &state, &out_bit_idx);
00773     internal_encode_tail_bits(output, &state, &out_bit_idx);
00774
00775     *output_len = expected_output_len;
00776     return k_rx_ok;
00777 }
00778
00779 /**
=====
00780  * Decoder API Implementation
00781  *
=====
00782  */
00783
00784 /**
00785  * @brief Validate decode parameters before Viterbi decoding
00786  *
00787  * Pre-conditions:
00788  * - dec, params, num_symbols_out must be non-nullptr
00789  * - dec must be initialized
00790  * - params->soft_bits, output, output_len must be non-nullptr
00791  * - params->soft_len must be non-zero and divisible by 2 (G1, G2 pairs)
00792  *
00793  * Validation checks:
00794  * - Calculated num_symbols from expected_output_len must be valid and within bounds
00795  * - Sufficient soft bits must be available for calculated num_symbols
00796  *   (NOT silently reduced - caller error if params are inconsistent)
00797  * - Survivors buffer must be large enough for num_symbols
00798  *
00799  * Post-condition:
00800  * - If successful, num_symbols_out contains the number of symbols to decode
00801  * - If parameters are invalid or inconsistent, returns appropriate error code
00802  */

```

```

00803 *
00804 */
00805 static rx_err_t internal_validate_decode_params(rx_fec_decoder_t* dec,
00806                                                const rx_fec_decode_soft_params_t* params,
00807                                                uint32_t* num_symbols_out)
00808 {
00809     const bool dec_null = (bool)(dec == nullptr);
00810     const bool params_null = (bool)(params == nullptr);
00811     const bool num_symbols_null = (bool)(num_symbols_out == nullptr);
00812     if ((bool)((int)dec_null | (int)params_null | (int)num_symbols_null)) {
00813         return k_rx_err_invalid_arg;
00814     }
00815
00816     const bool soft_bits_null = (bool)(params->soft_bits == nullptr);
00817     const bool output_null = (bool)(params->output == nullptr);
00818     const bool output_len_null = (bool)(params->output_len == nullptr);
00819     if ((bool)((int)soft_bits_null | (int)output_null | (int)output_len_null)) {
00820         return k_rx_err_invalid_arg;
00821     }
00822
00823     if (!dec->initialized) {
00824         return k_rx_err_invalid_state;
00825     }
00826
00827     /* Soft bits must come in pairs (G1, G2) */
00828     if (params->soft_len == k_fec_zero || (params->soft_len % k_fec_num_outputs) != k_fec_zero) {
00829         return k_rx_err_invalid_arg;
00830     }
00831
00832     const uint32_t num_symbols =
00833         (params->expected_output_len > k_fec_zero)
00834         ? (uint32_t)((params->expected_output_len * k_rx_bits_per_byte) + k_fec_tail_bits)
00835         : k_fec_zero;
00836
00837     if (num_symbols < k_fec_tail_bits) {
00838         return k_rx_err_invalid_size;
00839     }
00840
00841     if (num_symbols > k_fec_max_symbols) {
00842         return k_rx_err_invalid_size;
00843     }
00844
00845     /* Verify sufficient soft bits are available for the calculated num_symbols.
00846     * This is NOT a fallback condition - if the caller provides inconsistent
00847     * expected_output_len and soft_len parameters (e.g., expects to decode 100
00848     * bytes but only provides soft bits for 50 bytes), that is a programming error
00849     * that must be caught and reported, not silently degraded. Graceful degradation
00850     * would hide bugs in parameter preparation. */
00851     if (num_symbols * k_fec_num_outputs > params->soft_len) {
00852         return k_rx_err_invalid_size;
00853     }
00854
00855     /* Check survivors buffer is large enough */
00856     if (num_symbols > dec->survivors_len) {
00857         return k_rx_err_invalid_size;
00858     }
00859
00860     *num_symbols_out = num_symbols;
00861     return k_rx_ok;
00862 }
00863
00864 /**
00865  * @brief Initialize FEC decoder
00866  *
00867  * Initializes decoder state and associates it with provided survivors buffer.
00868  * The buffer is used for Viterbi algorithm path storage.
00869  *
00870  *
00871  *
00872  * @pre dec and survivors_buf must be non-nullptr
00873  * @pre survivors_len must be > 0 (minimum 1 entry per symbol)
00874  *
00875  * @post dec->initialized set to true
00876  * @post Branch table initialized
00877  */
00878 rx_err_t
00879 rx_fec_decoder_init(rx_fec_decoder_t* dec, uint64_t* survivors_buf, const uint32_t survivors_len)
00880 {
00881     if (dec == nullptr || survivors_buf == nullptr) {
00882         return k_rx_err_invalid_arg;
00883     }
00884
00885     /* Minimum survivors buffer size: at least 1 entry per symbol */
00886     if (survivors_len == k_fec_zero) {
00887         return k_rx_err_invalid_size;
00888     }
00889

```

```

00890 dec->survivors    = survivors_buf;
00891 dec->survivors_len = survivors_len;
00892
00893 /* Initialize branch table */
00894 internal_init_branch_table(dec->branch_table);
00895
00896 dec->initialized = true;
00897 return k_rx_ok;
00898 }
00899
00900 /**
00901  * @brief Deinitialize FEC decoder
00902  *
00903  *
00904  */
00905 rx_err_t rx_fec_decoder_deinit(rx_fec_decoder_t* dec)
00906 {
00907     if (dec == nullptr) {
00908         return k_rx_err_invalid_arg;
00909     }
00910
00911     dec->survivors    = nullptr;
00912     dec->survivors_len = k_fec_zero;
00913     dec->initialized   = false;
00914     return k_rx_ok;
00915 }
00916
00917 /**
00918  * @brief Decode FEC-encoded data using soft-decision Viterbi algorithm
00919  *
00920  * @details
00921  * Implements Rate-1/2 Viterbi soft-decision decoding with constraint length 7.
00922  * Accepts soft-decision values (signed integers in range [-128, 127]) to
00923  * achieve approximately 3 dB coding gain over hard-decision decoding.
00924  *
00925  * Algorithm steps:
00926  * 1. Validate parameters via internal_validate_decode_params()
00927  * 2. Forward pass: compute branch and path metrics for all symbols
00928  * 3. Calculate data bit count (num_symbols - tail bits)
00929  * 4. Traceback: extract maximum-likelihood decoded bit sequence
00930  * 5. Pack decoded bits into output byte array
00931  *
00932  *
00933  *
00934  * @pre dec must be initialized via rx_fec_decoder_init()
00935  * @pre params->soft_len must be divisible by 2 (G1/G2 symbol pairs)
00936  * @pre params->expected_output_len must be > 0 and <= k_fec_max_input_bytes
00937  *
00938  * @post *params->output_len written with actual decoded byte count on k_rx_ok
00939  * @post params->output contains decoded data bytes on k_rx_ok
00940  * @post dec path metric arrays updated (internal state consumed by traceback)
00941  *
00942  * @note Not thread-safe; caller must ensure exclusive access to dec and params
00943  * @note Soft values: positive = bit-1 likely, negative = bit-0 likely (signed).
00944  *
00945  * @see rx_fec_decoder_init() Initialize decoder before calling this function
00946  * @see rx_fec_decode_hard() Hard-decision wrapper (lower coding gain, simpler input)
00947  * @see rx_fec_encode() Encoder that produces the data this function decodes
00948  *
00949  * @since Version 1.0.0
00950  *
00951  * @code
00952  * // Example: soft-decision decode of a previously FEC-encoded buffer
00953  * rx_fec_decoder_t dec;
00954  * uint64_t survivors[k_fec_max_symbols];
00955  * rx_soft_bit_t soft_buf[k_fec_max_symbols * k_fec_num_outputs];
00956  * uint8_t out_buf[k_fec_max_input_bytes];
00957  * uint32_t out_len = k_fec_zero;
00958  *
00959  * (void)rx_fec_decoder_init(&dec, survivors, k_fec_max_symbols);
00960  *
00961  * // Populate soft_buf from channel (positive = bit-1 likely)
00962  * rx_fec_decode_soft_params_t p = {
00963  *     .soft_bits    = soft_buf,
00964  *     .soft_len     = encoded_len_bits,
00965  *     .expected_output_len = original_data_len,
00966  *     .output       = out_buf,
00967  *     .output_len   = &out_len,
00968  * };
00969  * rx_err_t err = rx_fec_decode_soft(&dec, &p);
00970  * @endcode
00971  */
00972 rx_err_t rx_fec_decode_soft(rx_fec_decoder_t* dec, const rx_fec_decode_soft_params_t* params)
00973 {
00974     uint32_t num_symbols = k_fec_zero;
00975     const rx_err_t err = internal_validate_decode_params(dec, params, &num_symbols);
00976     if (err != k_rx_ok) {

```

```

00977     return err;
00978 }
00979
00980 /* Forward pass: initialize path metrics and process symbols */
00981 internal_viterbi_forward_pass(dec, params->soft_bits, num_symbols);
00982
00983 /* Calculate data bits and output size */
00984 const uint32_t data_bits = num_symbols - k_fec_tail_bits;
00985
00986 const uint32_t output_bytes = (data_bits + k_fec_msb_bit_position) / k_rx_bits_per_byte;
00987
00988 /* Traceback: extract decoded bits */
00989 internal_viterbi_traceback(dec, num_symbols, data_bits, params->output, output_bytes);
00990
00991 *params->output_len = output_bytes;
00992 return k_rx_ok;
00993 }
00994
00995 /**
00996  * @brief Decode FEC-encoded data using hard-decision bits (converts to soft internally)
00997  *
00998  * @details
00999  * Convenience wrapper that converts hard-decision bits (0/1) to soft-decision
01000  * values and delegates to rx_fec_decode_soft(). Accepts raw bit-packed data
01001  * (MSB-first) as produced by standard digital logic or RSPI transfers.
01002  *
01003  * Algorithm steps:
01004  * 1. Validate all input pointers and size constraints
01005  * 2. Convert each hard bit to a soft value via rx_fec_hard_to_soft()
01006  *    (maps 0 -> negative confidence, 1 -> positive confidence)
01007  * 3. Build rx_fec_decode_soft_params_t from the converted soft bits
01008  * 4. Delegate to rx_fec_decode_soft() for full Viterbi decode
01009  *
01010  *
01011  *
01012  * @pre dec must be initialized via rx_fec_decoder_init()
01013  * @pre params->soft_bits_buffer must be large enough: soft_buffer_len >= data_len * 8
01014  * @pre params->data_len must be > 0 and <= k_fec_max_input_bytes
01015  *
01016  * @post *params->output_len written with actual decoded byte count on k_rx_ok
01017  * @post params->output contains decoded data bytes on k_rx_ok
01018  * @post params->soft_bits_buffer populated with converted soft values (side effect)
01019  *
01020  * @note Not thread-safe; caller must ensure exclusive access to dec and params
01021  * @note Hard-decision decoding has ~3 dB lower coding gain than soft-decision
01022  *
01023  * @see rx_fec_decode_soft() Preferred: use when soft values are available
01024  * @see rx_fec_decoder_init() Initialize decoder before calling this function
01025  * @see rx_fec_hard_to_soft() Conversion function used internally
01026  * @see rx_fec_encode() Encoder that produces data this function decodes
01027  *
01028  * @since Version 1.0.0
01029  *
01030  * @code
01031  * // Example: hard-decision decode of packed FEC-encoded bytes
01032  * rx_fec_decoder_t dec;
01033  * uint64_t survivors[k_fec_max_symbols];
01034  * rx_soft_bit_t soft_scratch[k_fec_max_symbols * k_fec_num_outputs];
01035  * uint8_t out_buf[k_fec_max_input_bytes];
01036  * uint32_t out_len = k_fec_zero;
01037  *
01038  * (void)rx_fec_decoder_init(&dec, survivors, k_fec_max_symbols);
01039  *
01040  * rx_fec_decode_hard_params_t p = {
01041  *     .data = received_bytes,
01042  *     .data_len = received_len,
01043  *     .soft_bits_buffer = soft_scratch,
01044  *     .soft_buffer_len = k_fec_max_symbols * k_fec_num_outputs,
01045  *     .expected_output_len = original_data_len,
01046  *     .output = out_buf,
01047  *     .output_len = &out_len,
01048  * };
01049  * rx_err_t err = rx_fec_decode_hard(&dec, &p);
01050  * @endcode
01051  */
01052 rx_err_t rx_fec_decode_hard(rx_fec_decoder_t* dec, const rx_fec_decode_hard_params_t* params)
01053 {
01054     if (dec == nullptr || params == nullptr) {
01055         return k_rx_err_invalid_arg;
01056     }
01057
01058     if (params->data == nullptr || params->output == nullptr || params->output_len == nullptr ||
01059         params->soft_bits_buffer == nullptr) {
01060         return k_rx_err_invalid_arg;
01061     }
01062
01063     if (!dec->initialized) {

```

```
01064     return k_rx_err_invalid_state;
01065 }
01066
01067 if (params->data_len == k_fec_zero) {
01068     return k_rx_err_invalid_arg;
01069 }
01070
01071 if (params->data_len > k_fec_max_input_bytes) {
01072     return k_rx_err_invalid_size;
01073 }
01074
01075 /* Convert hard bits to soft bits */
01076 const uint32_t num_bits = (uint32_t)(params->data_len * k_rx_bits_per_byte);
01077
01078 /* Ensure soft bits buffer is large enough */
01079 if (num_bits > params->soft_buffer_len) {
01080     return k_rx_err_invalid_size;
01081 }
01082
01083 for (uint32_t i = k_fec_zero; i < num_bits; i++) {
01084     const uint8_t bit = internal_get_bit(params->data, i);
01085     params->soft_bits_buffer[i] = rx_fec_hard_to_soft(bit);
01086 }
01087
01088 /* Prepare soft decode parameters */
01089 const rx_fec_decode_soft_params_t soft_params = {
01090     .soft_bits = params->soft_bits_buffer,
01091     .soft_len = num_bits,
01092     .expected_output_len = params->expected_output_len,
01093     .output = params->output,
01094     .output_len = params->output_len,
01095 };
01096
01097 return rx_fec_decode_soft(dec, &soft_params);
01098 }
```

